# Ontological Studio 튜토리얼 (전체)

> 출처: https://www.uengine.org/docs/ontological-studio/
> 00(소개) + 01~23장 병합본

## 목차


- 00. Ontological Studio 튜토리얼
- 01. 설치와 첫 실행
- 02. 화면 둘러보기
- 03. 온톨로지란 무엇인가
- 04. Ontological Studio의 구성
- 05. 데이터소스 연결하기
- 06. 가상 노드와 가상 관계
- 07. Cypher 기초 — 노드를 찾고 거르기
- 08. 관계 따라가기와 집계
- 09. Cypher 심화 — 실행 들여다보기, 말로 묻기, 직접 쓰기
- 10. 시간 여행 — 바이템포럴 질의
- 11. 골든 퀘스천과 골든 커맨드
- 12. 문서로 온톨로지 구축하기
- 13. 질의하기 — 근거와 함께 답 받기
- 14. 업무 시스템 등록하기
- 15. 골든 커맨드로 액션 만들기
- 16. 액션 실행하기 — 계획·승인·실행·취소
- 17. 대시보드 앱 — 말로 만드는 화면
- 18. 코드 앱 — 에이전트가 코드를 쓰는 앱
- 19. 코드 앱 SDK — 직접 읽고 고치기
- 20. 앱에 액션 버튼 달기
- 21. 종합 과제 — 품질 사고 대응 앱 만들기
- 22. Cypher·패브릭 치트시트
- 23. 자주 묻는 질문과 문제 해결

# Ontological Studio 튜토리얼

*분류: 시작하기 · 원문: https://www.uengine.org/docs/ontological-studio/index.html*

> 온톨로지가 무엇인지부터 시작해, 원천 데이터베이스를 연결하고, Cypher로 데이터를 조회하고, 에이전트와 함께 온톨로지를 구축하고, 업무 시스템에 액션을 실행하고, 그 위에 앱을 만드는 데까지 차근차근 따라 하는 실습서입니다.

▶ 영상: Ontological Studio 튜토리얼 — 영상 3:30

## 이 튜토리얼에서 하는 일

Ontological Studio는 **온톨로지를 구축하고, 질의와 액션을 실행하며, 업무 앱을 만드는** 데스크톱 프로그램입니다. 설치 파일 하나에 그래프 데이터베이스(Ontological DB), 데이터 패브릭, 온톨로지 구축 에이전트, 액션 실행기, 예제 업무 시스템이 모두 들어 있어서, 노트북 한 대로 처음부터 끝까지 실습할 수 있습니다.

이 튜토리얼에서는 한 회사의 **공급망 운영** 사례를 다룹니다. 이 회사의 데이터는 세 시스템에 흩어져 있습니다.

| 시스템 | 데이터 | 예시 |
|---|---|---|
| 구매 ERP (`erp`) | 협력사, 품목, 발주, 발주 품목 | 선전 골드라인 PCB에 PCB 기판 2,000장을 발주했다 |
| 물류 WMS (`wms`) | 입고 선적, 일별 재고 | 그 발주의 선적이 세관에 9일째 묶여 있다 |
| 고객 CRM (`crm`) | 고객, 판매 주문, 클레임 | 도쿄의 고객이 PCB 납땜 불량 클레임을 냈다 |

"3일 넘게 늦은 선적은 어느 협력사의 어느 발주 건인가?"라는 질문에 답하려면 세 시스템의 데이터를 연결해야 합니다. 튜토리얼을 마치면 여러분은 이 질문에 그래프 질의로 답하고, "그 선적을 긴급 처리해 줘"라는 지시를 승인 절차를 거쳐 실제 시스템에 실행하고, 이 모든 것을 한 화면에 모은 업무 앱을 만들 수 있게 됩니다.

## 학습 순서

- 1 · 시작하기 — 설치, 첫 실행, 화면 둘러보기
- 2 · 온톨로지 개념 — 클래스·속성·관계·인스턴스, 앱의 구성
- 3 · 데이터 연결 — 원천 DB를 복사 없이 가상 노드로
- 4 · Cypher 익히기 — MATCH부터 집계·진단·시간 여행까지
- 5 · 온톨로지 구축 — 골든 퀘스천·골든 커맨드, 문서로 구축, 질의
- 6 · 액션 — 시스템 등록, 액션 만들기, 승인·실행·취소
- 7 · 애플리케이션 — 대시보드, 코드 앱, SDK, 액션 버튼
- 8 · 마무리 과제 — 배운 내용을 활용하는 종합 실습

각 장은 **개념 → 따라 하기 → 확인 문제** 순서로 되어 있습니다. 확인 문제의 답은 접혀 있으니, 먼저 스스로 해 본 뒤 펼쳐 보세요.

## 누구를 위한 튜토리얼인가

- 데이터베이스 수업에서 SQL `SELECT`와 `JOIN`을 배워 본 대학생 수준이면 충분합니다.
- 그래프 데이터베이스나 Cypher는 몰라도 됩니다. 4부에서 처음부터 배웁니다.
- 프로그래밍은 7부(코드 앱)에서만 조금 필요합니다. 그것도 에이전트가 코드를 써 주므로, 읽을 수 있으면 됩니다.

## 준비물

| 항목 | 내용 |
|---|---|
| 컴퓨터 | macOS (Apple Silicon M1 이상, 또는 Intel) 또는 Windows 10/11 (x64), 메모리 8GB 권장 |
| 디스크 | 2GB 이상 여유 |
| 코딩 에이전트 | Claude Code 또는 Codex CLI — 설치하고 로그인해 두면 API 키 없이 구축·질의 에이전트가 동작합니다. 없어도 3·4부(데이터 연결, Cypher)는 그대로 실습할 수 있습니다. |
| 네트워크 | 설치 파일을 받을 때와 에이전트(LLM)를 쓸 때만 필요합니다. 데이터베이스는 모두 내 컴퓨터 안에서 돕니다. |

> **[팁] 먼저 영상으로 보고 싶다면**
>
> 제품 소개 페이지에 문서로 구축하기, 골든 커맨드로 액션 만들기, 온톨로지 앱 만들기 시연 영상이 있습니다. 이 튜토리얼은 그 과정을 한 단계씩 풀어 쓴 것입니다.

## 표기 약속

- `MATCH (s:Supplier) RETURN s.name` 처럼 고정폭 글자는 입력하는 문장, 화면의 버튼 이름은 「가상 노드 만들기」처럼 낫표로 씁니다.
- 화면 사진은 낮 테마로 찍었습니다. 일부 시연 장면(밤 테마)은 사진 설명에 따로 적었습니다.
- 숫자와 결과는 모두 예제 데이터로 실제로 실행해 얻은 값입니다. 여러분의 결과가 다르면, 앞 장에서 데이터를 바꾸는 실습(액션 실행 등)을 했는지 확인하세요.

# 설치와 첫 실행

*분류: 시작하기 · 1장 · 원문: https://www.uengine.org/docs/ontological-studio/install.html*

> 설치 파일을 내려받아 실행하고, 첫 실행 과정에서 준비되는 항목을 살펴봅니다. 코딩 에이전트를 연결하는 방법도 다룹니다.

▶ 영상: 설치와 첫 실행 — 영상 3:44

## 설치 파일 받기

릴리스 페이지에서 운영체제에 맞는 파일을 받습니다.

| 운영체제 | 파일 |
|---|---|
| macOS (Apple Silicon) | `Ontological-Studio-mac-arm64.dmg` |
| macOS (Intel) | `Ontological-Studio-mac-x64.dmg` |
| Windows 10/11 (x64) | `Ontological-Studio-Setup-win-x64.exe` |

Mac은 칩에 맞는 파일을 받아야 합니다. 화면 왼쪽 위 사과(Apple) 메뉴 › **이 Mac에 관하여**에서 「칩」이 Apple M1·M2·M3·M4…이면 **Apple Silicon**용, 「프로세서」가 Intel이면 **Intel**용입니다. 칩에 맞지 않는 DMG를 받으면 앱이 실행되지 않습니다.

> **[참고] Intel Mac은 0.1.1부터**
>
> Intel Mac용 설치 파일은 0.1.1부터 있습니다. 앱 안의 데이터베이스와 바깥 DB 커넥터(MySQL·Oracle·SQL Server)까지 모두 Intel용으로 따로 빌드했으므로, 이 튜토리얼의 모든 실습을 Intel Mac에서도 그대로 따라 할 수 있습니다.

## macOS에 설치하기

1. DMG를 열고 **Ontological Studio** 아이콘을 **Applications** 폴더로 끌어다 놓습니다.
2. 응용 프로그램 폴더에서 앱을 더블클릭합니다. “Apple은 ‘Ontological Studio’에 악성 코드가 없음을 확인할 수 없습니다” 창이 뜨면 「완료」를 누릅니다.
3. **시스템 설정** › **개인정보 보호 및 보안**을 열고, 아래쪽 보안 항목에서 「그래도 열기」를 누른 뒤 암호를 넣습니다. 한 번 허용하면 다음부터는 그냥 열립니다.

macOS 14 이하에서는 앱을 오른쪽 클릭 › 열기로 처음 실행해도 됩니다. 이런 확인을 거치는 까닭은 지금 배포판이 Apple 개발자 인증서로 서명·공증되지 않았기 때문입니다.

> **[주의] “손상되었기 때문에 열 수 없습니다”가 나오면**

> 「그래도 열기」 없이 “손상되었기 때문에 열 수 없습니다. 해당 항목을 휴지통으로 이동해야 합니다”만 나온다면, 앱 서명이 덜 된 채 올라갔던 0.1.1 Apple Silicon용 초기 배포본입니다. 휴지통으로 옮기지 말고, 앱을 Applications에 둔 채 터미널에서 다음을 실행한 뒤 다시 엽니다.
>
> ```
> xattr -dr com.apple.quarantine "/Applications/Ontological Studio.app"
> ```
>
> 인터넷에서 받은 파일이라는 표시(격리 속성)만 지우는 명령입니다. 앱 자체는 바뀌지 않습니다.

## Windows에 설치하기

1. EXE를 실행합니다. 관리자 권한 없이 현재 사용자에게만 설치됩니다.
2. SmartScreen이 “Windows의 PC 보호” 창을 띄우면 **추가 정보** › **실행**을 누릅니다.
3. 설치가 끝나면 시작 메뉴에서 Ontological Studio를 엽니다.

> **[참고] Windows에서 아직 안 되는 것**
>
> MySQL·Oracle·SQL Server 원천 연결은 macOS판(Apple Silicon·Intel)에만 있습니다(PostgreSQL은 모든 판에서 됩니다). Claude Code를 Windows에서 쓰려면 Git for Windows가 먼저 설치돼 있어야 합니다.

## 첫 실행에 일어나는 일

앱을 열면 부팅 화면이 단계별 진행 상황을 보여 줍니다.

| 단계 | 하는 일 | 걸리는 시간(Apple Silicon 기준, Intel Mac은 조금 더 걸릴 수 있습니다) |
|---|---|---|
| PostgreSQL | 내 컴퓨터 안에서 데이터베이스 서버를 띄웁니다 | 1초 안쪽 |
| 첫 실행 준비 | 예제 업무 DB(ERP·WMS·CRM)를 만들고 데이터 패브릭에 등록합니다 | 처음 한 번만 |
| Ontological | 그래프 엔진(Ontological DB), Bolt 게이트웨이, 포털을 띄웁니다 | 1초 안쪽 |
| Studio | 구축·질의 화면, 액션 실행기, 예제 업무 시스템 3개를 띄웁니다 | 약 4~5초 |

처음 실행할 때는 예제 온톨로지 「공급망 운영 (예제)」도 함께 만들어 둡니다. 클래스 7개, 관계 6개, 액션 3개, 골든 퀘스천 3개, 골든 커맨드 4개가 들어 있어서, 설치하자마자 질의와 액션을 해 볼 수 있습니다.

모든 서버는 `127.0.0.1`(내 컴퓨터)에서만 접근할 수 있습니다.

## 코딩 에이전트 연결하기

온톨로지 구축, 자연어 질의, 대시보드 구성안, 코드 앱은 **에이전트**가 합니다. Ontological Studio는 다음 순서로 에이전트를 찾습니다.

1. 이 컴퓨터에 설치된 **Claude Code** 또는 **Codex CLI** — CLI의 로그인 정보를 사용하므로 별도의 API 키가 필요 없습니다. 기본값입니다.
2. CLI가 없으면 **내장 에이전트** — 이때는 `studio.env` 파일에 `OPENAI_API_KEY`를 넣어야 대화가 됩니다.

홈 화면 왼쪽 아래에 지금 쓰는 에이전트가 보입니다. 바꾸려면 메뉴 **런타임** › **에이전트**를 고릅니다. 「설정」 카드를 누르면 `studio.env`가 열립니다.

> **[팁] 에이전트 없이도 할 수 있는 것**
>
> 데이터소스 연결, 가상 노드 만들기, 포털에서 Cypher 쓰기는 에이전트 없이 동작합니다. 이 튜토리얼의 3·4부는 에이전트가 없어도 끝까지 따라 할 수 있습니다.

## 데이터는 어디에 저장되나요

| 항목 | 위치 (macOS) |
|---|---|
| 사용자 데이터 전체 | `~/Library/Application Support/Ontological Studio/` |
| 데이터베이스 | 위 폴더의 `pgdata` |
| 비밀번호(설치마다 무작위로 생성) | 위 폴더의 `secrets.json` |
| 로그 | 위 폴더의 `logs/` — 홈 화면 「로그 폴더」로 바로 열 수 있습니다 |

처음 상태로 되돌리고 싶으면 앱을 끈 뒤 이 폴더를 지우고 다시 실행합니다. 예제 DB와 예제 온톨로지가 새로 만들어집니다.

> **[확인 문제] 확인**
>
> 앱을 실행해 홈 화면이 뜨는지 확인하고, 왼쪽 아래에 「Ontological 연결됨」과 에이전트 이름이 나오는지 보세요.

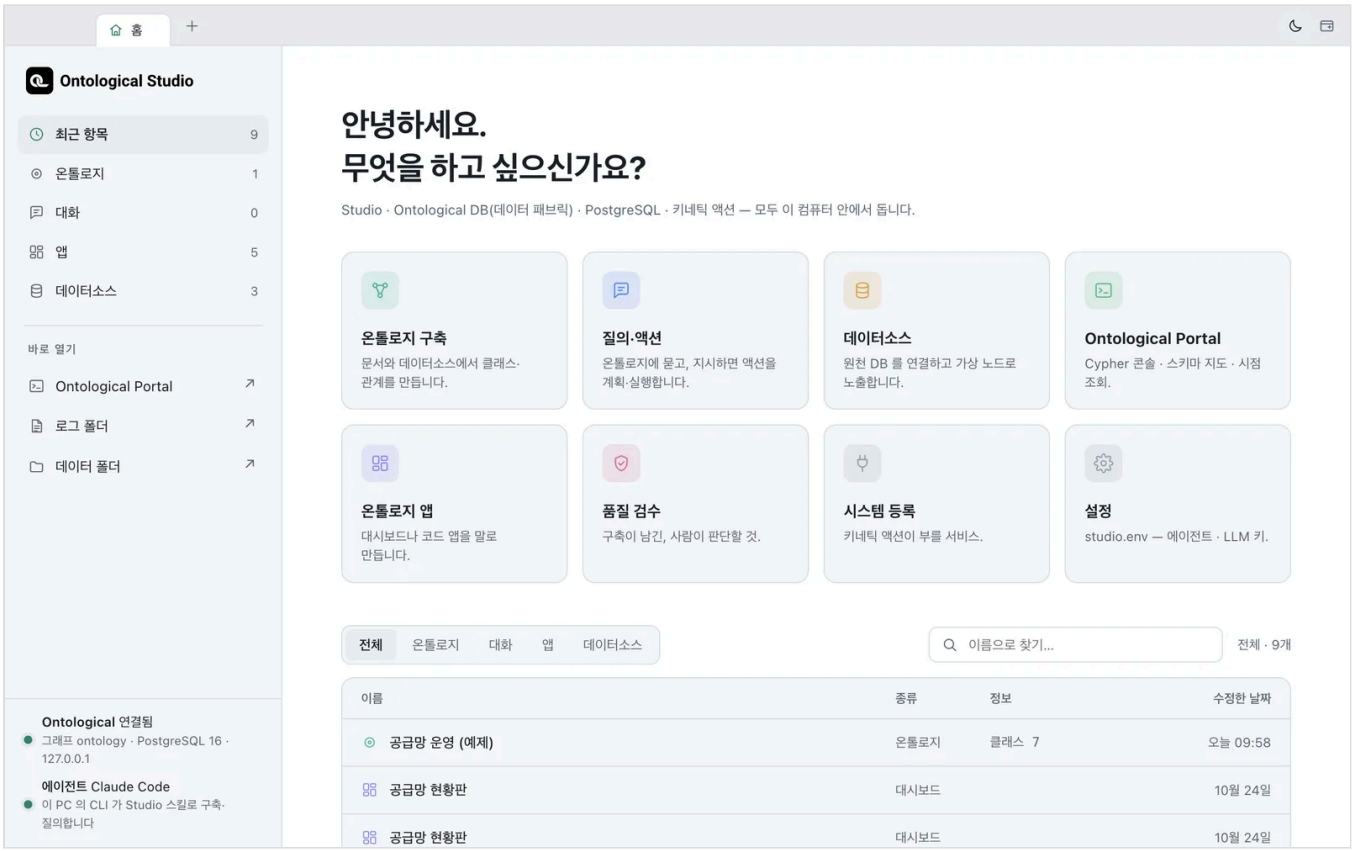
# 화면 둘러보기

*분류: 시작하기 · 2장 · 원문: https://www.uengine.org/docs/ontological-studio/tour.html*

> 홈 화면의 각 기능을 살펴보고 탭과 창을 사용하는 방법을 익힙니다. 이후 실습은 홈 화면에서 시작합니다.

▶ 영상: 화면 둘러보기 — 영상 2:55

## 홈 — 무엇을 하고 싶으신가요?




*홈 화면 — 여덟 개의 카드와 최근 항목. 왼쪽 아래에 Ontological 연결 상태와 에이전트가 보입니다.*

카드 하나가 기능 하나입니다. 카드를 누르면 **새 탭**으로 열립니다.

| 카드 | 하는 일 | 다루는 장 |
|---|---|---|
| 온톨로지 구축 | 문서·데이터소스·시스템을 근거로 에이전트가 클래스와 관계를 설계하고 채웁니다 | 11장~12장 |
| 질의·액션 | 온톨로지에 말로 묻고, 지시하면 액션을 계획·승인·실행합니다 | 13장, 16장 |
| 데이터소스 | 원천 DB를 연결하고 테이블을 가상 노드로 노출합니다 | 5장~6장 |
| Ontological Portal | Cypher 콘솔, 스키마 지도, 시간 여행 | 7장~10장 |
| 온톨로지 앱 | 대시보드나 코드 앱을 말로 만듭니다 | 17장~20장 |
| 품질 검수 | 구축이 남긴, 사람이 판단할 항목을 확인합니다 | 12장 |

| 카드 | 하는 일 | 다루는 장 |
|---|---|---|
| 시스템 등록 | 액션이 호출할 업무 시스템(마이크로서비스)을 등록합니다 | 14장 |
| 설정 | `studio.env` — 에이전트와 LLM 키 | 1장 |

아래쪽 **최근 항목** 목록에는 온톨로지·대화·앱·데이터소스가 표시됩니다. 행을 누르면 그 대상이 바로 열립니다. 위의 「전체 · 온톨로지 · 대화 · 앱 · 데이터소스」로 거르고, 오른쪽 칸으로 이름을 찾습니다.

## 탭과 창

- 탭 줄이 창의 제목 표시줄입니다. 첫 탭은 언제나 홈입니다.
- 각 탭은 서로 독립적이라, 구축 탭에서 에이전트가 일하는 동안 포털 탭에서 질의를 해 볼 수 있습니다.
- 탭을 오른쪽 클릭하고 「새 창으로 분리」를 고르면 다른 창으로 떼어 낼 수 있습니다. 화면이 두 개라면 편리합니다.

| 단축키 (Windows는 ⌘ 대신 Ctrl) | 동작 |
|---|---|
| ⌘T | 홈 탭으로 |
| ⌘W | 탭 닫기 |
| ⌘1 · ⌘2~8 · ⌘9 | 홈 · n번째 탭 · 마지막 탭 |
| ^Tab / ^⇧Tab | 다음 / 이전 탭 |
| ⌘R | 탭 새로 고침 |
| ⌘N | 새 창 |

## 낮과 밤

탭 표시줄 오른쪽의 해/달 버튼으로 테마를 바꿉니다. 열린 모든 탭과 대시보드·코드 앱이 함께 바뀝니다. 포털은 원래 밤 테마 전용이라 낮 테마에서도 어둡게 보입니다.

## 왼쪽 레일

Studio 탭 안의 왼쪽 세로 줄(레일)에도 같은 기능들이 아이콘으로 있습니다. 한 탭 안에서 화면을 바꿀 때는 레일을, 나란히 띄우고 싶을 때는 홈의 카드를 쓰면 됩니다.

> **[확인 문제]**
>
> 1. 홈에서 「데이터소스」 카드와 「Ontological Portal」 카드를 차례로 눌러 탭 두 개를 여세요.
> 2. 포털 탭을 새 창으로 분리했다가, 창을 닫아 보세요.
> 3. 테마를 밤으로 바꾸고, 열린 탭이 모두 함께 바뀌는지 확인하세요.
>
> **▼ 도움말**
>
> 창을 닫으면 그 안의 탭도 닫히지만, 앱의 데이터는 그대로 남습니다. 홈 탭에서 ⌘W를 누르면 창 전체가 닫히고, 창을 모두 닫으면 앱이 종료됩니다.

# 온톨로지란 무엇인가

*분류: 온톨로지 개념 · 3장 · 원문: https://www.uengine.org/docs/ontological-studio/ontology-basics.html*

> 클래스·속성·관계·인스턴스라는 네 가지 핵심 개념을 살펴보고, 테이블로 데이터를 다루는 방식과 온톨로지를 예제로 비교합니다.

▶ 영상: 온톨로지란 무엇인가 — 영상 3:56

## 한 문장으로

**온톨로지는 업무에 필요한 개념과 그 개념 사이의 관계를 정의한 모델입니다.**

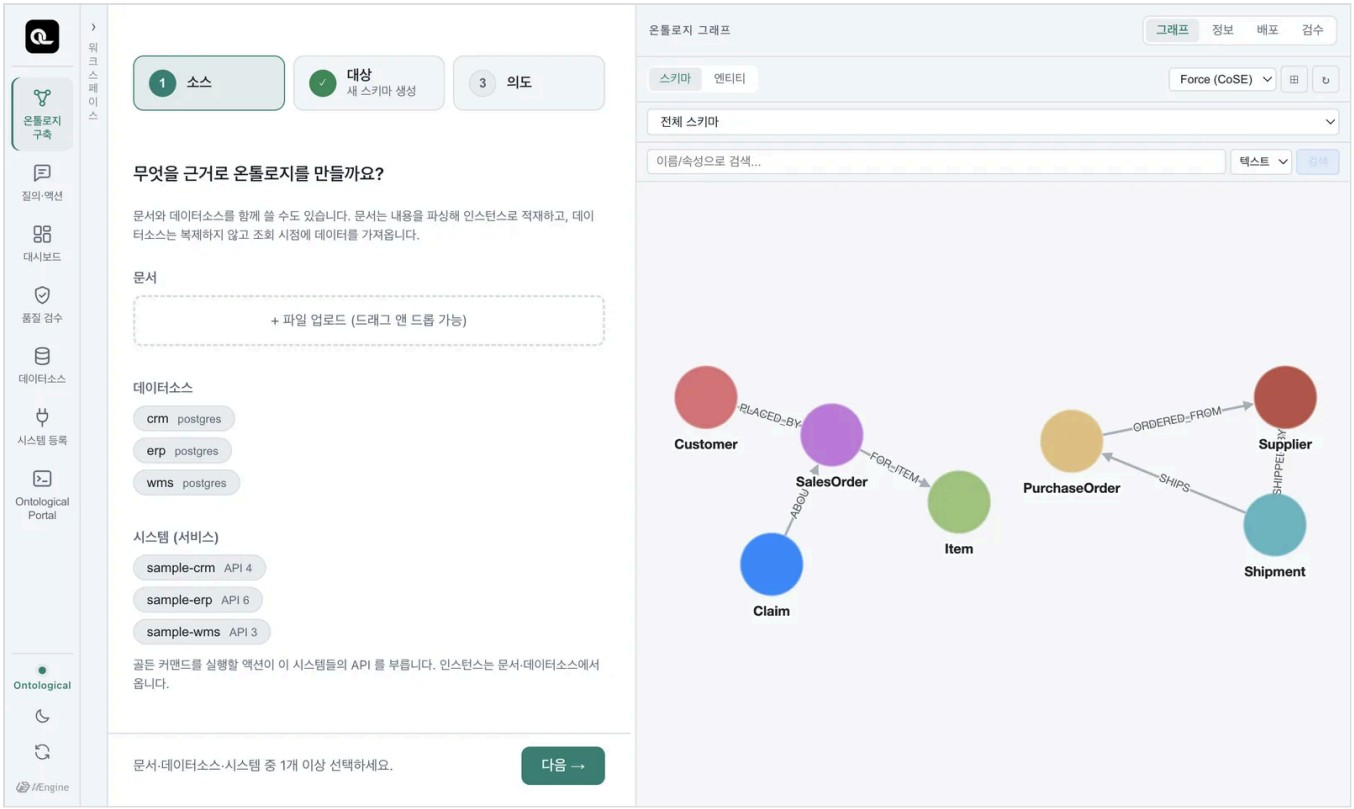
ER 다이어그램처럼 업무의 대상과 관계를 표현합니다. 여기에 실제 데이터를 연결해 **질의와 업무 실행의 기준**으로 활용할 수 있습니다.

## 네 가지 핵심 개념

| 개념 | 설명 | 예시 |
|---|---|---|
| **클래스** (class, 노드 타입·라벨) | 업무에 등장하는 것의 종류 | `Supplier` (협력사), `PurchaseOrder` (발주), `Shipment` (선적) |
| **속성** (property) | 클래스가 가진 값 | `Supplier.name`, `Shipment.delay_days` |
| **관계** (relationship) | 두 클래스 사이의 연결, 방향과 이름이 있음 | `(PurchaseOrder)-[:ORDERED_FROM]->(Supplier)` — 발주는 협력사에게 낸다 |
| **인스턴스** (instance, 노드) | 클래스에 속한 개별 데이터 | 협력사 `S-CN-01 Shenzhen Goldline PCB`, 발주 `PO-002` |

클래스와 관계가 **스키마**, 인스턴스와 그 연결이 **데이터**입니다. 그래프 데이터베이스에서는 인스턴스를 **노드**, 연결을 **엣지(관계)**라고 부릅니다.

## 예제 온톨로지 한눈에 보기




*예제 온톨로지 「공급망 운영 (예제)」의 스키마 — 클래스 7개와 그 사이의 관계*

| 클래스 | 설명 | 원천 시스템 |
|---|---|---|
| `Supplier` | 협력사 (5곳) | ERP |
| `Item` | 품목 (4개) | ERP |
| `PurchaseOrder` | 구매 발주 (8건) | ERP |
| `Shipment` | 입고 선적 (8건) | WMS |
| `Customer` | 고객 (3곳) | CRM |
| `SalesOrder` | 판매 주문 (5건) | CRM |
| `Claim` | 고객 클레임 (3건) | CRM |

| 관계 | 읽는 법 |
|---|---|
| `(Shipment)-[:SHIPS]->(PurchaseOrder)` | 선적은 발주를 싣고 온다 |
| `(PurchaseOrder)-[:ORDERED_FROM]->(Supplier)` | 발주는 협력사에게 냈다 |
| `(SalesOrder)-[:PLACED_BY]->(Customer)` | 판매 주문은 고객이 냈다 |
| `(SalesOrder)-[:FOR_ITEM]->(Item)` | 판매 주문은 품목에 대한 것이다 |
| `(Claim)-[:ABOUT]->(SalesOrder)` | 클레임은 판매 주문에 대한 것이다 |
| `(Shipment)-[:SHIPPED_BY]->(Supplier)` | 선적은 협력사가 보냈다 (구축된 온톨로지에만 있습니다 — 4장 참고) |

## 테이블과 무엇이 다른가

같은 질문을 SQL과 그래프로 비교해 봅시다. **"3일 넘게 늦은 선적은 어느 협력사의 어느 발주 건인가?"**

SQL로는 세 테이블을 조인합니다. 게다가 선적은 WMS에, 발주와 협력사는 ERP에 있으니, 원래는 두 DB를 하나로 모으는 작업부터 해야 합니다.

```
SELECT sh.shipment_no, po.po_id, s.name
FROM wms.shipment sh
JOIN erp.purchase_order po ON po.po_id = sh.ref_po_no
JOIN erp.supplier s       ON s.supplier_id = po.supplier_id
WHERE sh.delay_days > 3;
```

Cypher(그래프 질의 언어)로는 **관계 이름을 사용해 연결된 데이터를 조회합니다.**

```
MATCH (sh:Shipment)-[:SHIPS]->(po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier)
WHERE sh.delay_days > 3
RETURN sh.shipment_no, po.po_id, s.name
```

조인 조건(`po.po_id = sh.ref_po_no`)이 사라지고 `SHIPS`, `ORDERED_FROM`이라는 **업무에서 사용하는 관계 이름**이 남았습니다. 조인 키를 아는 일은 관계를 정의할 때 한 번만 하면 되고, 그다음부터는 사람도 AI 에이전트도 관계 이름만 알면 됩니다.

> **[참고] 온톨로지의 가치는 '다음 질문'에 있습니다**
>
> 질문마다 SQL 뷰를 만들면 질문 수만큼 유지보수할 것이 늘어납니다. 엔티티와 관계로 모델링해 두면, 처음 받는 질문도 **이미 있는 관계를 따라** 답할 수 있습니다. "그 협력사의 다른 미결 발주는?", "그 발주의 품목을 산 고객 중 클레임을 낸 곳은?" 같은 질문에도 새로 설계하지 않고 답할 수 있습니다.

## 왜 AI 에이전트에게 온톨로지가 필요한가

LLM 에이전트에게 테이블 수백 개의 DDL을 주면, `TB507.C041` 같은 코드명이 무엇인지, 어느 컬럼으로 조인해야 하는지를 매번 추측해야 합니다. 온톨로지는 그 추측을 **한 번, 사람이 검수한 형태로** 고정해 둡니다.

1. **의미** — `Shipment.delay_days`는 "도착 예정일보다 늦어진 일수"라는 설명과 함께 있습니다.
2. **경로** — 선적에서 협력사로 가는 길이 관계로 정해져 있습니다.
3. **행위** — 선적에 할 수 있는 일("긴급 처리")과 그 일을 되돌리는 방법이 **액션**으로 붙어 있습니다(6부).

그래서 Ontological Studio에서는 같은 온톨로지가 **질의의 기준, 실행의 기준, 앱의 기준**이 됩니다.

> **[확인 문제]**
>
> 1. 예제에서 `Claim`과 `Customer`를 연결하려면 어떤 관계를 어떤 방향으로 거쳐야 하나요?
> 2. "PCB 기판을 산 고객"을 찾으려면 어떤 클래스들을 지나야 하나요?
> 3. 다음 중 클래스, 속성, 관계, 인스턴스는 각각 무엇인가요? — `Item`, `PO-005`, `FOR_ITEM`, `safety_stock`

▼ 정답 보기

1. `(Claim)-[:ABOUT]->(SalesOrder)-[:PLACED_BY]->(Customer)` — ABOUT을 따라 판매 주문으로, PLACED_BY를 따라 고객으로 갑니다.
2. `Item` ← `FOR_ITEM` — `SalesOrder` — `PLACED_BY` → `Customer`
3. `Item` 클래스, `PO-005` 인스턴스(발주 하나), `FOR_ITEM` 관계, `safety_stock` 속성(품목의 안전재고)

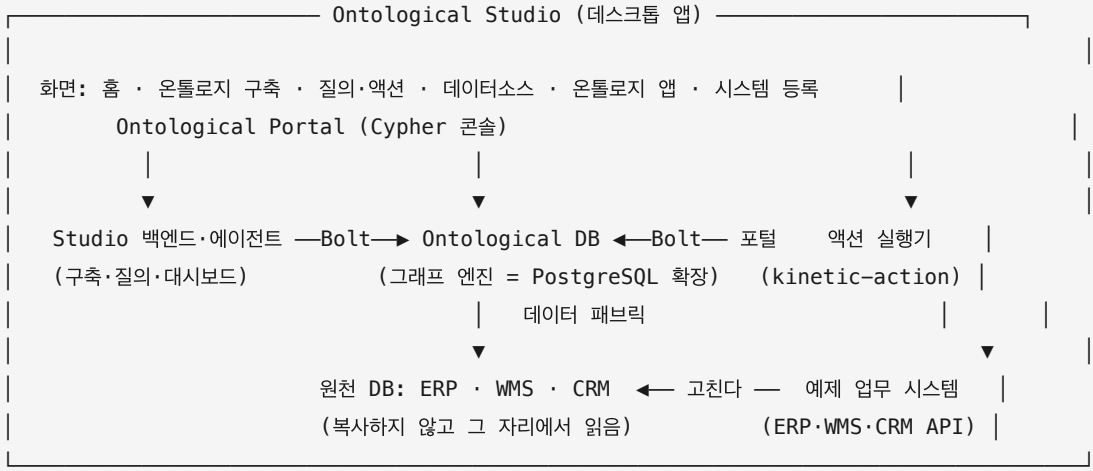
# Ontological Studio의 구성

*분류: 온톨로지 개념 · 4장 · 원문: https://www.uengine.org/docs/ontological-studio/architecture.html*

> 앱을 구성하는 각 요소의 역할과 데이터 흐름을 살펴봅니다. 이후 실습에서 어떤 구성 요소를 사용하는지 이해하는 데 도움이 됩니다.

▶ 영상: Ontological Studio의 구성 — 영상 3:54

## 구성도




| 구성 요소 | 역할 |
|---|---|
| **Ontological DB** | 온톨로지를 저장하고 Cypher를 실행하는 그래프 데이터베이스입니다. PostgreSQL의 확장이라, 그래프 질의가 PostgreSQL의 SQL로 번역되어 실행됩니다. 바이템포럴(시간 여행), 권한, 스키마 진화도 여기서 처리합니다. |
| **데이터 패브릭** | Ontological DB 안의 기능입니다. 다른 데이터베이스의 테이블을 **복사하지 않고** 그래프의 노드처럼 보이게 합니다(가상 노드). |
| **Bolt 게이트웨이** | Neo4j가 사용하는 Bolt 프로토콜을 지원합니다. Studio와 포털은 이를 통해 그래프를 읽고 씁니다. |
| **Studio** | 온톨로지 구축·질의·대시보드 화면과 그 뒤의 에이전트입니다. |
| **액션 실행기** | 액션을 계획하고, 정책을 판정하고, 사람의 승인을 받아 업무 시스템 API를 부르고, 취소하면 역순으로 되돌립니다. |
| **예제 업무 시스템** | 구매 ERP·물류 WMS·고객 CRM. 각자 다른 인증 방식을 쓰는 작은 REST 서비스입니다. 액션이 이들을 부르면, 원천 DB의 값이 바뀝니다. |

## 그래프가 두 개 있습니다

Ontological DB 안에는 여러 **그래프**(독립된 데이터 공간)를 둘 수 있습니다. 처음 설치하면 두 개가 보입니다.

| 그래프 | 포함된 데이터 | 주요 사용처 |
|---|---|---|
| `sample` | 예제 원천 DB의 테이블을 그대로 노출한 **가상 노드와 가상 관계** | 데이터소스 탭, 포털의 Cypher 실습(4부) |
| `ontology` | Studio가 구축한 **온톨로지**: 클래스 정의, 인스턴스, 관계, 액션 정의 | 질의·액션, 온톨로지 앱(5~7부) |

`sample`은 "원천을 있는 그대로 비춘 거울"이고, `ontology`는 "업무 관점으로 정리한 지식"입니다. 예를 들어 `ontology`에는 원천에 없는 `SHIPPED_BY`(선적→협력사) 관계가 있고, 클래스와 속성마다 설명이 붙어 있으며, 액션이 달려 있습니다.

> **[참고] 왜 나눠 두나요?**
>
> 원천의 테이블 이름과 온톨로지의 클래스 이름이 같은 일이 많습니다(`Supplier`). 둘을 한 그래프에 두면 읽기 전용 가상 노드와 구축한 노드가 부딪힙니다. 그래서 예제는 가상 노드를 `sample` 그래프에 따로 둡니다.

## 질문 하나가 지나가는 길

"3일 넘게 늦은 선적은?"이라는 질문이 질의·액션 탭에 들어오면:

1. 에이전트가 발행된 온톨로지의 스키마를 읽습니다(어떤 클래스·관계·속성이 있나).
2. Cypher를 만들어 Bolt로 Ontological DB에 보냅니다.
3. Ontological DB가 Cypher를 SQL로 번역해 실행합니다. 데이터 패브릭은 필요한 조건을 원천 DB에 전달합니다.
4. 결과와 함께 "어느 노드의 어떤 사실을 근거로 삼았는지"를 답에 적습니다.

## 지시 하나가 지나가는 길

"그 선적을 긴급 처리해 줘"라는 지시는 다르게 흐릅니다.

1. 에이전트가 이 지시에 맞는 **액션**(`Shipment.expediteShipment`)을 찾습니다.
2. 액션 실행기에 **계획**을 요청합니다. 실행기는 대상 선적을 고르고, 대상마다 **정책**으로 허용·거부를 판정합니다. 아직 아무것도 바뀌지 않습니다.
3. **사람이 계획 카드에서 승인**합니다.
4. 실행기가 업무 시스템 API를 차례로 부릅니다(WMS 우선순위 변경 → ERP 납기 독촉).
5. 원천 DB 값이 바뀌고, 데이터 패브릭이 그것을 그대로 읽으므로 그래프에서도 바로 보입니다.
6. "취소해 줘"라고 하면 성공한 단계를 **역순으로 되돌립니다**(보상).

에이전트도, 앱도 업무 시스템을 직접 부르지 않습니다. 언제나 이 실행기를 거칩니다.

> **[확인 문제]**
>
> 1. 포털에서 `MATCH (s:Supplier) RETURN s`를 실행했는데, 그래프 선택이 `ontology`일 때와 `sample`일 때 결과가 어디서 오는지 각각 설명해 보세요.
> 2. 액션을 실행한 결과를 확인하려면 업무 시스템에 따로 로그인해야 할까요?
>
> **▼ 정답 보기**

> 1. `sample` 이면 ERP의 `erp.supplier` 테이블을 그 자리에서 읽은 가상 노드가, `ontology` 면 구축 때 온톨로지에 적재된 `Supplier` 인스턴스가 나옵니다.
> 2. 아니요. 업무 시스템은 원천 DB를 변경하고, 데이터 패브릭은 그 원천을 질의할 때마다 읽으므로 데이터소스 탭이나 포털( `sample` 그래프)에서 바로 확인할 수 있습니다.

# 데이터소스 연결하기

*분류: 데이터 연결 · 5장 · 원문: https://www.uengine.org/docs/ontological-studio/datasource.html*

> 원천 데이터베이스를 연결하고, 테이블과 컬럼을 살펴보고, 데이터를 복사하지 않고 미리보기까지 해 봅니다. 실습을 통해 데이터 패브릭의 동작 방식을 익힙니다.

▶ 영상: 데이터소스 연결하기 — 영상 3:56

## 데이터 패브릭이란

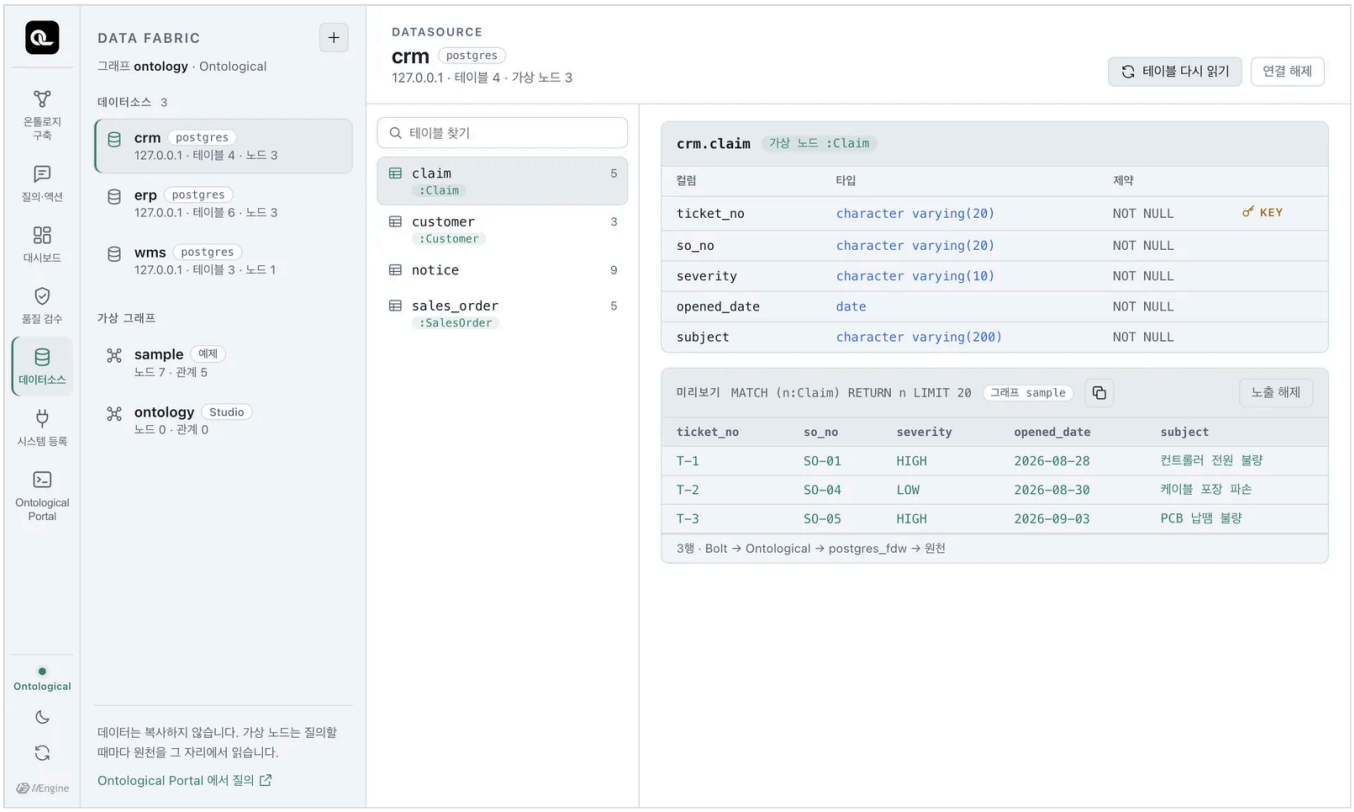
회사의 데이터는 여러 데이터베이스에 흩어져 있습니다. 보통은 이것을 데이터 웨어하우스로 **복사**(ETL)해 한곳에 모은 뒤 분석합니다. 복사본은 원본보다 늦고, 권한을 두 군데서 관리해야 하며, 원본이 바뀌면 다시 복사해야 합니다.

**데이터 패브릭**은 데이터를 원천에 둔 채 질의할 때마다 읽어 옵니다. Ontological DB는 원천 테이블을 그래프의 노드처럼 조회할 수 있도록 **가상 노드**를 정의합니다. 데이터 자체를 복사하지 않고 노드와 관계의 정의를 사용합니다.

| | 복사(ETL) | 데이터 패브릭 |
|---|---|---|
| 데이터 위치 | 웨어하우스에 사본 | 원천 그대로 |
| 신선도 | 마지막 적재 시점 | 질의하는 순간 |
| 권한 | 원천과 사본 두 곳 | 원천 한 곳 |
| 쓰기 | 사본에 써도 원천은 그대로 | 가상 노드는 읽기 전용, 쓰기는 원천 시스템으로 |

## 데이터소스 탭 열기

홈에서 「데이터소스」 카드를 누릅니다.



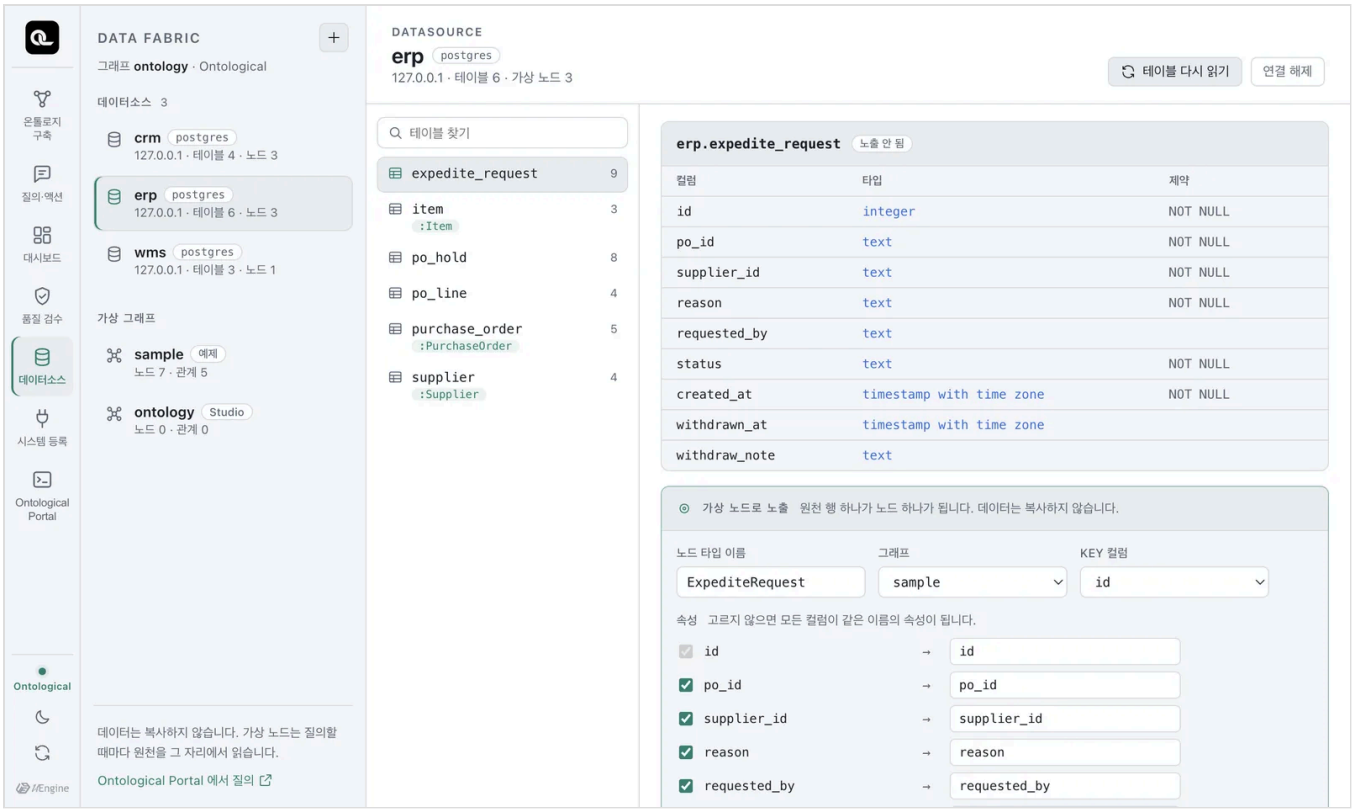

*데이터소스 탭 — 왼쪽에 데이터소스 3개(crm·erp·wms)와 가상 그래프, 가운데 테이블 목록, 오른쪽 테이블 상세와 미리보기*

| 영역 | 내용 |
|---|---|
| 왼쪽 위 **데이터소스** | 연결된 원천 DB. 예제로 `crm`, `erp`, `wms` 세 개가 이미 연결돼 있습니다. |
| 왼쪽 아래 **가상 그래프** | 가상 노드가 속한 그래프. 예제는 `sample` 입니다. 구축으로 만든 저장 그래프(`ontology` 등)도 함께 보이지만 가상 노드가 없으니 “노드 0”으로 표시됩니다. |
| 가운데 | 고른 데이터소스의 테이블. `:Claim` 처럼 초록 라벨이 붙은 테이블은 이미 가상 노드로 노출된 것입니다. |
| 오른쪽 | 테이블의 컬럼·타입·제약과, 가상 노드라면 미리보기 20행 |

미리보기 아래의 `3행 · Bolt → Ontological → postgres_fdw → 원천` 이라는 줄이 이 장의 핵심입니다. 이 행들은 어딘가에 복사된 것이 아니라, **지금 원천 `crm.claim` 테이블에서 읽어 온 것**입니다.

## 테이블 살펴보기

왼쪽에서 `erp` 를 누르면 ERP의 테이블 6개가 나옵니다.




*erp 데이터소스 — 테이블 6개 중 item·purchase_order·supplier가 가상 노드로 노출돼 있습니다*

예제 원천의 테이블을 정리하면 다음과 같습니다.

| 데이터소스 | 테이블 | 키 | 주요 컬럼 | 가상 노드 |
|---|---|---|---|---|
| `erp` | `supplier` | `supplier_id` | name, country, tier | `Supplier` |
| | `item` | `item_id` | name, safety_stock | `Item` |
| | `purchase_order` | `po_id` | supplier_id, order_date, amount_usd, status | `PurchaseOrder` |
| | `po_line` | `po_line_id` | po_id, item_id, qty | (없음 — 6장에서 만듭니다) |
| | `po_hold`, `expedite_request` | | 액션이 남기는 기록(보류, 납기 독촉) | |
| `wms` | `shipment` | `shipment_no` | ref_po_no, vendor_code, ship_status, delay_days, eta, priority | `Shipment` |
| | `stock_daily` | (날짜, 품목, 창고) | on_hand_qty | |
| | `priority_change` | | 액션이 남기는 기록(우선순위 변경 이력) | |
| `crm` | `customer` | `cust_no` | name, country | `Customer` |
| | `sales_order` | `so_no` | cust_no, sku_code, qty, status | `SalesOrder` |
| | `claim` | `ticket_no` | so_no, severity, opened_date, subject | `Claim` |
| | `notice` | | 액션이 남기는 기록(고객 안내) | |

> **[참고] 컬럼 이름이 시스템마다 다릅니다**
>
> 같은 "발주 번호"가 ERP에서는 `po_id`, WMS에서는 `ref_po_no` 입니다. 같은 "품목"이 ERP에서는 `item_id`, CRM에서는 `sku_code` 입니다. 실제 회사에서도 흔한 일이고, 이 차이를 관계로 이어 주는 것이 온톨로지의 역할입니다.

## 새 데이터소스 연결하기

왼쪽 위 「+」를 누르면 연결 폼이 열립니다. 엔진을 고르면 그 엔진에 필요한 칸이 나옵니다.

| 엔진 | 필요한 값 | 비고 |
|---|---|---|
| PostgreSQL | 호스트·포트·DB·스키마·계정·비밀번호 | macOS·Windows |
| MySQL / MariaDB | 호스트·포트·DB·계정·비밀번호 | macOS (Apple Silicon·Intel) |
| Oracle | 호스트·포트·서비스 이름·소유자 스키마·계정·비밀번호 | macOS (Apple Silicon·Intel) |
| SQL Server | 호스트·포트·DB·스키마(기본 dbo)·계정·비밀번호 | macOS (Apple Silicon·Intel) |

데이터소스 이름은 영문 소문자·숫자·밑줄만 씁니다. 원천은 같은 PC든 사내망이든 인터넷이든 접속만 되면 됩니다. 원천 계정은 **읽기 권한만 있는 계정**을 쓰기를 권합니다.

화면에서 실행할 문장을 미리 확인할 수 있습니다. 예를 들어 PostgreSQL을 연결하면 실제로는 다음 문장이 실행됩니다(비밀번호는 화면에 `'••••'`로만 보입니다).

```
CREATE DATASOURCE mydb WITH ENGINE = 'postgres',
  PARAMETERS = {host: '10.0.0.5', port: 5432, database: 'sales', schema: 'public',
                user: 'reader', password: '••••'}
```

> **[팁] 학교 실습 DB를 연결해 보세요**
>
> 수업에서 쓰는 PostgreSQL이나 MySQL이 있다면 연결해 보세요. 연결하는 순간 테이블 목록과 컬럼 타입이 카탈로그로 들어옵니다. 데이터는 한 행도 복사되지 않습니다.

## 화면 뒤의 문장들

데이터소스 탭은 모든 작업을 Ontological DB의 **패브릭 문장**으로 합니다. 화면 아래 「실행한 문장」에서 실행 내역을 확인할 수 있습니다. 같은 문장을 포털에서 직접 입력해도 됩니다.

| 문장 | 하는 일 |
|---|---|
| `SHOW DATASOURCES` | 연결된 데이터소스 목록 |
| `SHOW TABLES FROM erp` | 데이터소스의 테이블과, 가상 노드로 노출됐는지 |
| `SHOW COLUMNS FROM erp.po_line` | 컬럼·타입·NULL 여부·키 |
| `REFRESH DATASOURCE erp` | 원천에 테이블이 늘었을 때 다시 읽기 |

| 문장 | 하는 일 |
|---|---|
| `DROP DATASOURCE erp` | 연결 해제 (가상 노드가 쓰고 있으면 엔진이 거절합니다) |

포털에서 `SHOW TABLES FROM erp`를 실행하면 이렇게 나옵니다.

| table | columns | virtual_node |
|---|---|---|
| erp.expedite_request | 9 | |
| erp.item | 3 | Item |
| erp.po_hold | 8 | |
| erp.po_line | 4 | |
| erp.purchase_order | 5 | PurchaseOrder |
| erp.supplier | 4 | Supplier |

> **[확인 문제]**
>
> 1. 데이터소스 탭에서 `wms`의 `shipment` 테이블을 열고, 미리보기에서 지연일(`delay_days`)이 가장 큰 선적을 찾으세요.
> 2. `crm.claim` 미리보기에는 몇 행이 있나요? 그 행들은 어디에 저장돼 있나요?
> 3. `SHOW COLUMNS FROM wms.shipment`가 반환할 컬럼 이름을 예상해 보고, 포털에서 확인하세요.
>
> **▼ 정답 보기**
>
> 1. `SHP-03` (9일, CUSTOMS_HOLD — 세관 보류).
> 2. 3행(T-1, T-2, T-3). Ontological DB가 아니라 원천 `crm` 스키마의 `claim` 테이블에 있습니다. 미리보기는 질의하는 순간 원천에서 읽어 온 것입니다.
> 3. `shipment_no`, `ref_po_no`, `vendor_code`, `ship_status`, `delay_days`, `eta`, `priority`. (`priority`는 액션이 고치는 컬럼입니다.)

# 가상 노드와 가상 관계

*분류: 데이터 연결 · 6장 · 원문: https://www.uengine.org/docs/ontological-studio/virtual-graph.html*

> 아직 노출되지 않은 테이블 `erp.po_line`(발주 품목)을 가상 노드로 만들고, 발주·품목과 관계로 이어 봅니다. 서로 다른 데이터베이스의 테이블을 하나의 그래프로 연결하는 과정을 실습합니다.

▶ 영상: 가상 노드와 가상 관계 — 영상 3:50

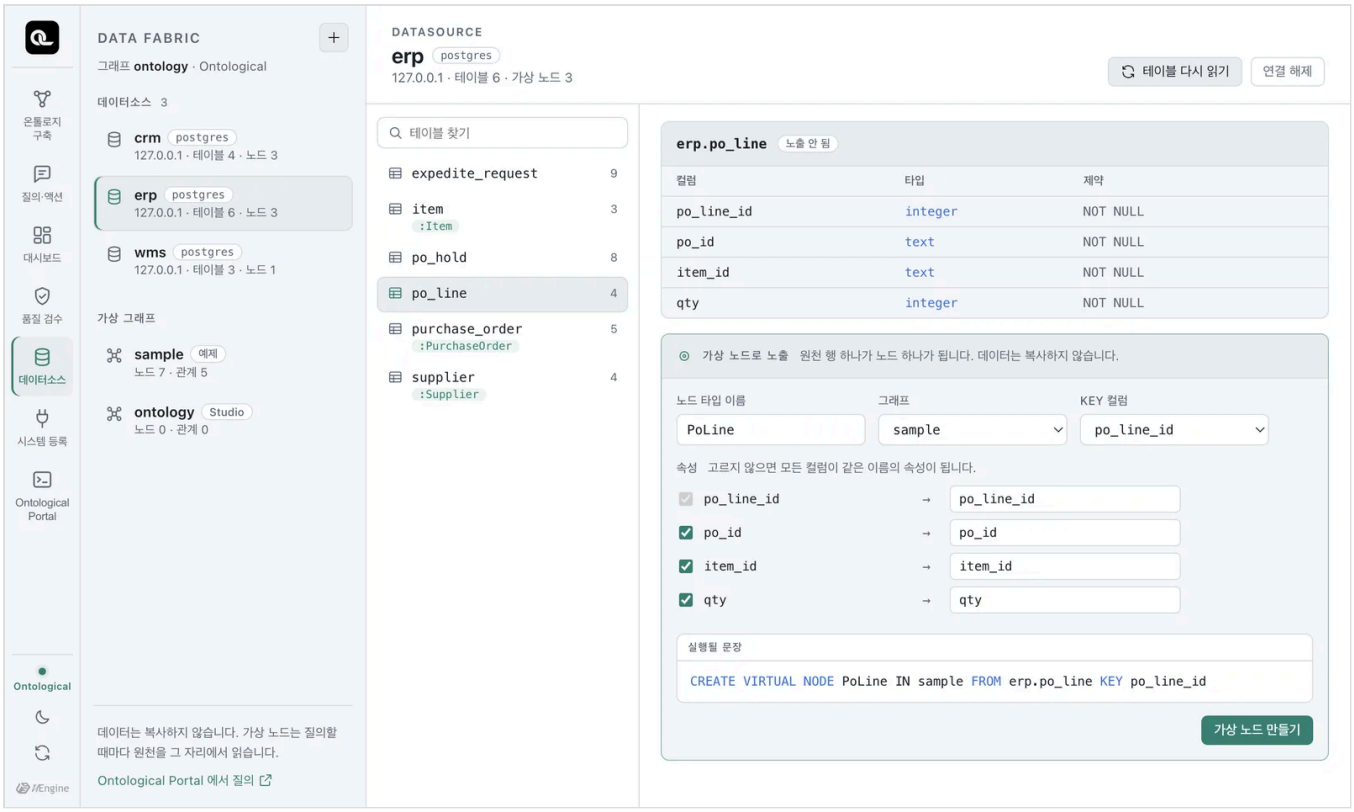
## 가상 노드란

**가상 노드 타입**은 원천 테이블 하나를 그래프의 노드 타입으로 정의한 것입니다.

- 원천 **행 하나 = 노드 하나**, **컬럼 = 속성**입니다.
- **KEY 컬럼**이 노드를 구별하는 값입니다(보통 기본키).
- 데이터는 복사되지 않습니다. `MATCH (n:PoLine)`을 실행할 때마다 원천 `erp.po_line`을 읽습니다.
- **읽기 전용**입니다. 가상 노드에 `CREATE`나 `SET`을 하면 엔진이 거절합니다. 값을 바꾸려면 원천 시스템에서 바꿔야 하고, Ontological Studio에서는 그 일을 **액션**이 합니다(6부).

## 따라 하기 — 발주 품목을 노드로

1. 데이터소스 탭에서 `erp`를 고르고, 테이블 목록에서 `po_line`을 누릅니다. 아직 「노출 안 됨」입니다.
2. 오른쪽 아래 **가상 노드로 노출** 상자에서 노드 타입 이름을 `PoLine`, 그래프를 `sample`, KEY 컬럼을 `po_line_id`로 둡니다.
3. 속성은 그대로 두면 모든 컬럼이 같은 이름의 속성이 됩니다. 이름을 바꾸고 싶으면 오른쪽 칸을 고칩니다(예: `qty` → `quantity`).
4. 「실행될 문장」을 확인하고 「가상 노드 만들기」를 누릅니다.

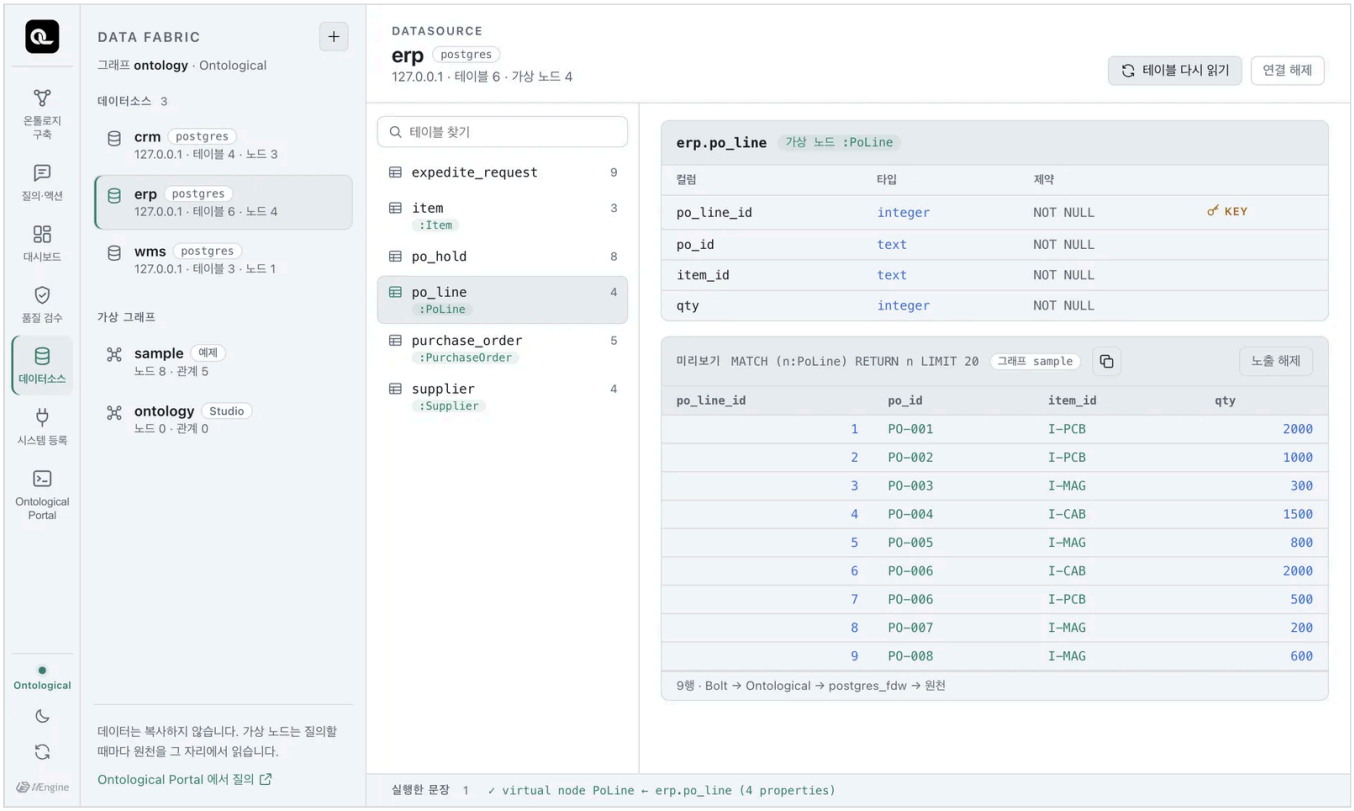



*가상 노드로 노출 — 노드 타입 이름·그래프·KEY 컬럼·속성을 고르면 아래에 실행될 문장이 미리 보입니다*

실행되는 문장은 이것입니다.

```
CREATE VIRTUAL NODE PoLine IN sample FROM erp.po_line KEY po_line_id
```

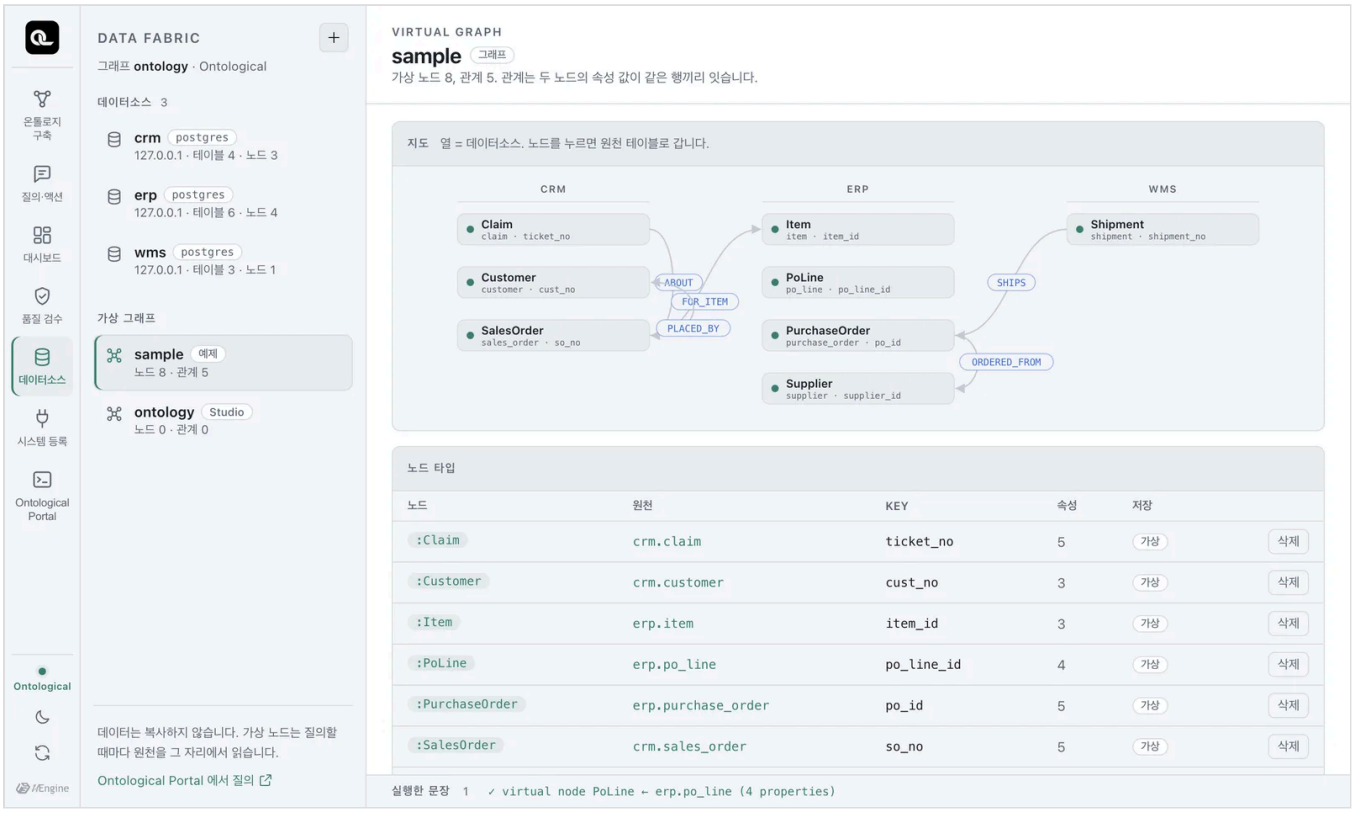
속성 이름을 바꾸면 `PROPERTIES (po_line_id, po_id, item_id, qty AS quantity)`처럼 붙습니다. 만들자마자 미리보기에 원천의 9행이 나옵니다.



*가상 노드 :PoLine이 만들어지고, 원천 erp.po_line의 9행이 미리보기에 나왔습니다*

## 가상 그래프 보기

왼쪽 아래 가상 그래프에서 `sample`을 누르면, 데이터소스별로 묶인 노드와 관계를 그래프로 볼 수 있습니다. `PoLine`은 아직 다른 노드와 연결되어 있지 않습니다.

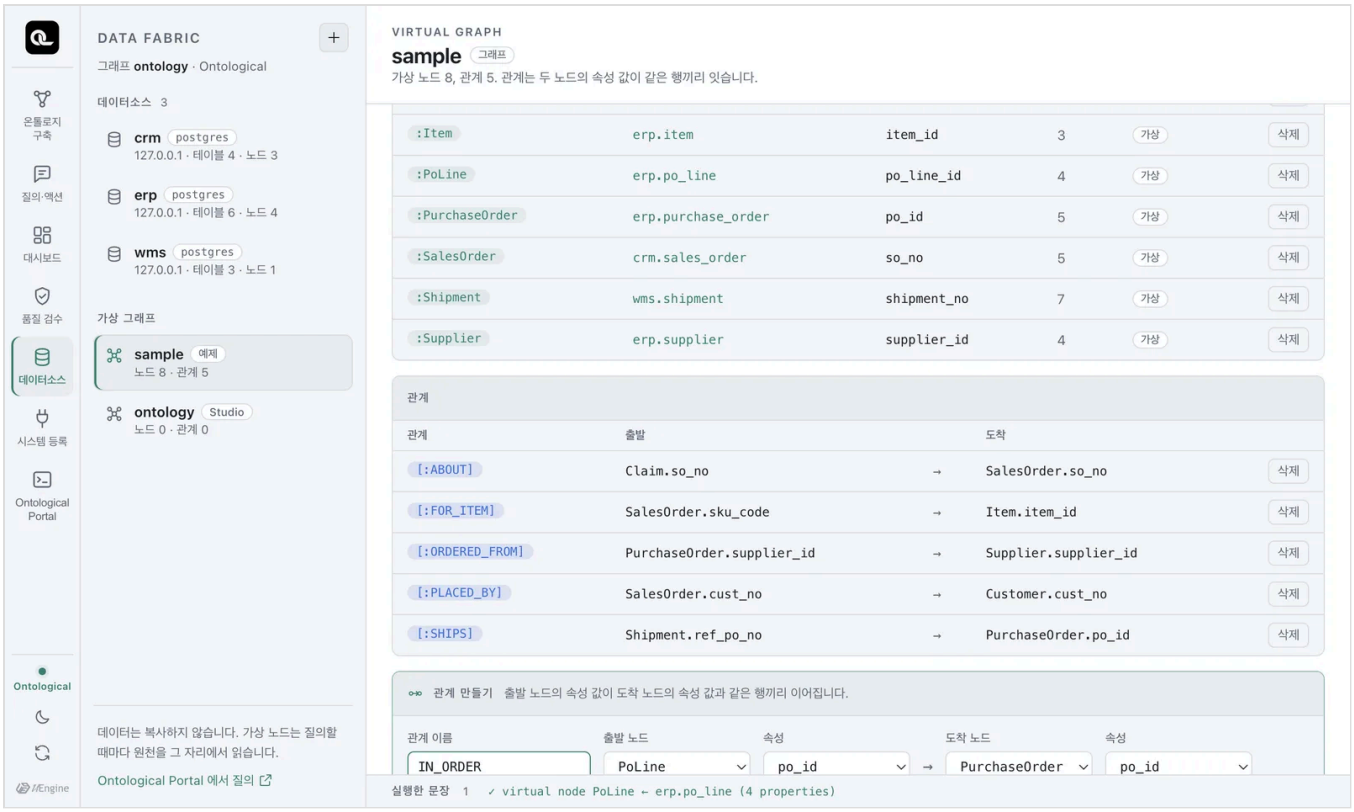



*가상 그래프 sample — 데이터소스별 가상 노드와 가상 관계*

## 따라 하기 — 관계로 잇기

발주 품목은 특정 발주에 포함된 특정 품목을 뜻합니다. 이 두 연결을 관계로 정의합니다.

1. 가상 그래프 화면 아래 관계 만들기 칸에 관계 이름 `IN_ORDER`를 적습니다.
2. 출발 노드 `PoLine`, 속성 `po_id` → 도착 노드 `PurchaseOrder`, 속성 `po_id`를 고릅니다.
3. 「관계 만들기」를 누릅니다.

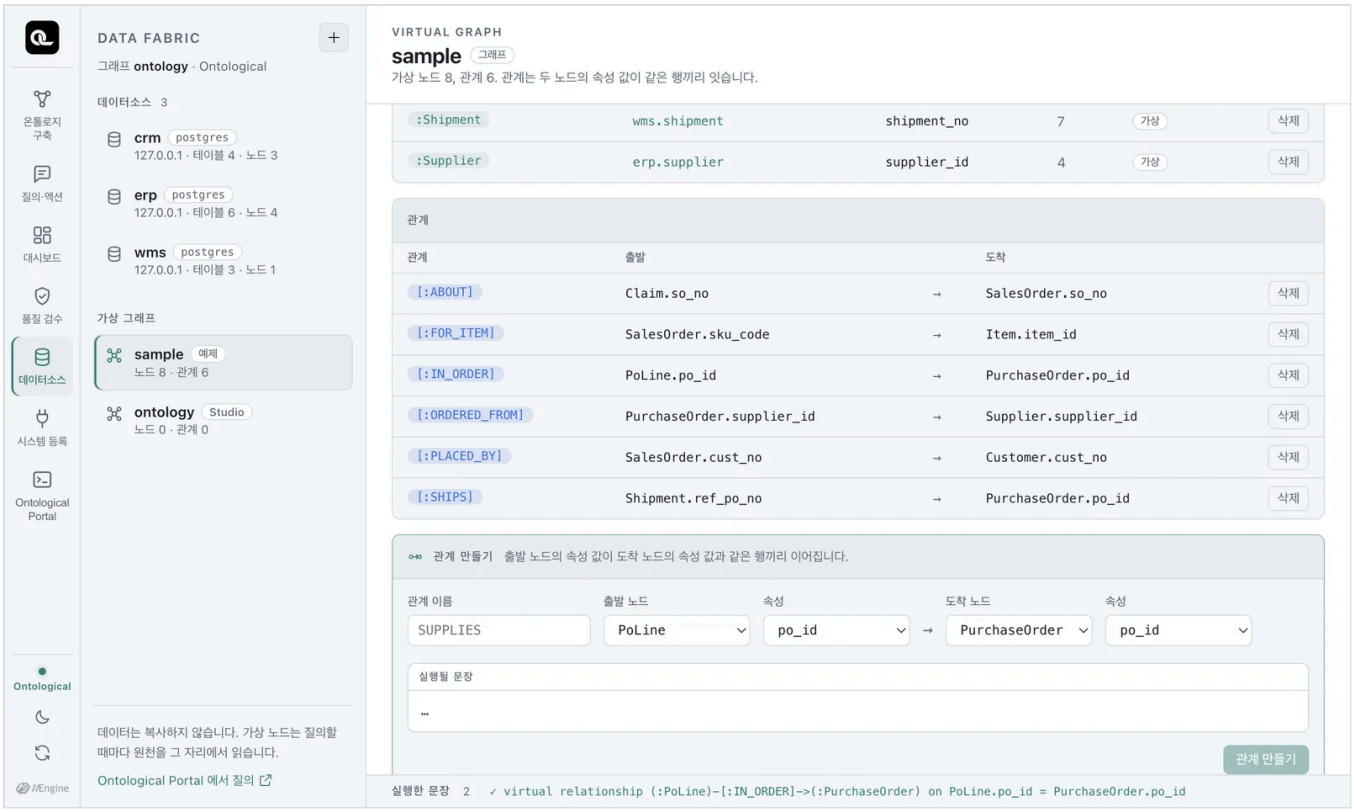



*가상 관계 만들기 — 출발 노드의 속성과 도착 노드의 속성이 같은 값이면 관계로 이어집니다*

```
CREATE VIRTUAL RELATIONSHIP IN_ORDER IN sample FROM PoLine(po_id) TO PurchaseOrder(po_id)
```

같은 방법으로 품목과도 잇습니다. 이번에는 포털에서 직접 문장을 입력해 봅시다(포털은 7장에서 자세히 다룹니다. 그래프 선택을 `sample`로 두세요).

```
CREATE VIRTUAL RELATIONSHIP OF_ITEM IN sample FROM PoLine(item_id) TO Item(item_id)
```




*관계를 만든 뒤의 가상 그래프 — 실행한 문장이 아래에 남습니다*

가상 관계는 데이터를 복사하지 않습니다. **조인 규칙**(`PoLine.po_id = PurchaseOrder.po_id`)을 이름 붙여 저장해 둔 것이고, 질의할 때 그 규칙대로 원천을 조인합니다.

## 바로 써 보기

이제 "협력사별로 어떤 품목을 몇 개나 발주했나"를 묻는 질의를 쓸 수 있습니다. 협력사 → 발주 → 발주 품목 → 품목으로 네 노드의 관계를 따라 조회합니다.

```
MATCH (s:Supplier)<-[:ORDERED_FROM]-(po:PurchaseOrder)<-[:IN_ORDER]-(l:PoLine)-[:OF_ITEM]->(i:Item)
RETURN s.name, i.name, sum(l.qty) AS qty
ORDER BY s.name, i.name
```

| s.name | i.name | qty |
|---|---|---|
| Hanoi Assembly | PCB 기판 | 500 |
| Hanoi Assembly | 전원 케이블 | 2000 |
| Ningbo Magnetics | 마그넷 코어 | 1000 |
| Shenzhen Goldline PCB | PCB 기판 | 3000 |
| 대성케이블 | 전원 케이블 | 1500 |
| 한빛정밀 | 마그넷 코어 | 900 |

그리고 이번에는 **두 시스템에 걸친** 질문입니다. "3일 넘게 늦은 선적이 싣고 오는 품목과 그 안전재고는?" — 선적은 WMS에, 발주·발주 품목·품목은 ERP에 있습니다.

```
MATCH (i:Item)<-[:OF_ITEM]-(l:PoLine)-[:IN_ORDER]->(po:PurchaseOrder)<-[:SHIPS]-(sh:Shipment)
WHERE sh.delay_days > 3
RETURN DISTINCT i.name, i.safety_stock, sh.shipment_no, sh.delay_days
```

| i.name | i.safety_stock | sh.shipment_no | sh.delay_days |
|---|---|---|---|
| PCB 기판 | 500 | SHP-02 | 7 |
| PCB 기판 | 500 | SHP-03 | 9 |

두 선적 모두 PCB 기판을 싣고 오고 있습니다. 재고 담당자에게 먼저 알려야 할 품목이 나왔습니다.

## 만든 것을 지우려면

```
DROP VIRTUAL RELATIONSHIP OF_ITEM IN sample
DROP VIRTUAL RELATIONSHIP IN_ORDER IN sample
DROP VIRTUAL NODE PoLine IN sample
```

관계가 이어져 있는 노드는 지울 수 없습니다. 엔진이 `virtual relationships still join on 'PoLine' (IN_ORDER, OF_ITEM); drop them first` 라며 거절하고, 데이터소스 탭은 이 이유를 한국어로 바꿔 보여 줍니다. 관계부터 지우세요. 원천 테이블은 아무 영향을 받지 않습니다.

> **[팁] 이어지는 장에서 PoLine을 씁니다**
>
> 7~9장의 몇몇 예제가 `PoLine` 과 두 관계를 씁니다. 지우지 말고 두세요.
>
> **[주의] 가상 노드에 쓰려고 하면**
>
> `CREATE (x:Supplier {supplier_id:'S-XX'})` 처럼 가상 노드 타입에 쓰기를 하면 다음과 같이 거절됩니다.
>
> ```
> 'Supplier' is a virtual node type and read-only: its rows live in "erp"."supplier".
> Write to the source system, or MATERIALIZE VIRTUAL NODE Supplier to take a copy
> ```
>
> 원천 데이터를 고치는 일은 그 데이터의 주인인 업무 시스템이 해야 합니다. 꼭 사본이 필요하면 `MATERIALIZE VIRTUAL NODE` 로 복사본을 만들 수 있지만, 복사한 뒤에는 원천의 변경 사항이 자동으로 반영되지 않습니다.
>
> **[확인 문제]**
>
> 1. `wms.stock_daily` 를 `Stock` 가상 노드로 만들 수 있을까요? 무엇을 KEY로 삼아야 할지 생각해 보세요.
> 2. `SalesOrder` 와 `Item` 사이의 `FOR_ITEM` 관계는 어떤 두 속성을 잇고 있을까요? `SHOW VIRTUAL RELATIONSHIPS` 로 확인하세요.
> 3. 위의 "협력사별 품목 발주 수량" 질의를 고쳐서, 미결( `OPEN` ) 발주만 세어 보세요.
>
> **▼ 정답 보기**
>
> 1. 이 테이블의 기본키는 (snapshot_date, sku_code, warehouse_cd) 세 컬럼입니다. 그런데 `KEY` 는 컬럼 하나만 받습니다( `KEY (a, b, c)` 는 `expected the key column` 오류). `KEY sku_code` 로 두면 만들어지기는 하지만, 같은 품목이 날짜·창고별로 여러 행(10행)이라 노드 하나를 키로 구별할 수 없습니다. 원천에 세 값을

하나로 이은 식별자 컬럼(예: 뷰)을 두는 것이 맞습니다. 데이터 모델링에서 “식별자”가 왜 중요한지 보여 주는 예입니다.

2. `SalesOrder.sku_code` → `Item.item_id`. CRM은 품목을 `sku_code`라 부르고 ERP는 `item_id`라 부릅니다.
3. `WHERE po.status = 'OPEN'`을 `RETURN` 앞에 넣습니다.

```
MATCH (s:Supplier)<-[:ORDERED_FROM]-(po:PurchaseOrder)<-[:IN_ORDER]-(l:PoLine)-[:OF_ITEM]->(i:Ite
m)
WHERE po.status = 'OPEN'
RETURN s.name, i.name, sum(l.qty) AS qty ORDER BY s.name, i.name
```

# Cypher 기초 — 노드를 찾고 거르기

*분류: Cypher 익히기 · 7장 · 원문: https://www.uengine.org/docs/ontological-studio/cypher-basics.html*

> Ontological Portal을 열고, 그래프 질의 언어 Cypher의 기본 문장 MATCH·WHERE·RETURN·ORDER BY·LIMIT를 예제 데이터로 하나씩 실행해 봅니다.

▶ 영상: Cypher 기초 — 노드를 찾고 거르기 — 영상 2:55

## Cypher는 그림으로 쓰는 질의

Cypher는 그래프의 구조를 **그림을 그리듯 표현하는** 질의 언어입니다. Neo4j에서 시작해 지금은 ISO 표준 그래프 질의 언어(GQL)의 바탕이 되었습니다.

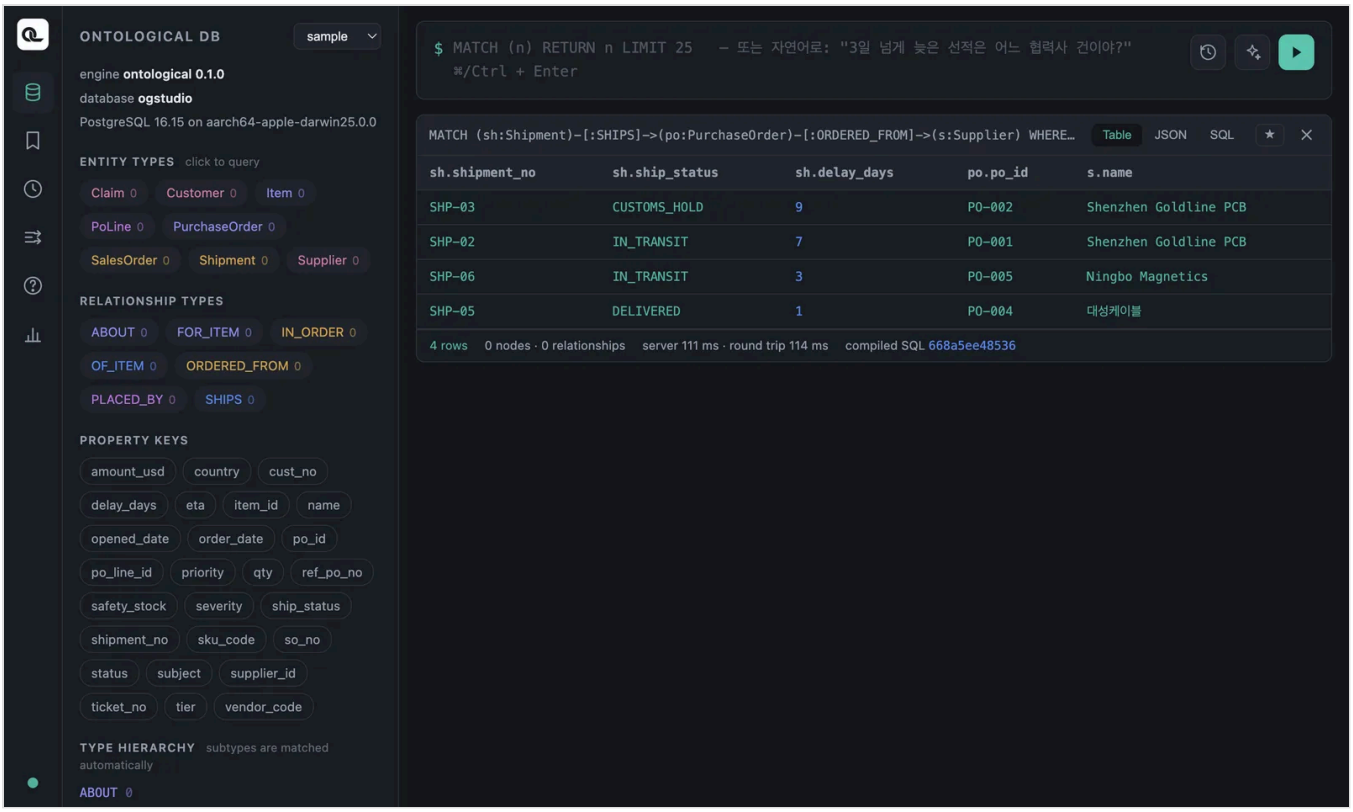
| 그림 | Cypher | 뜻 |
|---|---|---|
| ○ | `(s:Supplier)` | `Supplier` 라벨을 가진 노드 하나, 이름은 `s` |
| ○→○ | `(po)-[:ORDERED_FROM]->(s)` | `po` 에서 `s` 로 가는 `ORDERED_FROM` 관계 |
| 속성 | `s.name` | 노드 `s` 의 `name` 값 |

SQL의 `SELECT … FROM … WHERE` 와 비교하면 이해하기 쉽습니다.

| SQL | Cypher |
|---|---|
| `FROM supplier s` | `MATCH (s:Supplier)` |
| `WHERE s.country = 'China'` | `WHERE s.country = 'China'` |
| `SELECT s.name` | `RETURN s.name` |
| `ORDER BY … LIMIT 3` | `ORDER BY … LIMIT 3` |
| `JOIN … ON …` | 관계 패턴 `-[:REL]->` (8장) |

## 포털 열기

홈의 「Ontological Portal」 카드를 누릅니다. 왼쪽 위의 그래프 선택에서 **`sample`** 을 고르세요. 4부의 모든 예제는 `sample` 그래프에서 실행합니다.




*Ontological Portal — 왼쪽에 이 그래프의 노드 타입·관계 타입·속성 키, 위쪽에 질의 입력칸, 아래에 결과*

| 영역 | 내용 |
|---|---|
| 왼쪽 ENTITY TYPES | 노드 라벨. 누르면 그 라벨을 조회하는 질의가 입력됩니다. |
| 왼쪽 RELATIONSHIP TYPES / PROPERTY KEYS | 관계 이름과 속성 이름. 질의를 쓸 때 참고합니다. |
| 위 입력칸 | Cypher를 쓰고 ⌘Enter(Windows는 Ctrl+Enter) 또는 ▶ 버튼으로 실행합니다. `(n:` 을 치면 라벨, `n.` 을 치면 속성을 자동완성합니다. |
| 결과 카드 | Table·JSON·SQL 보기, 노드를 반환하면 Graph 보기도 생깁니다. ★로 저장, ✕로 닫기. |

## 첫 질의

```
MATCH (s:Supplier)
RETURN s.supplier_id, s.name, s.country, s.tier
```

| s.supplier_id | s.name | s.country | s.tier |
|---|---|---|---|
| S-KR-01 | 한빛정밀 | Korea | 1 |
| S-KR-02 | 대성케이블 | Korea | 2 |
| S-CN-01 | Shenzhen Goldline PCB | China | 1 |
| S-CN-02 | Ningbo Magnetics | China | 2 |
| S-VN-01 | Hanoi Assembly | Vietnam | 2 |

`MATCH` 는 찾을 그래프 패턴을 지정하고, `RETURN` 은 조회 결과에 포함할 항목을 지정합니다. 노드 전체를 조회하려면 `RETURN s` 라고 씁니다. 그러면 모든 속성이 담긴 객체가 반환되고, 결과 카드에 Graph 보기가 생깁니다.

## 거르기 — WHERE

```
MATCH (s:Supplier)
WHERE s.country = 'China' AND s.tier = 1
RETURN s.name
```

결과는 `Shenzhen Goldline PCB` 한 곳입니다. `WHERE` 에서 사용할 수 있는 조건은 다음과 같습니다.

| 쓰임 | 예 |
|---|---|
| 비교 | `=` `<>` `<` `<=` `>` `>=` — `po.amount_usd >= 30000 AND po.amount_usd < 70000` |
| 논리 | `AND` `OR` `NOT` — `NOT s.country = 'Korea'` |
| 목록에 포함 | `sh.ship_status IN ['IN_TRANSIT', 'CUSTOMS_HOLD']` |
| 문자열 | `s.name STARTS WITH 'S'`, `CONTAINS 'Gold'`, `ENDS WITH 'PCB'` |
| 값 없음 | `po IS NULL`, `po IS NOT NULL` |

조건이 하나뿐이면 패턴 안에 속성과 값을 직접 적어도 됩니다.

```
MATCH (s:Supplier {supplier_id: 'S-CN-01'})
RETURN s
```

> **[참고] 날짜 비교**
>
> 날짜 컬럼( `opened_date`, `eta` 등)은 `'2026-08-29'` 같은 문자열과 바로 비교할 수 있습니다. 엔진이 원천 타입( `date` )으로 바꿔 줍니다. `date()` 함수는 아직 없습니다.
>
> ```
> MATCH (c:Claim) WHERE c.opened_date >= '2026-08-29'
> RETURN c.ticket_no, c.opened_date
> ```

## 정렬과 자르기 — ORDER BY · SKIP · LIMIT

금액이 큰 발주 세 건:

```
MATCH (po:PurchaseOrder)
RETURN po.po_id, po.amount_usd, po.status
ORDER BY po.amount_usd DESC
LIMIT 3
```

| po.po_id | po.amount_usd | po.status |
|---|---|---|
| PO-001 | 82000 | OPEN |

| po.po_id | po.amount_usd | po.status |
|---|---|---|
| PO-005 | 60000 | OPEN |
| PO-002 | 41000 | OPEN |

`ORDER BY a, b DESC` 처럼 여러 기준을 줄 수 있고, `SKIP 2 LIMIT 3` 은 "두 개 건너뛰고 세 개" — 페이지를 나눌 때 사용합니다. `RETURN po.po_id AS 발주` 처럼 `AS` 로 컬럼 이름을 바꿀 수 있습니다(한글도 됩니다).

## 중복 없애기와 세기

```
MATCH (s:Supplier) RETURN DISTINCT s.country
```

```
MATCH (s:Supplier) RETURN count(s) AS suppliers
```

첫 질의는 Vietnam·Korea·China 세 행, 둘째는 `5` 입니다.

## 값 바꿔 보여 주기 — 함수와 CASE

```
MATCH (sh:Shipment)
RETURN sh.shipment_no,
       CASE WHEN sh.delay_days > 3 THEN '지연'
            WHEN sh.delay_days > 0 THEN '주의'
            ELSE '정상' END AS 상태
ORDER BY sh.shipment_no
```

| sh.shipment_no | 상태 |
|---|---|
| SHP-01 | 정상 |
| SHP-02 | 지연 |
| SHP-03 | 지연 |
| SHP-04 | 정상 |
| SHP-05 | 주의 |
| SHP-06 | 주의 |
| SHP-07 | 정상 |
| SHP-08 | 정상 |

Ontological DB가 지원하는 함수는 다음과 같습니다. 목록에 없는 함수를 쓰면 엔진이 이 목록을 알려 주는 오류를 냅니다.

| 분류 | 함수 |
|---|---|
| 집계 | `count` `sum` `avg` `min` `max` `collect` |
| 문자열 | `toUpper` `toLower` `trim` `substring` `replace` `split` `toString` |
| 숫자 | `abs` `ceil` `floor` `round` `sqrt` `rand` `toInteger` `toFloat` |

| 분류 | 함수 |
|---|---|
| 그래프 | `id` `elementId` `labels` `type` `length` `size` `keys` `exists` |
| 기타 | `coalesce` `timestamp` `datetime` `vector.similarity` `vector.distance` `genai.vector.encode` |

## 결과를 보는 세 가지 방법

결과 카드 오른쪽 위에서 고릅니다.

- **Table** — 결과를 테이블로 보여 줍니다. 가장 자주 사용하는 보기입니다.
- **JSON** — 노드 객체를 그대로 보여 줍니다. 앱 코드에서 받는 형식과 같습니다(19장).
- **SQL** — Cypher를 변환한 PostgreSQL SQL입니다. Ontological DB가 질의를 어떻게 실행하는지 확인할 수 있습니다(9장).

> **[확인 문제]**
>
> `sample` 그래프에서 다음을 Cypher로 써 보세요.
>
> 1. 상태가 `OPEN`인 발주의 번호와 금액을 금액이 작은 순으로 조회하세요.
> 2. 세관 보류(`CUSTOMS_HOLD`)이거나 운송 중(`IN_TRANSIT`)인 선적의 번호와 도착 예정일(`eta`).
> 3. 고객이 몇 곳인지 확인하세요.
> 4. 클레임 중 심각도(`severity`)가 `HIGH`인 것의 번호와 제목.
>
> **▼ 정답 보기**
>
> ```
> MATCH (po:PurchaseOrder) WHERE po.status = 'OPEN'
> RETURN po.po_id, po.amount_usd ORDER BY po.amount_usd
> ```
>
> PO-007(12000), PO-008(27000), PO-006(33000), PO-002(41000), PO-005(60000), PO-001(82000) — 모두 6건입니다.
>
> ```
> MATCH (sh:Shipment) WHERE sh.ship_status IN ['IN_TRANSIT', 'CUSTOMS_HOLD']
> RETURN sh.shipment_no, sh.eta
> ```
>
> SHP-02, SHP-03, SHP-06 세 건.
>
> ```
> MATCH (cu:Customer) RETURN count(cu) AS customers
> ```
>
> 3곳.
>
> ```
> MATCH (c:Claim) WHERE c.severity = 'HIGH' RETURN c.ticket_no, c.subject
> ```
>
> T-1「컨트롤러 전원 불량」, T-3「PCB 납땜 불량」.

# 관계 따라가기와 집계

*분류: Cypher 익히기 · 8장 · 원문: https://www.uengine.org/docs/ontological-studio/cypher-relations.html*

> 관계 패턴을 따라 노드를 조회하고, 여러 데이터베이스를 하나의 질의로 연결해 count·sum·collect로 집계합니다. Cypher가 SQL보다 편해지는 지점입니다.

▶ 영상: 관계 따라가기와 집계 — 영상 3:53

## 관계 패턴

```
MATCH (po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier)
RETURN po.po_id, po.status, s.name
```

| po.po_id | po.status | s.name |
|---|---|---|
| PO-001 | OPEN | Shenzhen Goldline PCB |
| PO-002 | OPEN | Shenzhen Goldline PCB |
| PO-003 | CLOSED | 한빛정밀 |
| PO-004 | CLOSED | 대성케이블 |
| PO-005 | OPEN | Ningbo Magnetics |
| PO-006 | OPEN | Hanoi Assembly |
| PO-007 | OPEN | Ningbo Magnetics |
| PO-008 | OPEN | 한빛정밀 |

발주는 ERP의 `purchase_order`, 협력사는 ERP의 `supplier`입니다. 관계 `ORDERED_FROM`이 `purchase_order.supplier_id = supplier.supplier_id` 조인을 대신했습니다.

### 방향

| 패턴 | 뜻 |
|---|---|
| `(a)-[:R]->(b)` | a에서 b로 |
| `(a)<-[:R]-(b)` | b에서 a로 — 같은 관계를 반대편에서 읽기 |
| `(a)-[:R]-(b)` | 방향 상관없이 |

관계에는 정의된 방향이 있습니다(`PurchaseOrder → Supplier`). 협력사에서 출발하고 싶으면 화살표를 반대로 씁니다.

```
MATCH (s:Supplier)<-[:ORDERED_FROM]-(po:PurchaseOrder)
RETURN s.name, collect(po.po_id) AS orders
```

| s.name | orders |
|---|---|
| 대성케이블 | ["PO-004"] |
| Shenzhen Goldline PCB | ["PO-001", "PO-002"] |
| 한빛정밀 | ["PO-003", "PO-008"] |
| Ningbo Magnetics | ["PO-005", "PO-007"] |
| Hanoi Assembly | ["PO-006"] |

> **[주의] 노드마다 라벨을 붙이세요**
>
> Ontological DB에서는 패턴의 **노드마다 라벨**을 적어야 빠르고 정확합니다. `(s:Supplier)<-[:ORDERED_FROM]-(po)` 보다 `(po:PurchaseOrder)` 처럼 쓰세요. 특히 데이터 패브릭의 가상 관계는 라벨 없이 `-[r]->`(관계 이름도 없이)로 찾으면 결과가 비어 나옵니다. 관계 이름과 양쪽 라벨을 모두 적는 습관을 들이세요.

## 여러 걸음 — 다른 데이터베이스를 건너서

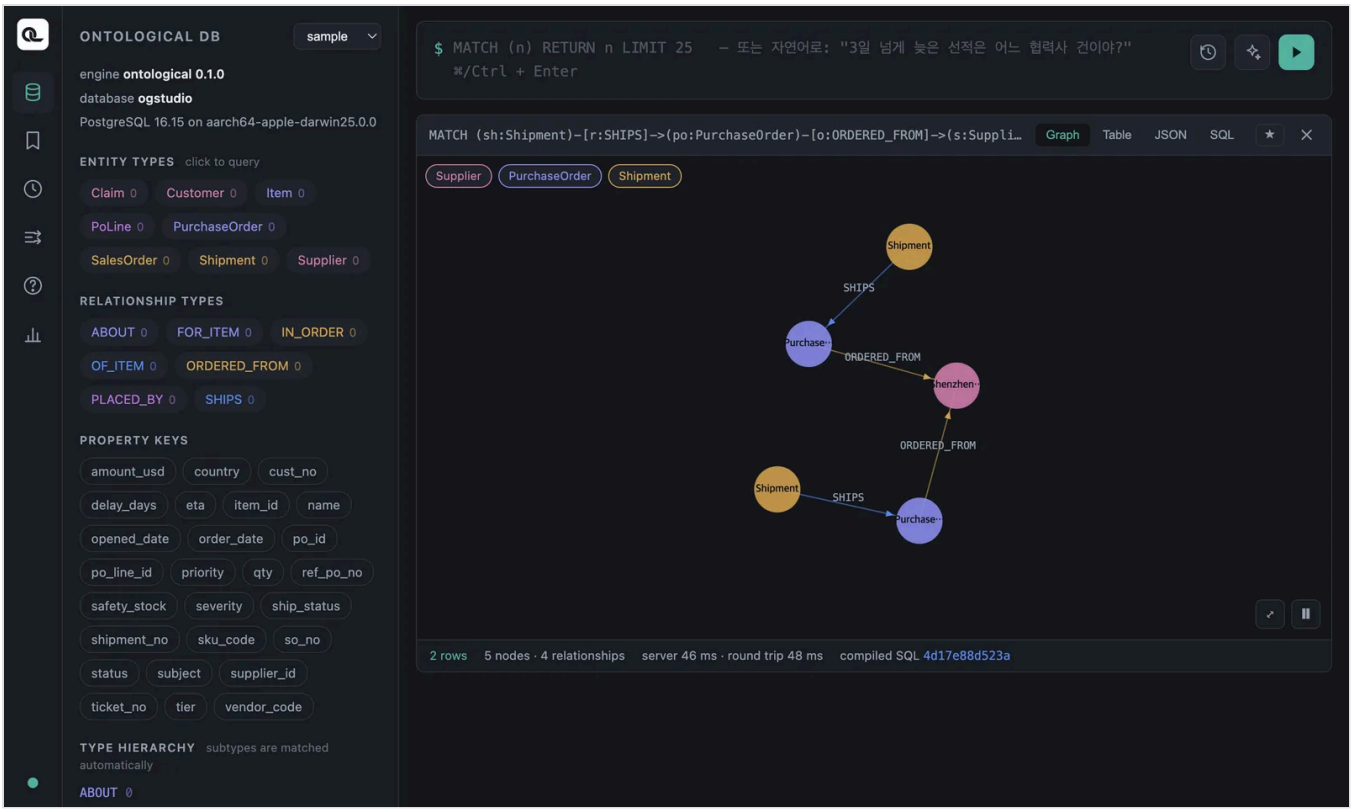
2장에서 본 질문입니다. 선적(WMS) → 발주(ERP) → 협력사(ERP).

```
MATCH (sh:Shipment)-[:SHIPS]->(po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier)
WHERE sh.delay_days > 0
RETURN sh.shipment_no, sh.ship_status, sh.delay_days, po.po_id, s.name
ORDER BY sh.delay_days DESC
```

| sh.shipment_no | sh.ship_status | sh.delay_days | po.po_id | s.name |
|---|---|---|---|---|
| SHP-03 | CUSTOMS_HOLD | 9 | PO-002 | Shenzhen Goldline PCB |
| SHP-02 | IN_TRANSIT | 7 | PO-001 | Shenzhen Goldline PCB |
| SHP-06 | IN_TRANSIT | 3 | PO-005 | Ningbo Magnetics |
| SHP-05 | DELIVERED | 1 | PO-004 | 대성케이블 |

노드 전체를 반환하면 그래프로 볼 수 있습니다. 관계에도 변수(`r`, `o`)를 붙여 함께 반환합니다.

```
MATCH (sh:Shipment)-[r:SHIPS]->(po:PurchaseOrder)-[o:ORDERED_FROM]->(s:Supplier)
WHERE sh.delay_days > 3
RETURN sh, r, po, o, s
```




*결과 카드의 Graph 보기 — 늦은 선적 두 건이 각자의 발주를 거쳐 같은 협력사로 모입니다*

선적 두 건이 서로 다른 발주를 거쳐 **같은 협력사로 연결되는** 모습을 볼 수 있습니다. 테이블에서는 알아보기 어려웠던 사실입니다. 지연된 선적이 한 협력사에 집중되어 있습니다.

### 고객 쪽으로

CRM 안에서 클레임 → 판매 주문 → 고객:

```
MATCH (c:Claim)-[:ABOUT]->(so:SalesOrder)-[:PLACED_BY]->(cu:Customer)
RETURN c.ticket_no, c.severity, c.subject, so.so_no, cu.name
```

| c.ticket_no | c.severity | c.subject | so.so_no | cu.name |
|---|---|---|---|---|
| T-1 | HIGH | 컨트롤러 전원 불량 | SO-01 | 서울전자 |
| T-2 | LOW | 케이블 포장 파손 | SO-04 | Berlin Motion |
| T-3 | HIGH | PCB 납땜 불량 | SO-05 | Tokyo Devices |

### 갈래가 둘인 패턴

쉼표로 여러 패턴을 적고 **같은 변수**를 사용하면 패턴을 연결할 수 있습니다. “납땜 불량 클레임이 난 품목을 공급한 협력사는?”이라는 질문에 답하려면 CRM의 클레임에서 품목을 찾고, 그 품목을 ERP의 발주 품목과 협력사에 연결합니다. (6장에서 만든 `PoLine`이 필요합니다.)

```
MATCH (i:Item)<-[:FOR_ITEM]-(so:SalesOrder)<-[:ABOUT]-(c:Claim),
      (i:Item)<-[:OF_ITEM]-(l:PoLine)-[:IN_ORDER]->(po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier)
WHERE c.subject CONTAINS '납땜'
RETURN DISTINCT i.name, s.name
```

| i.name | s.name |
|---|---|
| PCB 기판 | Hanoi Assembly |
| PCB 기판 | Shenzhen Goldline PCB |

두 데이터베이스(CRM·ERP)에 걸친 다섯 종류의 노드를 하나의 질의로 조회했습니다.

## 없는 짝도 남기기 — OPTIONAL MATCH

SQL의 `LEFT JOIN`에 해당합니다. 연결된 데이터가 없어도 행을 유지하며, 데이터가 없는 쪽은 `null`이 됩니다.

```
MATCH (sh:Shipment)
OPTIONAL MATCH (sh:Shipment)-[:SHIPS]->(po:PurchaseOrder)
RETURN sh.shipment_no, po.po_id
ORDER BY sh.shipment_no
```

8건 중 `SHP-08`만 `po.po_id`가 `null`입니다. 이 선적은 ERP에 없는 발주 `PO-999`를 가리키고 있습니다 — 일부러 넣어 둔 **데이터 품질 문제**입니다. 연결된 데이터가 없는 항목만 고르려면:

```
MATCH (sh:Shipment)
OPTIONAL MATCH (sh:Shipment)-[:SHIPS]->(po:PurchaseOrder)
WITH sh, po WHERE po IS NULL
RETURN sh.shipment_no, sh.ref_po_no
```

조회 결과는 `SHP-08`, `PO-999`입니다.

> **[참고] NOT EXISTS 대신**
>
> Neo4j의 `WHERE NOT EXISTS { MATCH … }` 하위 질의는 아직 지원하지 않습니다. 위처럼 `OPTIONAL MATCH` + `IS NULL`로 씁니다.

## 묶어서 세기 — 집계

`RETURN`에 집계 함수와 보통 값이 함께 있으면, **일반 컬럼의 값이 같은 행이 한 그룹**이 됩니다(SQL의 `GROUP BY`를 따로 쓰지 않습니다).

```
MATCH (po:PurchaseOrder)
RETURN po.status, count(*) AS n, sum(po.amount_usd) AS total,
       avg(po.amount_usd) AS average, max(po.amount_usd) AS biggest
```

| po.status | n | total | average | biggest |
|---|---|---|---|---|
| CLOSED | 2 | 24000 | 12000 | 15000 |
| OPEN | 6 | 255000 | 42500 | 82000 |

협력사별 미결 발주 금액:

```
MATCH (po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier)
WHERE po.status = 'OPEN'
RETURN s.name AS supplier, count(po) AS open_orders, sum(po.amount_usd) AS open_amount
ORDER BY open_amount DESC
```

| supplier | open_orders | open_amount |
|---|---|---|
| Shenzhen Goldline PCB | 2 | 123000 |
| Ningbo Magnetics | 2 | 72000 |
| Hanoi Assembly | 1 | 33000 |
| 한빛정밀 | 1 | 27000 |

협력사별 최장 지연 일수:

```
MATCH (s:Supplier)<-[:ORDERED_FROM]-(po:PurchaseOrder)<-[:SHIPS]-(sh:Shipment)
RETURN s.name, max(sh.delay_days) AS worst
ORDER BY worst DESC
```

Shenzhen Goldline PCB 9일, Ningbo Magnetics 3일, 대성케이블 1일, 나머지 0일.

## 단계를 나누기 — WITH

`WITH`는 질의를 두 단계로 나눕니다. 앞 단계의 결과를 다음 단계가 이어받습니다. 집계한 값으로 다시 거를 때(SQL의 `HAVING`) 씁니다.

```
MATCH (po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier)
WITH s, count(po) AS n
WHERE n >= 2
RETURN s.name, n
```

발주가 두 건 이상인 협력사 — Ningbo Magnetics, Shenzhen Goldline PCB, 한빛정밀(각 2건).

`collect()`로 모은 목록의 길이는 `size()`로 확인합니다.

```
MATCH (po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier)
WITH s, collect(po.po_id) AS pos
RETURN s.name, size(pos) AS n, pos
ORDER BY n DESC
```

**[주의] 알려진 제약 — 금액 합계로 거르기**

`WITH s, sum(po.amount_usd) AS total WHERE total > 50000` 처럼 **소수 타입 컬럼의 합계로 다시 거르면** 현재 버전에서는 `invalid input syntax for type bigint` 오류가 납니다.
`sum(toFloat(po.amount_usd))` 로 실수로 바꿔 더하면 됩니다.

```
MATCH (po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier)
WITH s, sum(toFloat(po.amount_usd)) AS total
WHERE total > 50000
RETURN s.name, total ORDER BY total DESC
```

## 목록에서 시작하기 — UNWIND

목록을 행으로 펼칩니다. 여러 키를 한 번에 조회할 때 편합니다.

```
UNWIND ['SHP-02', 'SHP-03'] AS no
MATCH (sh:Shipment {shipment_no: no})
RETURN sh.shipment_no, sh.eta
```

**[확인 문제]**

1. 미결(`OPEN`) 판매 주문을 고객별로 모아, 고객 이름과 주문 번호 목록을 보여 주세요.
2. 선적이 두 건 이상인 발주는 무엇인가요?
3. 심각도 `HIGH` 클레임을 낸 고객과 그 판매 주문을 찾으세요.
4. (도전) 협력사마다 미결 발주가 몇 건인지, **미결 발주가 없는 협력사도 0으로** 나오게 쓰세요.

**▼ 정답 보기**

```
MATCH (cu:Customer)<-[:PLACED_BY]-(so:SalesOrder)
WHERE so.status = 'OPEN'
RETURN cu.name, collect(so.so_no) AS open_orders
```

Tokyo Devices `["SO-03", "SO-05"]`, 서울전자 `["SO-02"]`.

```
MATCH (po:PurchaseOrder)<-[:SHIPS]-(sh:Shipment)
WITH po, count(sh) AS shipments WHERE shipments >= 2
RETURN po.po_id, shipments
```

PO-001 (SHP-01, SHP-02).

```
MATCH (cu:Customer)<-[:PLACED_BY]-(so:SalesOrder)<-[:ABOUT]-(c:Claim)
WHERE c.severity = 'HIGH'
RETURN cu.name, so.so_no, c.subject
```

서울전자 SO-01「컨트롤러 전원 불량」, Tokyo Devices SO-05「PCB 납땜 불량」.

```
MATCH (s:Supplier)
OPTIONAL MATCH (s:Supplier)<-[:ORDERED_FROM]-(po:PurchaseOrder)
WHERE po.status = 'OPEN'
RETURN s.name, count(po) AS open_n ORDER BY open_n DESC
```

대성케이블이 0으로 나옵니다. `count(po)` 는 `null` 을 세지 않기 때문입니다( `count(*)` 였다면 1이 됩니다).

# Cypher 심화 — 실행 들여다보기, 말로 묻기, 직접 쓰기

*분류: Cypher 익히기 · 9장 · 원문: https://www.uengine.org/docs/ontological-studio/cypher-advanced.html*

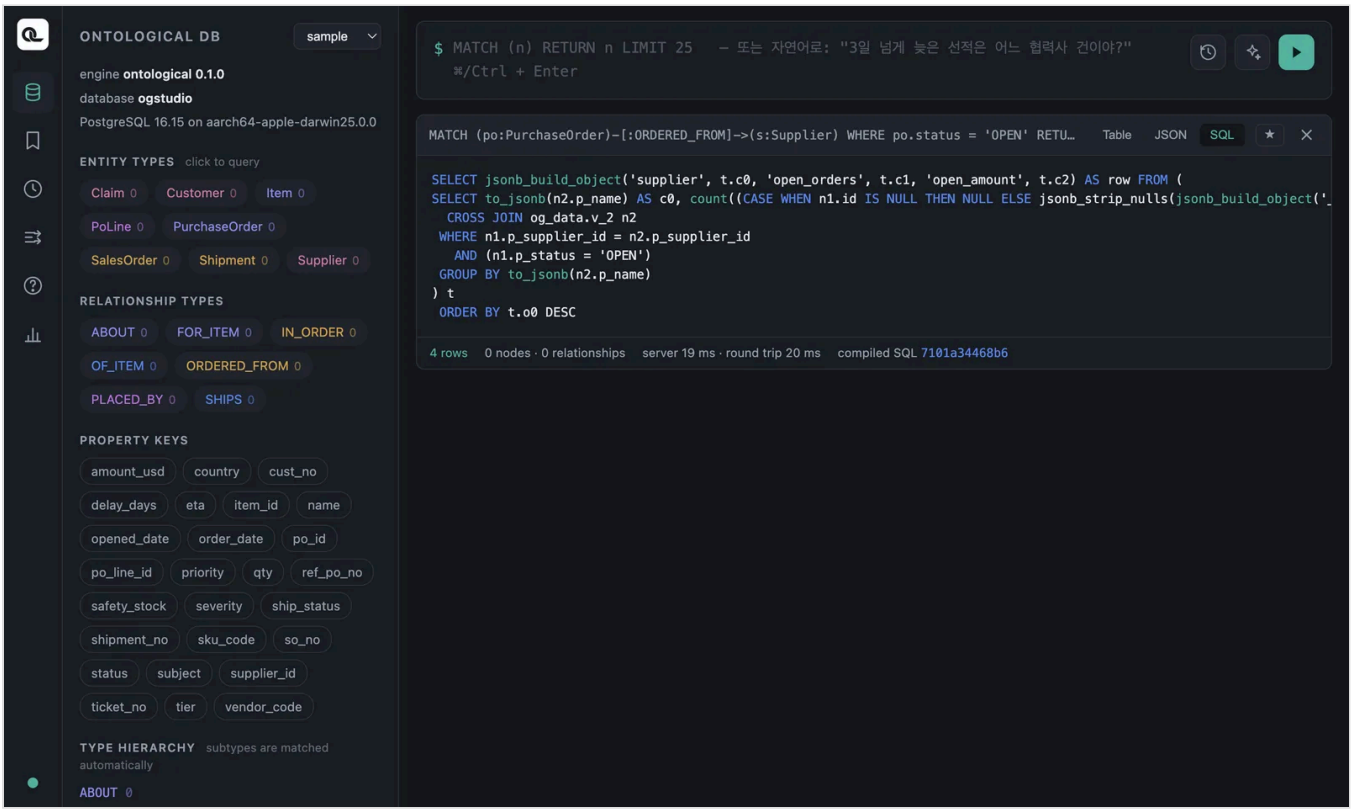
> 질의가 실제로 어떻게 실행되는지(SQL 변환과 패브릭 실행 계획)를 살펴보고, 자연어로 Cypher를 만들고, 내 그래프를 만들어 노드와 관계를 직접 써 봅니다.

▶ 영상: Cypher 심화 — 실행 들여다보기, 말로 묻기, 직접 쓰기 — 영상 3:59

## 번역된 SQL 보기

Ontological DB는 Cypher를 PostgreSQL SQL로 **변환**해 실행합니다. 결과 카드의 **SQL** 탭에서 변환 결과를 볼 수 있습니다.

```
MATCH (po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier)
WHERE po.status = 'OPEN'
RETURN s.name AS supplier, count(po) AS open_orders, sum(po.amount_usd) AS open_amount
ORDER BY open_amount DESC
```




*SQL 탭 — Cypher 한 문장이 PostgreSQL 질의로 번역된 모습*

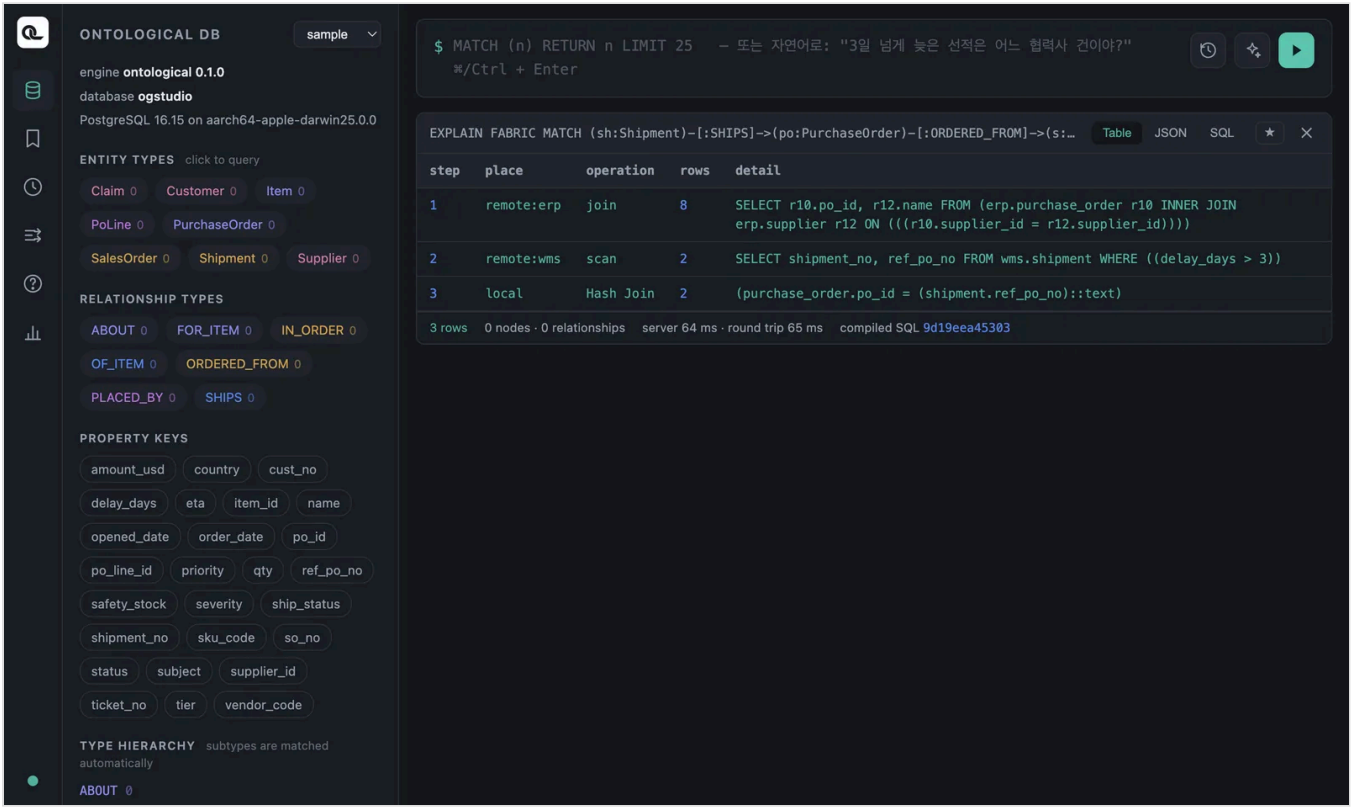
결과 카드 아래의 `compiled SQL 668a5ee48536` 같은 값은 변환된 SQL을 식별하는 값입니다. 같은 Cypher면 시점(10장)을 바꿔도 지문이 같습니다 — 질의는 그대로이고 조건만 바뀐다는 뜻입니다.

## 패브릭 실행 계획 — EXPLAIN FABRIC

원천이 여러 DB에 걸친 질의는 **어느 시스템에서 어떤 작업을 실행하는지**가 성능을 좌우합니다. 질의 앞에 `EXPLAIN FABRIC`을 붙이면 단계별 계획을 보여 줍니다.

```
EXPLAIN FABRIC
MATCH (sh:Shipment)-[:SHIPS]->(po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier)
WHERE sh.delay_days > 3
RETURN sh.shipment_no, s.name
```

| step | place | operation | rows | detail |
|---|---|---|---|---|
| 1 | remote:erp | join | 8 | `SELECT r10.po_id, r12.name FROM (erp.purchase_order r10 INNER JOIN erp.supplier r12 ON …)` |
| 2 | remote:wms | scan | 2 | `SELECT shipment_no, ref_po_no FROM wms.shipment WHERE ((delay_days > 3))` |
| 3 | local | Hash Join | 2 | `(purchase_order.po_id = (shipment.ref_po_no)::text)` |




*EXPLAIN FABRIC — 원천마다 내려 보낸 SQL과 넘어온 행 수*

읽는 법:

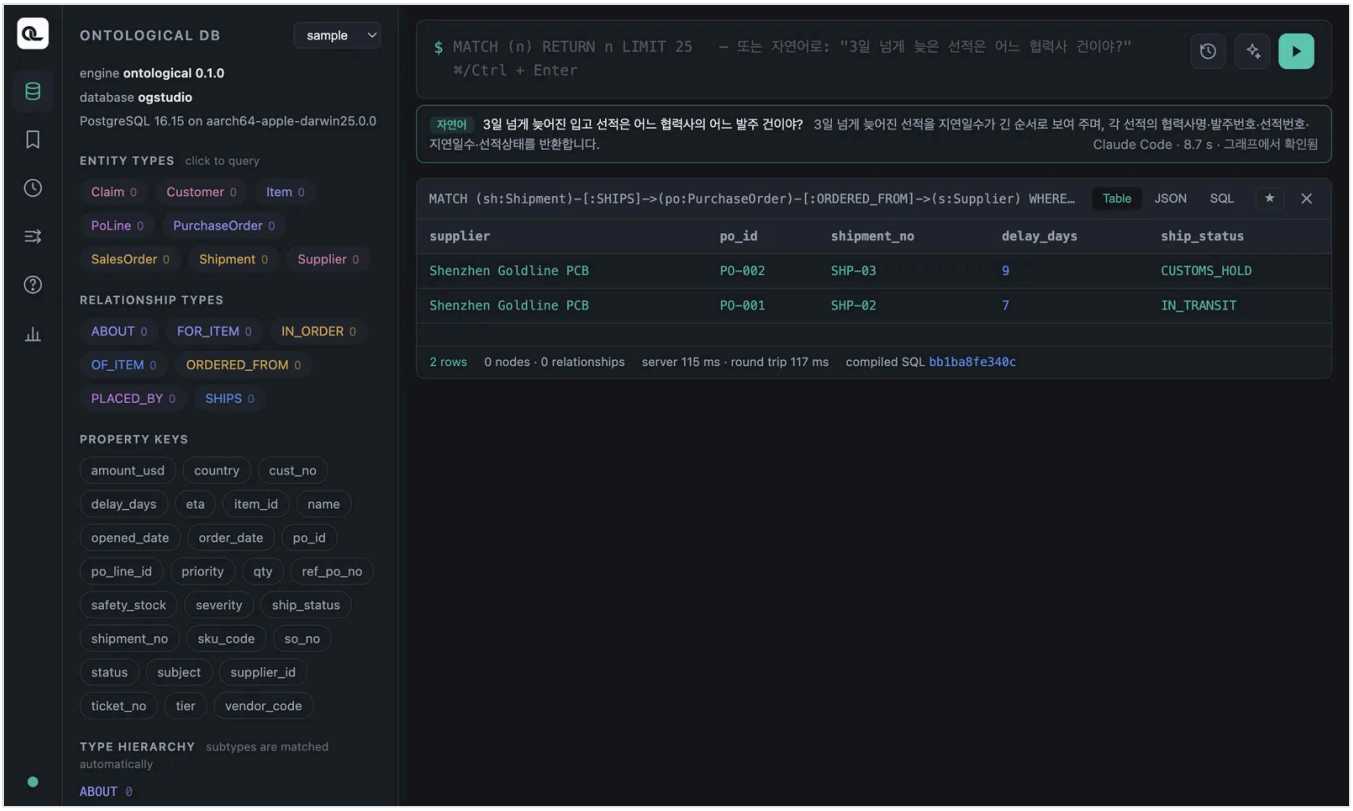
1. **ERP 안의 조인은 ERP에서** 했습니다(발주 ⋈ 협력사). 같은 DB 안의 일은 원천에 맡깁니다.
2. **WMS에는 조건을 붙여** 보냈습니다(`delay_days > 3`). 8행 전부가 아니라 2행만 넘어왔습니다.
3. 서로 다른 DB를 잇는 마지막 조인만 Ontological DB가 했습니다.

원천 데이터가 많을수록 이 차이가 커집니다. 조건이 원천으로 내려가지 않는 질의(예: 두 DB의 값을 서로 비교하는 조건)는 계획에서 `local`로 보입니다. 그런 질의는 원천에서 많은 행을 가져올 수 있으므로 조건을 조정해 보는 것이 좋습니다.

## 말로 묻기 — 자연어 → Cypher

입력칸에 Cypher 대신 **한국어 문장**을 쓰고 실행하면, 에이전트가 이 그래프의 스키마를 읽고 Cypher를 만들어 줍니다(첫 단어가 Cypher 키워드가 아니면 자연어로 인식합니다. ✨ 버튼이나 ⇧⌘Enter로도 됩니다).

```
3일 넘게 늦어진 입고 선적은 어느 협력사의 어느 발주 건이야?
```




*자연어 질의 — 에이전트가 만든 Cypher와 설명, 그래프에서 미리 실행해 확인했다는 표시*

에이전트가 만든 Cypher는 화면에 그대로 보이고, 실행하기 전에 그래프에서 **먼저 시험 실행**해 봅니다. 실패하면 엔진의 오류 메시지를 참고해 한 번 수정합니다. 쓰기 문장(`CREATE`, `SET` 등)은 실행하지 않고 입력칸에만 둡니다. 이 예에서는 Claude Code가 8.7초 만에 다음을 만들었습니다.

```
MATCH (sh:Shipment)-[:SHIPS]->(po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier)
WHERE sh.delay_days > 3
RETURN s.name AS supplier, po.po_id AS po_id, sh.shipment_no AS shipment_no,
       sh.delay_days AS delay_days, sh.ship_status AS ship_status
ORDER BY sh.delay_days DESC LIMIT 100
```

> **[팁] 자연어 질의를 활용해 보세요**
>
> 만들어진 Cypher를 읽고 고쳐 쓰는 연습을 하세요. 에이전트가 어떤 관계를 골랐는지 보면 스키마를 이해하는 데도 도움이 됩니다. 이 기능은 앱 안의 포털에서만 동작합니다(코딩 에이전트가 필요합니다).

## 직접 쓰기 — 내 그래프 만들기

가상 노드는 읽기 전용이지만, Ontological DB에 **저장되는 노드**는 Cypher로 직접 생성하고 수정할 수 있습니다. 예제를 건드리지 않도록 연습용 그래프를 만듭시다. 그래프 선택과 상관없이 실행됩니다.

```
CREATE GRAPH tut
```

이제 그래프 선택에서 `tut`을 고르고(목록에 안 보이면 포털을 새로 고침), 노드를 만듭니다.

```
CREATE (:Employee {emp_no: 'E1', name: '김하늘', dept: '구매', salary: 4200})
```

```
CREATE (:Employee {emp_no: 'E2', name: '박바다', dept: '물류', salary: 3900})
```

관계를 만들려면, 먼저 양쪽 노드를 `MATCH`로 찾은 뒤 `CREATE`합니다. 관계에도 속성을 붙일 수 있습니다.

```
MATCH (a:Employee {emp_no: 'E1'}), (b:Employee {emp_no: 'E2'})
CREATE (a)-[:MENTORS {since: '2025-03'}]->(b)
```

```
MATCH (a:Employee)-[m:MENTORS]->(b:Employee)
RETURN a.name, m.since, b.name
```

결과: `김하늘 · 2025-03 · 박바다`.

### 고치기·지우기

| 하고 싶은 일 | 문장 |
|---|---|
| 값 바꾸기 | `MATCH (e:Employee {emp_no:'E1'}) SET e.salary = 4500 RETURN e.salary` |
| 없으면 만들기 | `MERGE (d:Dept {name:'구매'}) RETURN d.name` |
| 없으면 관계 만들기 | `MATCH (e:Employee), (d:Dept {name:'구매'}) WHERE e.dept = d.name MERGE (e)-[:WORKS_IN]->(d)` |
| 노드와 그 관계 지우기 | `MATCH (e:Employee {emp_no:'E2'}) DETACH DELETE e` |

`MERGE`는 같은 모양이 이미 있으면 그것을 쓰고, 없으면 만듭니다. 두 번 실행해도 `Dept` 노드가 하나만 생깁니다. `DETACH DELETE`는 노드에 붙은 관계까지 함께 지웁니다(관계가 남아 있으면 `DELETE`만으로는 삭제할 수 없습니다).

> **[참고] 온톨로지 그래프는 Studio에게**
>
> `ontology` 그래프는 Studio가 관리합니다. 클래스 정의(`_OntologyClass` 같은 노드)와 액션 정의가 함께 들어 있으므로, 포털에서 직접 고치지 말고 「온톨로지 구축」 화면과 에이전트로 다루세요. 직접 쓰기 연습은 이렇게 따로 만든 그래프에서 합니다.

## 저장과 기록

- 결과 카드의 ★는 질의를 **저장**합니다. 왼쪽 레일의 책갈피 아이콘에서 다시 꺼냅니다.
- 입력칸에서 ↑↓로 **이전 질의**를 불러옵니다. 시계 아이콘 목록에는 이 그래프에서 최근에 수행한 쓰기 작업(감사 기록)가 보입니다.

## Ontological DB의 Cypher — 알아 둘 차이

Neo4j의 Cypher를 이미 안다면 다음 차이만 기억하세요.

| 항목 | Ontological DB 0.1 |
|---|---|
| 라벨 | 패턴의 노드마다 라벨을 적는 것이 원칙. `WHERE n:Label` 형태의 라벨 조건은 쓰지 않습니다. |
| 하위 질의 | `EXISTS { … }`, `CALL { … }`는 아직 없습니다 → `OPTIONAL MATCH … IS NULL` |
| 가변 길이 경로 | `-[:R*1..3]->`는 저장된 관계에서만 됩니다. 가상 관계에는 아직 안 됩니다. |
| 날짜 | `date()` 함수가 없습니다 → 날짜 컬럼과 `'YYYY-MM-DD'` 문자열을 바로 비교 |
| 함수 | 7장의 목록만. 없는 함수를 쓰면 오류가 지원 목록을 알려 줍니다. |
| 가상 노드 | 읽기 전용 — 값은 원천 시스템(또는 액션)으로 바꿉니다 |
| 더 있는 것 | `SHOW …` 패브릭 문장, `EXPLAIN FABRIC`, 시간 여행(10장), 상위 라벨로 하위 타입까지 찾기 |

> **[확인 문제]**
>
> 1. `EXPLAIN FABRIC`으로 "클레임 → 판매 주문 → 고객" 질의(8장)의 계획을 보세요. 이 질의는 몇 개의 원천에 접근하나요?
> 2. `tut` 그래프에 `Dept {name:'물류'}`를 `MERGE`로 두 번 만들고, `MATCH (d:Dept) RETURN d.name, count(*)`로 몇 개인지 확인하세요.
> 3. 자연어로 "납땜 불량 클레임을 낸 고객은 누구야?"라고 물어보고, 에이전트가 만든 Cypher가 8장의 답과 같은 결과를 내는지 비교하세요.
>
> **▼ 정답 보기**
>
> 1. 세 노드가 모두 CRM에 있으므로 원천 한 곳(`remote:crm`)에서 조인까지 끝납니다. 같은 DB 안의 일은 원천에 맡기는 것을 확인할 수 있습니다.
> 2. `MERGE`는 이미 있으면 새로 만들지 않으므로 `물류` 행의 개수는 1입니다(앞에서 만든 `구매` 도 1).
> 3. Tokyo Devices(SO-05, T-3 「PCB 납땜 불량」)가 나와야 합니다. 에이전트의 질의는 매번 조금씩 다를 수 있지만 결과는 같아야 합니다.

# 시간 여행 — 바이템포럴 질의

*분류: Cypher 익히기 · 10장 · 원문: https://www.uengine.org/docs/ontological-studio/time-travel.html*

> "그때 사실은 무엇이었나"와 "그때 우리는 무엇을 알고 있었나"는 다른 질문입니다. 두 개의 시간축을 이해하고, 같은 Cypher를 시점만 바꿔 다시 물어봅니다.

▶ 영상: 시간 여행 — 바이템포럴 질의 — 영상 3:58

## 왜 시간이 두 개 필요한가

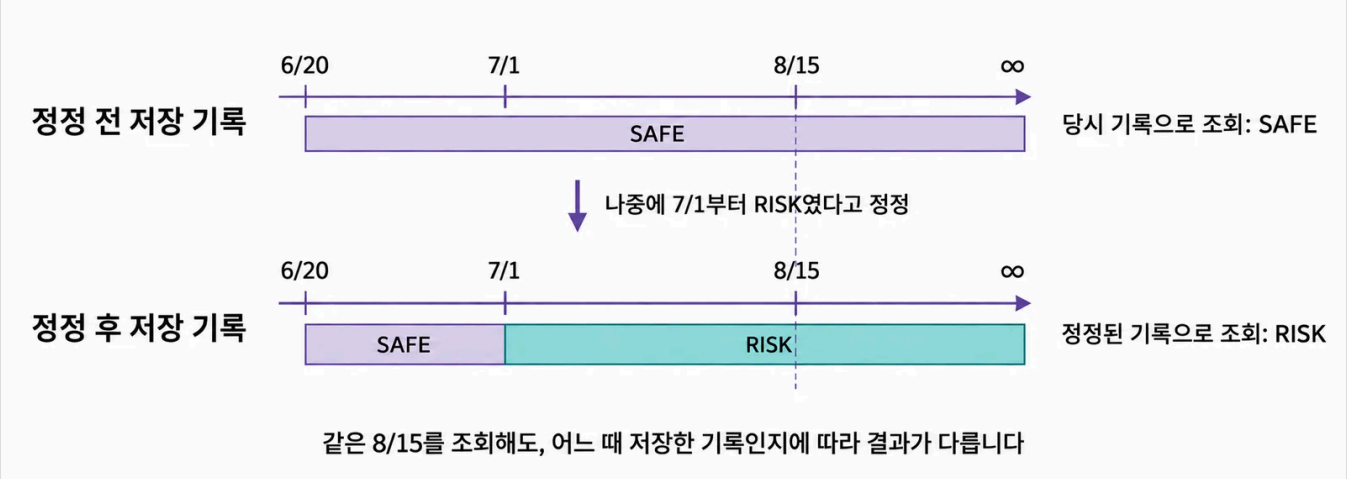
인사팀이 김하늘 씨의 연봉을 이렇게 기록했다고 합시다.

1. 2021년 1월부터 4,000 — 입사.
2. 2024년 1월부터 4,600 — 인상. **실제 연봉이 바뀐 것**(변경)입니다.
3. 몇 달 뒤, 2024년 인상액이 사실 4,700이었다는 것을 알고 고칩니다. 실제 연봉은 그대로였지만 **기록한 숫자가 틀렸던 것**(정정)입니다.

이제 감사에서 묻습니다. "2025년 6월의 연봉은?" 답은 4,700입니다. 그런데 "**정정하기 전에 낸 보고서**에는 왜 4,600이라고 적혀 있었나?"라고 물으면, 그 시점에 **우리가 알던** 값을 재현할 수 있어야 합니다.

| 시간축 | 묻는 것 | Ontological DB 설정 |
|---|---|---|
| **유효 시간** (valid time) | 세상에서 **언제** 그것이 사실이었나 | `valid_at` |
| **시스템 시간** (system time, 기록 시간) | 우리가 **언제부터** 그렇게 기록하고 있었나 | `system_at` (as known at) |

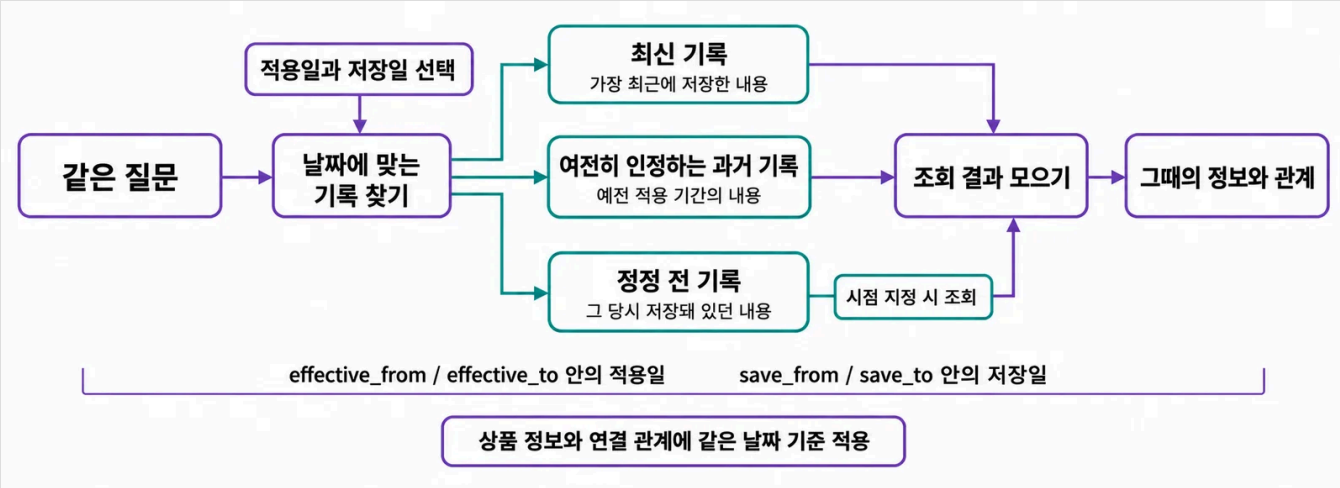
두 축을 모두 가진 데이터를 **바이템포럴**(bitemporal)이라고 합니다.




*같은 상품의 적용 기간과 저장 기간 — 정정 전 기록으로 보면 8월 15일은 SAFE, 7월 1일부터 RISK였다고 정정한 뒤의 기록으로 보면 RISK*

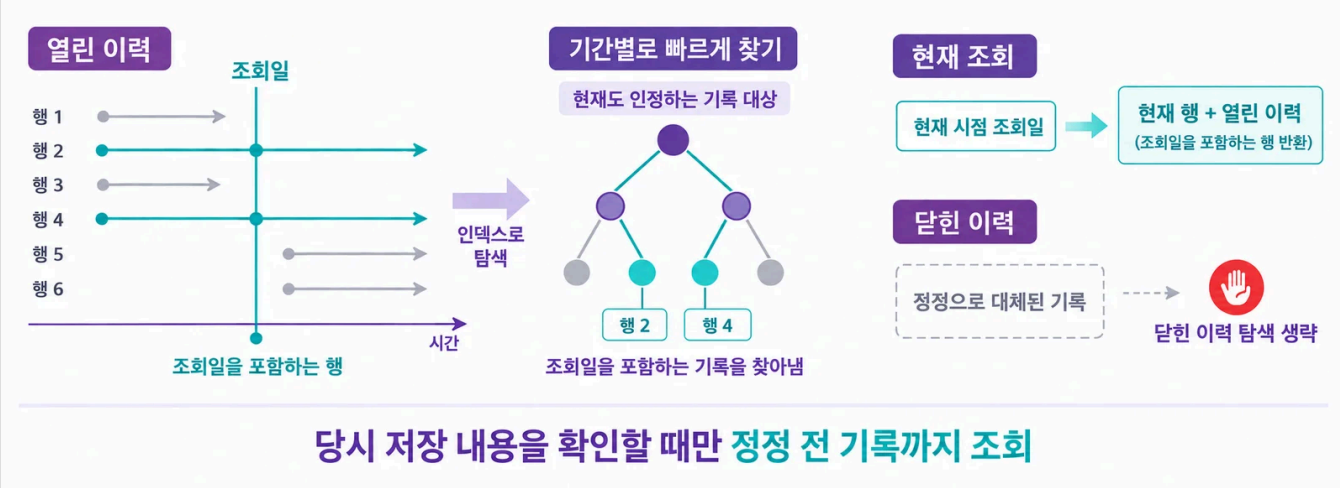
## 같은 질의, 다른 시점

Ontological DB에서는 시점을 **질의 문장과 별도로 설정합니다.** Cypher는 그대로 두고, "언제를 기준으로 볼지"만 바꿉니다.




*시점 조회 흐름 — 같은 질문에 적용일과 저장 기준을 고르면, 그 날짜에 맞는 기록(현재·과거·정정 전)을 찾아 그때의 정보와 관계로 답합니다*

- 관계에도 **같은 시점**을 적용합니다. 2022년 기준으로 물으면 2022년에 있던 관계만 따라갑니다.
- 현재를 물을 때는 현재 행만 읽고, 정정되기 전의 기록은 시점을 지정했을 때만 찾습니다. 그래서 이력이 많아도 평소 질의는 느려지지 않습니다.



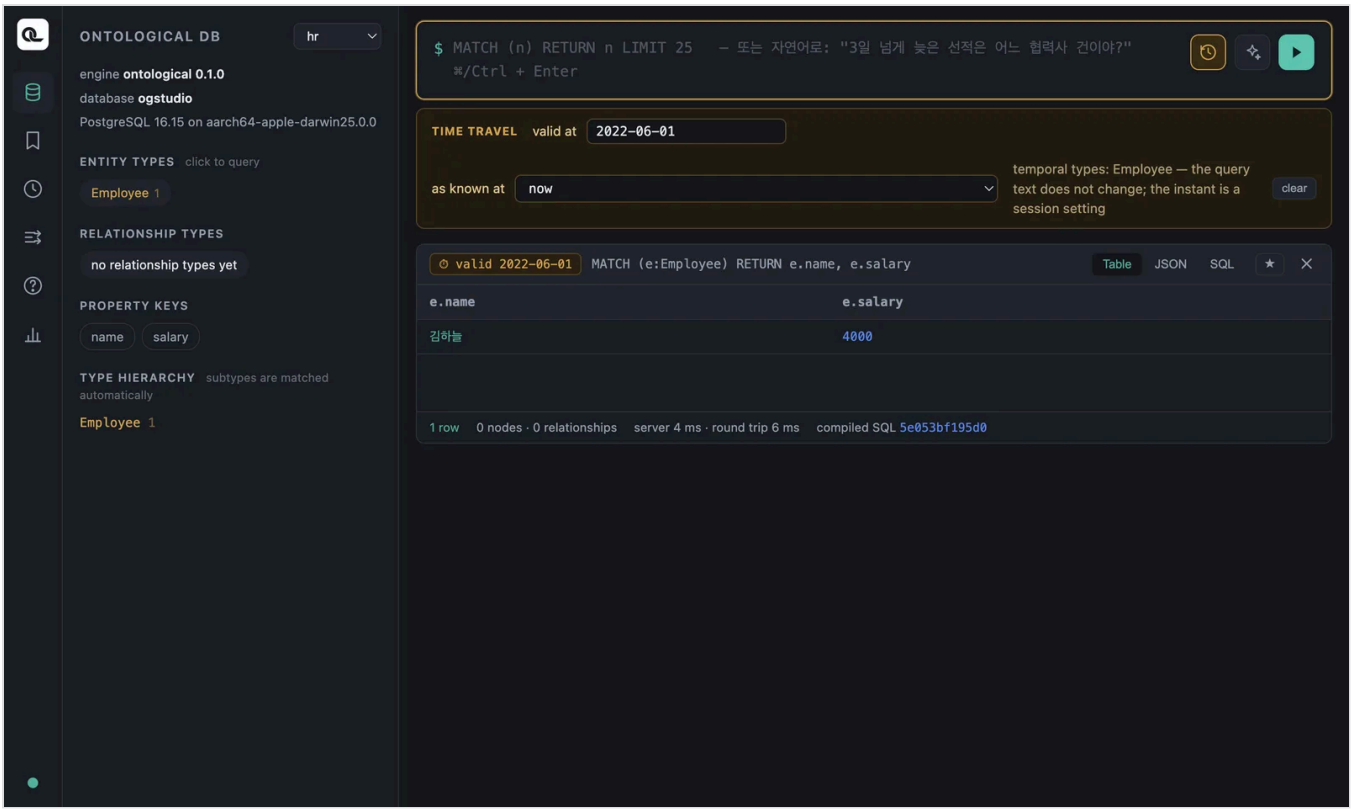

*빠른 시점 검색 — 현재 조회는 현재 행과 열린 이력만, 정정으로 대체된 닫힌 이력은 시점을 지정한 조회에서만 탐색*

## 포털의 시간 여행 막대

포털 입력칸 오른쪽의 시계 버튼(⏱)을 누르면 **TIME TRAVEL** 막대가 열립니다.

| 칸 | 뜻 |
|---|---|
| valid at | 유효 시간. `2022-06-01` 처럼 날짜를 적습니다. |
| as known at | 시스템 시간. `now` 또는 이 그래프에 최근 일어난 쓰기의 "직후" 시각을 목록에서 고릅니다. |

시점을 정하면 입력칸 테두리가 노랗게 바뀌고, 결과 카드 위에 `⏱ valid 2022-06-01` 처럼 시점이 붙습니다. 해제하려면 「clear」를 누릅니다.




*시간 여행 막대 — valid at 2022-06-01로 같은 질의를 실행하면 그때의 연봉 4000이 나옵니다*

> **[참고] 시간 버전은 타입마다 켭니다**
>
> 이력을 남길 노드·관계 타입마다 시간 버전을 켜야 합니다. 시간 막대 오른쪽에 `temporal types: Employee` 처럼 켜진 타입이 보입니다. 켜지 않은 타입은 시점을 정해도 현재 값을 답합니다. 예제의 가상 노드(원천 DB를 비춘 것)는 원천이 이력을 갖고 있지 않으므로 시간 여행의 대상이 아닙니다.

## 따라 하기 (고급, 선택) — 연봉 이력 만들기

시간 버전을 켜고 "언제부터"를 지정해 쓰는 일은 현재 버전에서는 **SQL 세션**에서 합니다. 데이터베이스 수업에서 `psql`을 써 봤다면 따라 해 보세요. (macOS 기준. 건너뛰어도 다음 장을 진행하는 데 문제없습니다.)

**1. 접속 정보 찾기.** 앱 데이터 폴더(`~/Library/Application Support/Ontological Studio/`)의 `ports.json`에서 `pg` 포트를, `secrets.json`에서 `pgPassword`를 봅니다. 사용자는 `og`, 데이터베이스는 `ogstudio`입니다.

```
APP="/Applications/Ontological Studio.app/Contents/Resources/runtime/pg/bin/psql"
DATA="$HOME/Library/Application Support/Ontological Studio"
PORT=$(python3 -c "import json;print(json.load(open('$DATA/ports.json'))['pg'])")
export PGPASSWORD=$(python3 -c "import json;print(json.load(open('$DATA/secrets.json'))['pgPasswor
d'])")
"$APP" -h 127.0.0.1 -p $PORT -U og -d ogstudio
```

> **[주의] 관리자 계정입니다**

> `og` 는 이 앱의 데이터베이스 관리자입니다. 아래 예제 외의 문장은 실행하지 마세요. 실수로 예제를 망가뜨렸다면 1장의 방법으로 처음 상태로 되돌릴 수 있습니다.

**2. 그래프와 타입을 만들고 시간 버전을 켭니다.**

```
SELECT og_create_graph('hr');
SELECT og_create_type('hr', 'Employee', 'entity');
SELECT og_add_property('hr', 'Employee', 'name', 'string'),
       og_add_property('hr', 'Employee', 'salary', 'int');
SELECT og_enable_temporal('hr', 'Employee');
```

**3. 세 번 기록합니다.** `SET LOCAL og.valid_from`이 "언제부터의 사실인가"입니다. 트랜잭션이 끝나면 사라지므로 기록마다 따로 줍니다.

```
BEGIN;  -- 2021년 입사
SET LOCAL og.valid_from = '2021-01-01';
SELECT og_cypher('hr', $$CREATE (:Employee {name: '김하늘', salary: 4000})$$);
COMMIT;

BEGIN;  -- 2024년부터 인상 (변경)
SET LOCAL og.valid_from = '2024-01-01';
SELECT og_cypher('hr', $$MATCH (e:Employee {name: '김하늘'}) SET e.salary = 4600$$);
COMMIT;

BEGIN;  -- 4600이 아니라 4700이었다 (정정)
SET LOCAL og.valid_from = '-infinity';
SELECT og_cypher('hr', $$MATCH (e:Employee {name: '김하늘'}) SET e.salary = 4700$$);
COMMIT;
```

정정의 `'-infinity'`가 핵심입니다. "유효 구간(2024년부터)은 그대로 두고 값만 고친다"는 뜻입니다. 이것을 `'2024-01-01'`로 주면 또 하나의 변경이 되어 버립니다.

**4. 포털에서 물어봅니다.** 포털을 새로 고치고 그래프를 `hr`로 바꾼 뒤, 질의는 언제나 같습니다.

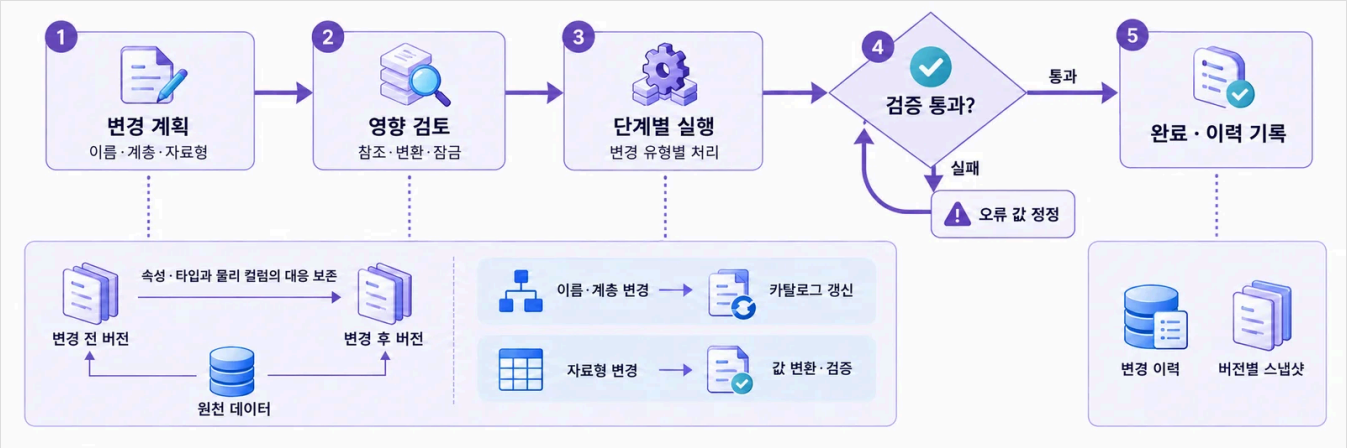
```
MATCH (e:Employee) RETURN e.name, e.salary
```

| valid at | as known at | 결과 | 해석 |
|---|---|---|---|
| (비움) | now | 4700 | 지금 알고 있는 지금의 연봉 |
| 2022-06-01 | now | 4000 | 2022년의 연봉 |
| 2025-06-01 | now | 4700 | 정정까지 반영한 2025년의 연봉 |
| 2025-06-01 | "…SET e.salary = 4600" 직후 | 4600 | **정정 전 보고서**가 알던 2025년의 연봉 |
| 2020-06-01 | now | (없음) | 입사 전에는 직원이 아니었다 |

네 번째 결과에서 바이템포럴의 장점을 확인할 수 있습니다. 값을 덮어쓰는 보통 데이터베이스에서는 "그때 왜 그렇게 판단했나"를 다시 볼 수 없습니다.

## 스키마도 이력을 남깁니다

데이터뿐 아니라 스키마도 이력을 남깁니다. 속성 타입을 바꾸는 등의 **스키마 변경**도 계획 → 영향 검토 → 단계별 실행 → 검증 → 완료의 단계를 밟고 이력이 남습니다. 변환할 수 없는 값이 하나라도 있으면 완료되지 않습니다. 포털의 스키마 디자이너에서 변경을 계획하고 진행 단계와 이력을 볼 수 있습니다.




*스키마 변경 단계 — 계획, 영향 검토, 단계별 실행, 검증, 완료와 이력 기록. 검증에 실패하면 오류 값을 정정해 다시 검증합니다*

더 깊은 내용은 Ontological DB 소개를 참고하세요.

> **[확인 문제]**
>
> 1. 다음은 변경일까요, 정정일까요? ⓐ 7월 1일부터 협력사 등급이 1에서 2로 내려갔다. ⓑ 협력사 국가를 Korea로 잘못 적었는데 사실 Vietnam이었다.
> 2. "2023년 감사 보고서를 낼 때 우리가 알던 2022년 연봉"을 물으려면 두 칸을 각각 무엇으로 정해야 하나요?
> 3. 시간 버전을 켜지 않은 타입에 `valid_at` 을 주고 질의하면 어떻게 될까요?
>
> **▼ 정답 보기**
>
> 1. ⓐ 변경 — 실제 등급이 바뀌었으므로 `valid_from = '2026-07-01'`. ⓑ 정정 — 처음부터 Vietnam이었으므로 `valid_from = '-infinity'` 로 값만 고칩니다.
> 2. valid at = 2022년의 날짜, as known at = 2023년 감사 보고서를 낸 시각(그 직전의 쓰기).
> 3. 시점과 상관없이 현재 값을 반환합니다. 이력을 남기지 않았으니 돌아갈 과거가 없습니다.

# 골든 퀘스천과 골든 커맨드

*분류: 온톨로지 구축 · 11장 · 원문: https://www.uengine.org/docs/ontological-studio/golden.html*

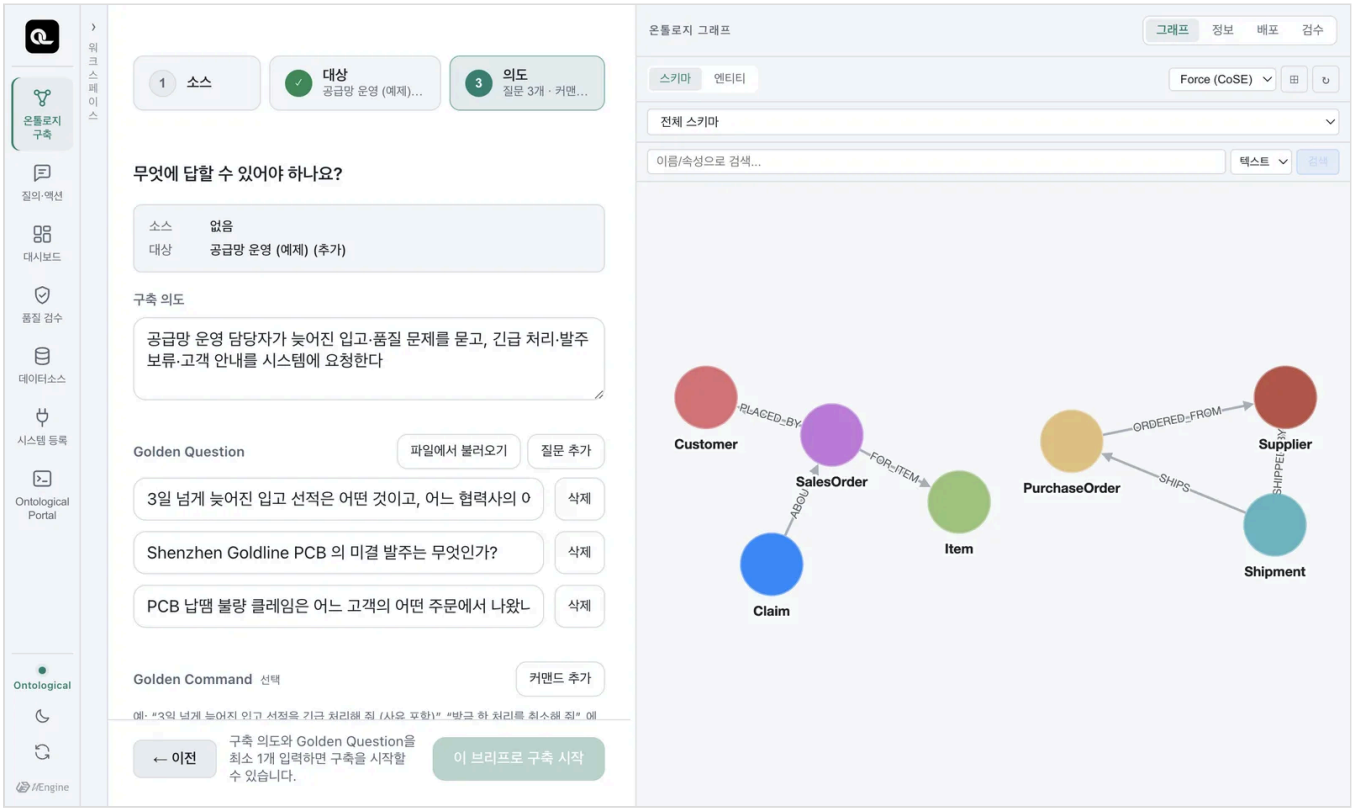
> 온톨로지를 만들기 전에 먼저 정하는 두 가지 — 이 온톨로지가 **답해야 할 질문**과 **해야 할 일**. 구축 에이전트는 이것을 기준으로 설계하고, 끝나면 이것으로 검증합니다.

▶ 영상: 골든 퀘스천과 골든 커맨드 — 영상 3:28

## 무엇을 위한 온톨로지인가

온톨로지에 모든 정보를 담으려 하면 구축 범위가 지나치게 넓어집니다. 그래서 Ontological Studio는 구축을 시작하기 전에 세 가지를 적게 합니다.

| 항목 | 질문 | 예제「공급망 운영」 |
|---|---|---|
| **구축 의도** | 누가, 무엇을 하려고 쓰나? | 공급망 운영 담당자가 늦어진 입고·품질 문제를 묻고, 긴급 처리·발주 보류·고객 안내를 시스템에 요청한다 |
| **골든 퀘스천** (Golden Question) | 이 온톨로지로 반드시 **답해야** 할 질문 | 3일 넘게 늦어진 입고 선적은 어떤 것이고, 어느 협력사의 어느 발주 건인가? |
| **골든 커맨드** (Golden Command) | 이 온톨로지로 반드시 **할 수 있어야** 할 지시 | 3일 넘게 늦어진 입고 선적을 긴급 처리해 줘 (사유: 생산 차질 우려) |



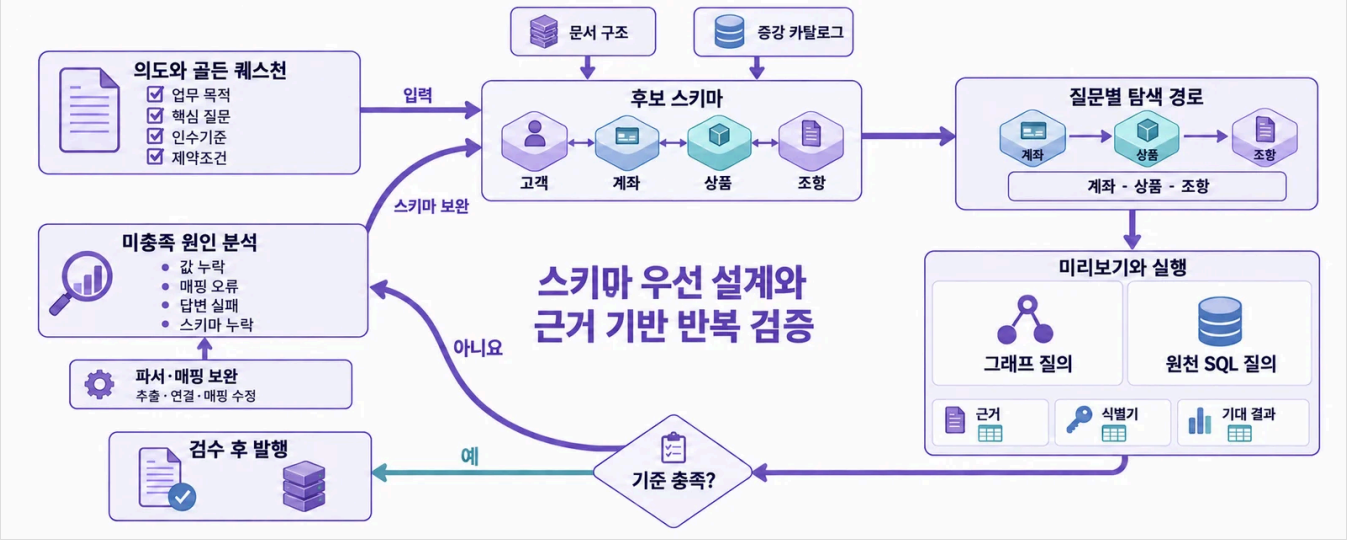

*구축 브리프 — 구축 의도, 골든 퀘스천 3개, 골든 커맨드 4개*

> **[팁] 예제 브리프는 어디서 보나요?**
>
> 「온톨로지 구축」의 대상 선택에서 「**공급망 운영 (예제) (추가 인제스천)**」을 고르면 의도 단계에 예제 브리프(골든 퀘스천 3개, 골든 커맨드 4개)가 채워져 보입니다. 「새 온톨로지」를 고르면 빈 브리프에서 시작합니다.

## 골든 퀘스천 — 설계의 출발점, 완성의 기준

골든 퀘스천은 소프트웨어 공학의 **인수 기준**(acceptance criteria)과 같습니다.

1. **설계할 때** — 에이전트는 질문에서 거꾸로 필요한 것을 찾습니다. "어느 협력사의 어느 발주"라는 말에서 `Shipment`, `PurchaseOrder`, `Supplier` 클래스와 그 사이의 관계가 필요하다는 것을 압니다. 이것을 **스키마 우선 설계**라고 합니다.
2. **검증할 때** — 구축이 끝나면 에이전트가 골든 퀘스천을 직접 실행해, 기대한 답과 근거가 나오는지 확인합니다. 안 나오면 스키마나 매핑을 고쳐 다시 실행합니다.
3. **쓸 때** — 질의 화면과 온톨로지 앱에 예시 질문으로 붙습니다.




*스키마 우선 설계와 근거 기반 반복 검증 — 의도와 골든 퀘스천에서 후보 스키마와 질문별 탐색 경로를 만들고, 실행 결과의 미충족 원인을 분석해 보완합니다*

### 좋은 골든 퀘스천 쓰기

| 덜 좋은 예 | 더 좋은 예 | 이유 |
|---|---|---|
| 협력사 정보를 알려 줘 | Shenzhen Goldline PCB의 미결 발주는 무엇인가? | 대상과 조건이 구체적이어야 검증할 수 있습니다 |
| 품질이 어때? | PCB 납땜 불량 클레임은 어느 고객의 어떤 주문에서 나왔나? | 답의 모양(고객, 주문)이 정해져 있습니다 |
| 선적, 발주, 협력사 | 3일 넘게 늦어진 입고 선적은 어떤 것이고, 어느 협력사의 어느 발주 건인가? | 여러 관계를 따라가는 질문으로 필요한 스키마 경로를 확인합니다 |

- 실제 업무 담당자가 **입으로 하는 질문**으로 씁니다.
- 한 개의 관계로 끝나는 질문만 있으면 스키마가 얕아집니다. **두세 관계를 거치는 질문**을 섞으세요.
- 답이 하나로 정해지는 질문이 좋습니다. 검증이 쉬워집니다.

## 골든 커맨드 — 액션 설계의 출발점

골든 커맨드는 **행위**에 대한 인수 기준입니다. 구축 에이전트는 골든 커맨드마다:

1. 등록된 업무 시스템의 API(OpenAPI 스펙)를 살펴 이 지시를 **실행할 API**를 찾고,
2. 그 일을 **되돌릴 API**(보상)를 짝짓고,
3. 누가 어떤 대상에 쓸 수 있는지 **정책**을 붙여 **액션**을 만들고,
4. 실제로 실행하지 않고 계획만 세워 보는 **드라이런**으로 검증합니다.

예제의 골든 커맨드 네 개와 그로부터 만들어진 액션:

| 골든 커맨드 | 액션 | 실행 (보상) |
|---|---|---|
| 3일 넘게 늦어진 입고 선적을 긴급 처리해 줘 (사유: 생산 차질 우려) | `Shipment.expediteShipment` | WMS 우선순위 URGENT (이전 값으로) → ERP 납기 독촉 (철회) |
| Shenzhen Goldline PCB의 미결 발주를 모두 보류해 줘 (사유: PCB 납땜 불량 조사) | `PurchaseOrder.holdPurchaseOrder` | ERP 발주 보류 (해제) |
| PCB 납땜 불량 클레임을 낸 고객에게 조치 안내를 보내 줘 | `Customer.notifyCustomer` | CRM 안내 발송 (철회) |
| 방금 한 처리를 취소해 줘 | (별도 액션 없음) | 위 액션들의 **보상 흐름**으로 충족 |

마지막 줄의 "취소" 커맨드는 새 액션을 만들지 않습니다. 기존 액션에 정의된 보상을 사용하기 때문입니다.

### 좋은 골든 커맨드 쓰기

- **대상을 고르는 조건**을 담습니다: "3일 넘게 늦어진", "Shenzhen Goldline PCB의 미결".
- **사유**를 함께 씁니다. 실행 기록과 업무 시스템에 남습니다.
- 되돌리는 지시("방금 한 처리를 취소해 줘")도 하나 넣어 두면, 에이전트가 보상까지 검증합니다.

## 두 개념을 한 장으로

| | 골든 퀘스천 | 골든 커맨드 |
|---|---|---|
| 묻는 것 | 무엇에 **답해야** 하나 | 무엇을 **해야** 하나 |
| 만드는 것 | 클래스·관계·속성(스키마), 인스턴스 | 액션(단계·보상·정책) |
| 재료 | 문서, 데이터소스 | 등록된 업무 시스템의 API |
| 검증 | 질문을 실행해 답과 근거 확인 | 드라이런으로 계획·정책 확인 |
| 앱에서 | 대시보드·코드 앱이 **무엇을 보여 줄지** | 앱의 버튼이 **무엇을 할 수 있을지** |

> **[확인 문제]**
>
> 여러분 학과의 **수강 신청 시스템**을 온톨로지로 만든다고 생각해 봅시다.
>
> 1. 구축 의도를 한 문장으로 써 보세요.
> 2. 골든 퀘스천 세 개를 써 보세요. 적어도 하나는 두 개 이상의 관계를 거쳐야 합니다.

3. 골든 커맨드 두 개를 쓰고, 각각을 되돌리는 방법을 적어 보세요.

**▼ 예시 답안**

1. 학과 조교가 수강 현황과 정원 문제를 묻고, 대기자 승급과 분반 안내를 학사 시스템에 요청한다.
2. 「데이터베이스」 과목의 분반별 신청 인원과 정원은? / 정원을 넘긴 분반을 신청한 학생 중 졸업 예정자는 누구인가? / 김교수가 담당하는 과목 중 강의실 수용 인원보다 신청자가 많은 것은?
3. “정원이 남은 분반의 대기자 1순위를 승급해 줘” — 보상: 승급 취소(대기 1순위로 되돌림). “폐강 과목 신청자에게 안내 메일을 보내 줘” — 보상: 안내 철회 메일.

# 문서로 온톨로지 구축하기

*분류: 온톨로지 구축 · 12장 · 원문: https://www.uengine.org/docs/ontological-studio/build.html*

> 협력사 품질 보고서 한 장을 올려, 에이전트가 클래스와 관계를 설계하고 사실을 인스턴스로 옮기고 골든 퀘스천으로 검증하는 과정을 따라갑니다. 마지막에 발행해서 질의에 쓸 수 있게 합니다.

▶ 영상: 문서로 온톨로지 구축하기 — 영상 4:03

## 준비 — 이 장에 필요한 것

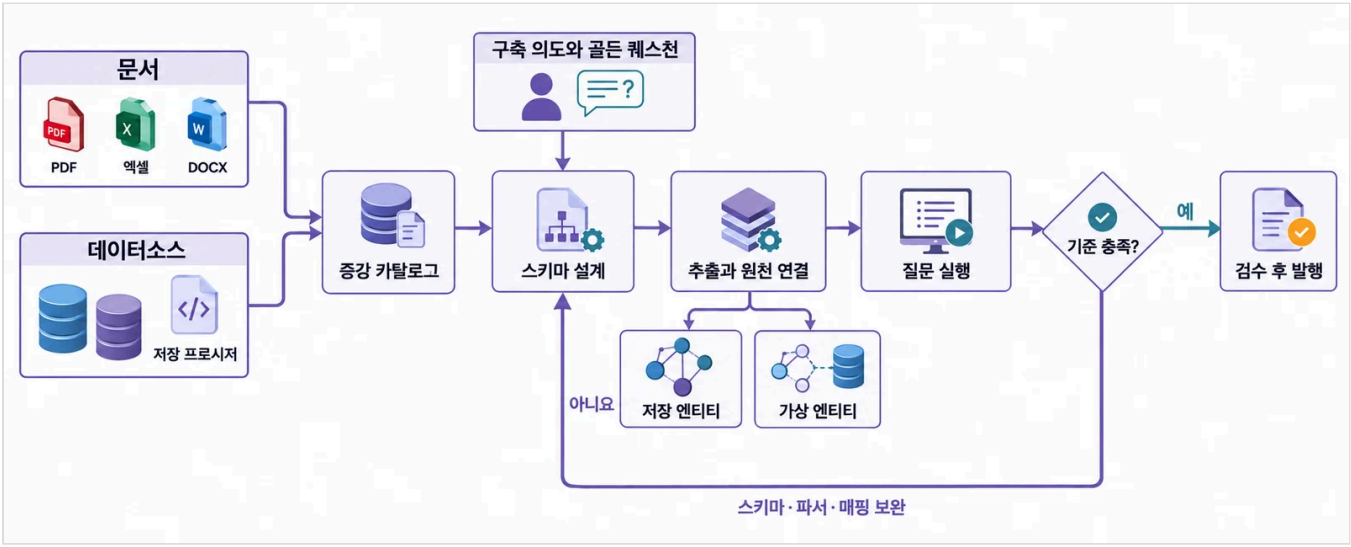
- 1장의 **코딩 에이전트**(Claude Code 또는 Codex)가 연결돼 있어야 합니다.
- 실습 문서: 협력사 품질 현황 — 2026년 9월 (마크다운 한 장). 내려받아 두세요.

문서에는 이런 내용이 문장으로 적혀 있습니다.

> **골드라인 PCB** — 중국 선전에 있는 1차 협력사. PCB 기판을 공급한다. 담당자는 구매팀 김민수 대리. … 완제품 **스마트 컨트롤러 SC-100**은 PCB 기판, 마그넷 코어, 전원 케이블 세 부품으로 조립한다. … **QI-0903** (9월 3일, 심각도 높음) — 골드라인 PCB가 납품한 PCB 기판에서 납땜 불량이 발견됐다. …

사람은 문서를 읽고 내용을 이해할 수 있습니다. 하지만 "납땜 불량이 난 부품은 어느 완제품에 들어가고, 그 협력사의 담당자는 누구인가?"라고 물으면, 문서 곳곳의 정보를 직접 연결해야 합니다. 이 장에서는 이러한 정보를 온톨로지로 구성합니다.

## 구축의 전체 흐름

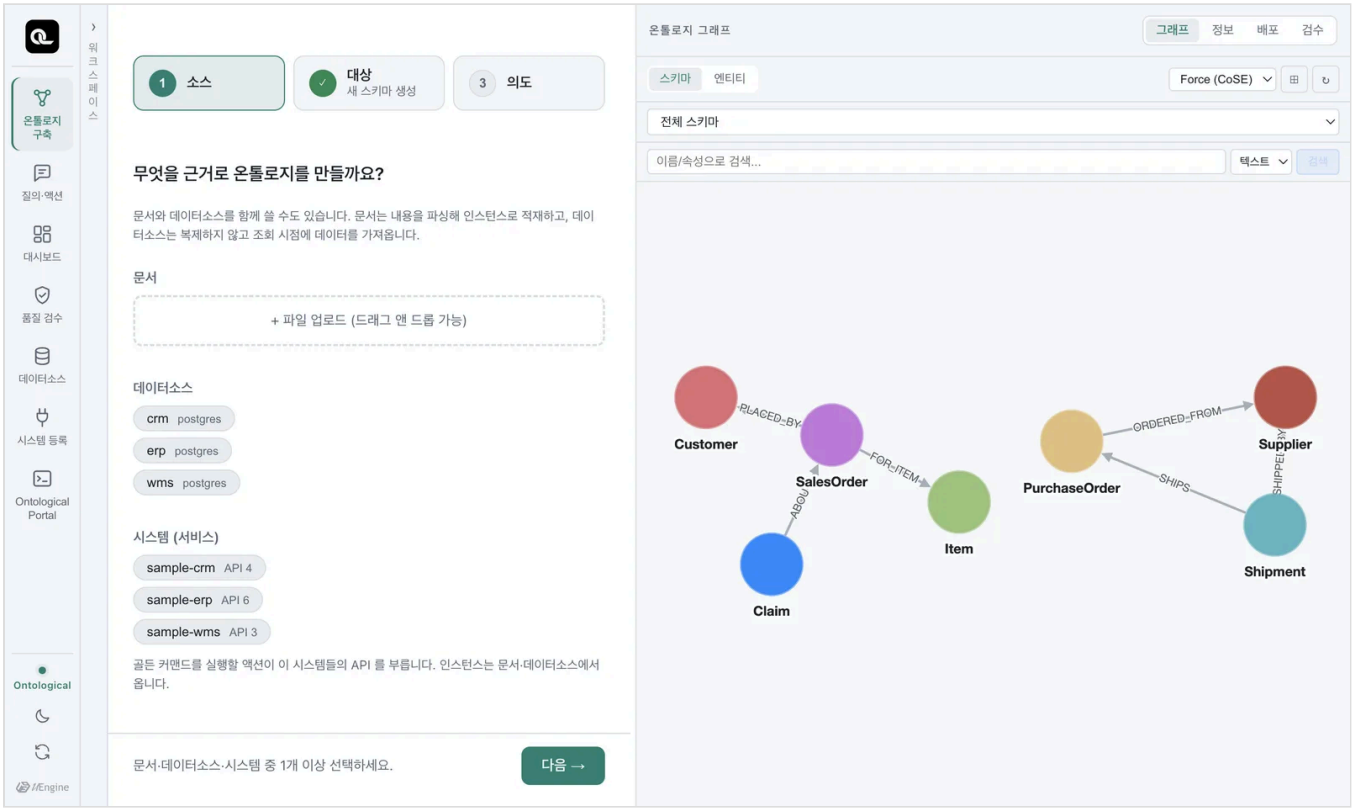



*온톨로지 구축의 전체 흐름 — 문서·데이터소스와 구축 의도·골든 퀘스천에서 출발해 증강 카탈로그, 스키마 설계, 추출과 원천 연결, 질문 실행을 거쳐 기준을 충족하면 검수 후 발행합니다*

| 단계 | 에이전트가 하는 일 | 사람이 하는 일 |
|---|---|---|
| 소스 | — | 문서를 올리고, 데이터소스·시스템을 고른다 |
| 의도 | 문서를 읽고 구축 의도와 골든 퀘스천 **초안**을 쓴다 | 초안을 읽고 고친다, 질문을 더한다 |
| 설계 | 질문에서 필요한 클래스·관계·속성을 정한다 | (지켜본다) |
| 추출·적재 | 문서의 사실을 인스턴스와 관계로 옮긴다 | (지켜본다) |
| 검증 | 골든 퀘스천을 직접 실행해 답이 나오는지 본다 | 보고서를 읽는다 |
| 검수·발행 | — | 품질 검수 항목을 판단하고, 발행을 결정한다 |

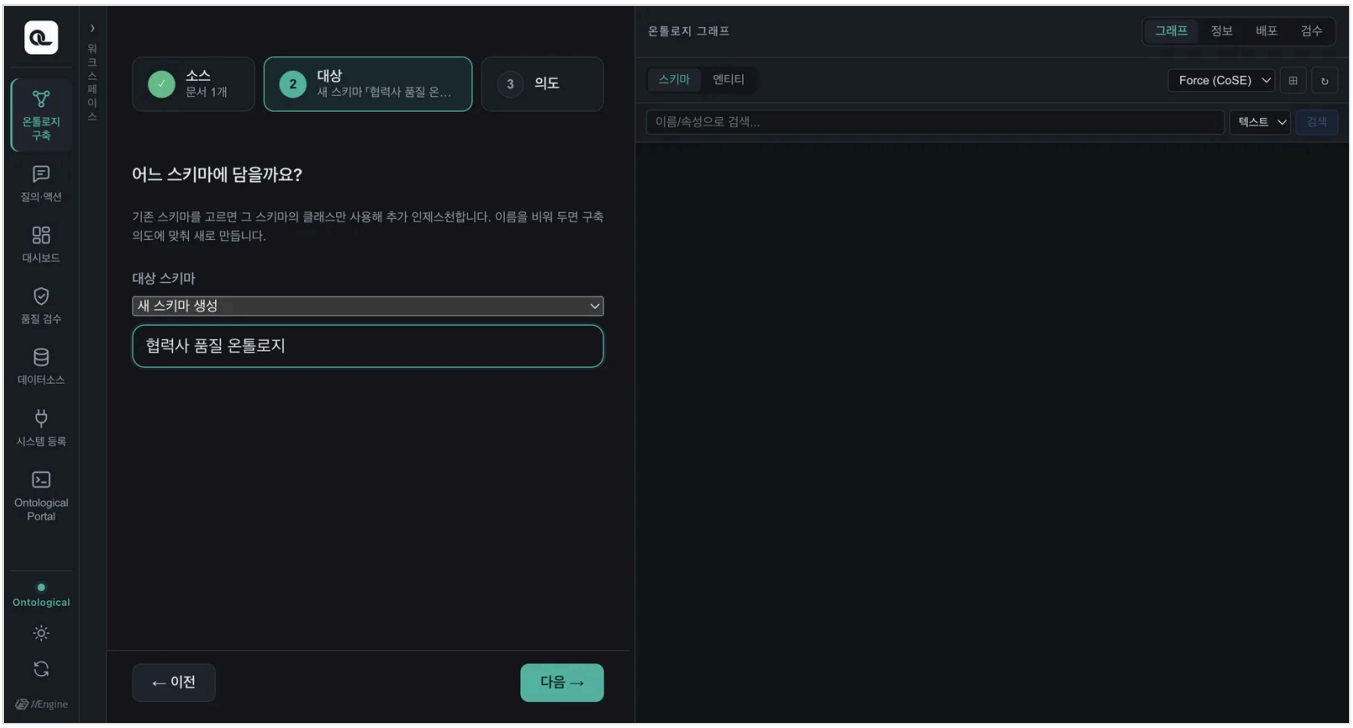
## 1단계 — 소스와 대상

홈의 「온톨로지 구축」 카드를 엽니다. 왼쪽이 구축 브리프, 오른쪽이 온톨로지 그래프입니다.




*구축의 첫 단계 「소스」 — 문서 업로드, 데이터소스, 시스템(서비스)을 근거로 고릅니다. 오른쪽은 이미 있는 예제 온톨로지의 스키마*

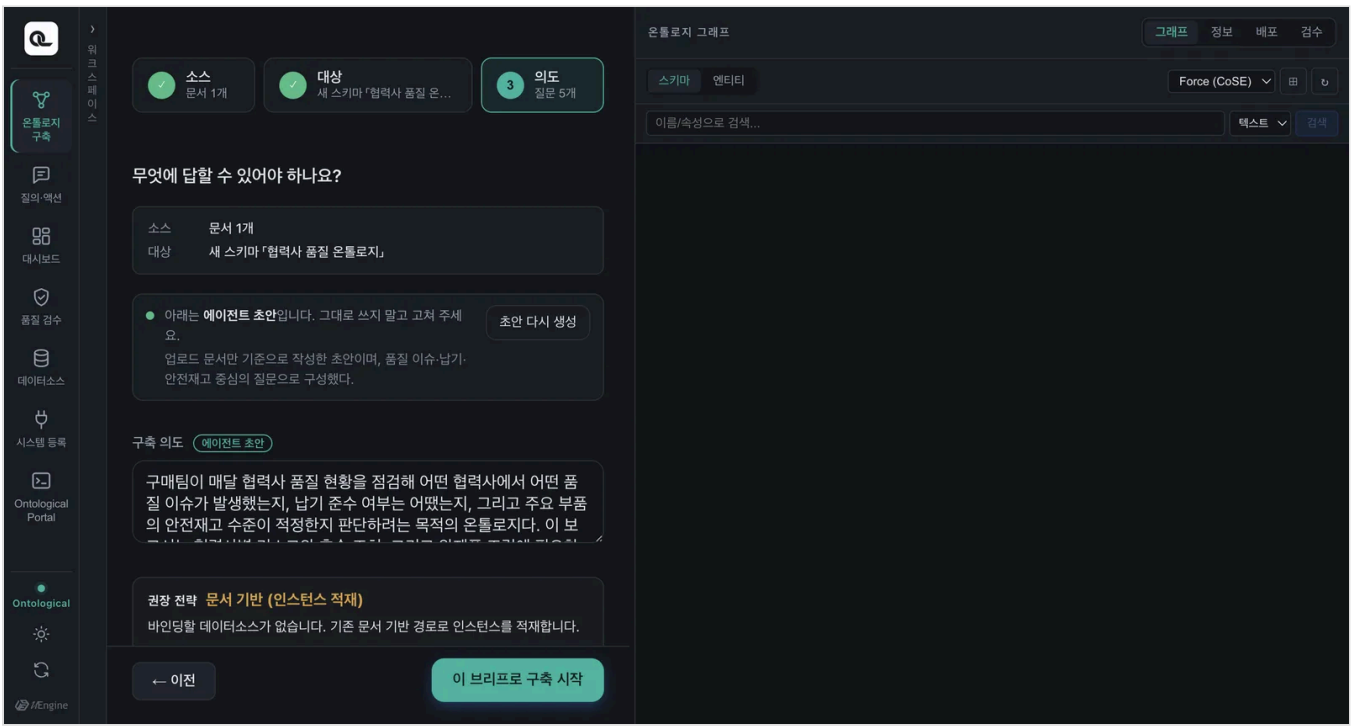
1. **소스**: 「파일 업로드」에 실습 문서를 끌어다 놓습니다. 이번에는 문서 하나만 씁니다. (데이터소스·시스템도 함께 고를 수 있습니다 — 문서는 내용을 파싱해 인스턴스로 적재하고, 데이터소스는 복제하지 않고 조회 시점에 가져옵니다.)
2. **대상**: 「새 스키마 생성」을 고르고 이름을 `협력사 품질 온톨로지`로 적습니다. 이미 있는 온톨로지에 더하려면 목록에서 고릅니다(추가 인제스천).




*문서를 올리고 새 스키마 이름을 적은 모습 (시연 영상의 한 장면, 밤 테마)*

## 2단계 — 의도와 골든 퀘스천

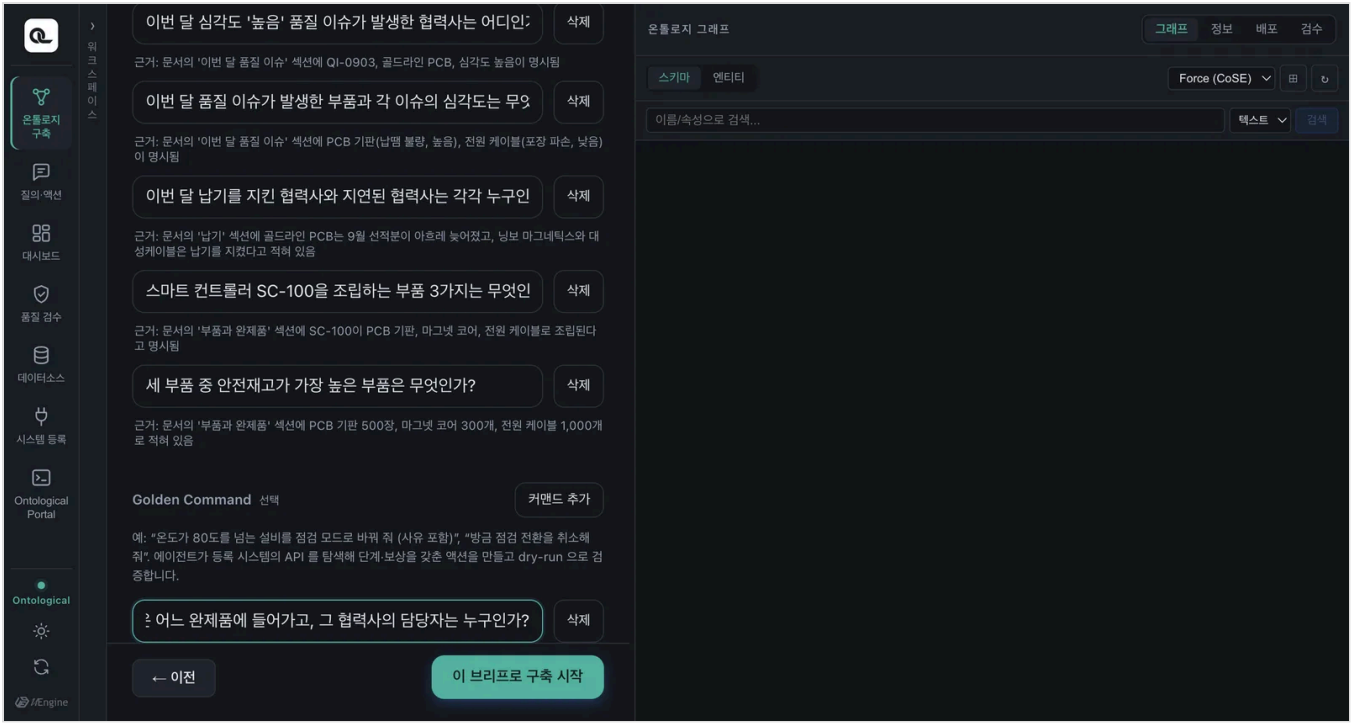
「의도」로 넘어가면 에이전트가 문서를 먼저 읽고 **구축 의도와 골든 퀘스천 초안**을 써 줍니다(실측 약 50초). 「초안」 표시가 붙어 있으니, 읽고 고친 뒤 확정합니다.




*에이전트가 쓴 구축 의도와 골든 퀘스천 초안 (밤 테마)*

여기에 정말 묻고 싶은 질문을 하나 더 넣습니다.

납땜 불량이 난 부품은 어느 완제품에 들어가고, 그 협력사의 담당자는 누구인가?



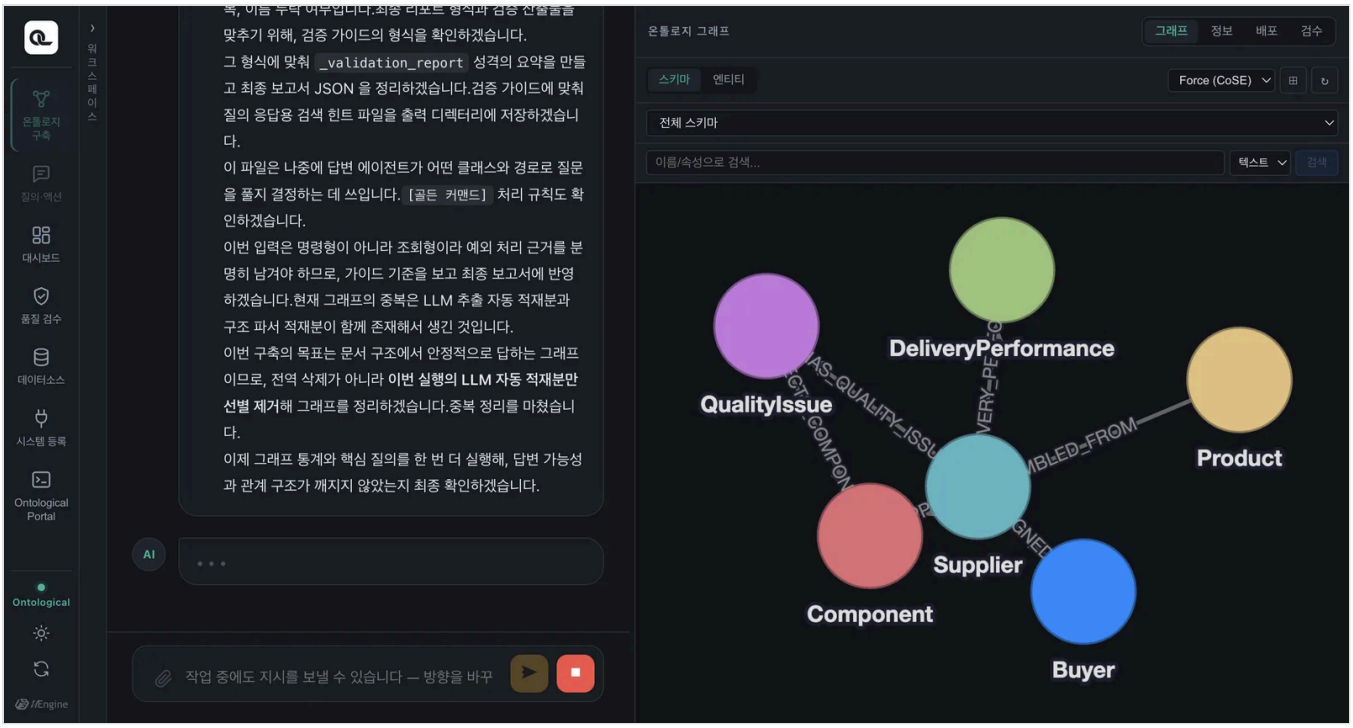

*골든 퀘스천을 추가한 브리프 (밤 테마)*

이 질문에 답하려면 품질 이슈 → 부품 → 완제품과 부품 → 협력사 → 담당자 관계를 따라가야 합니다. 이런 질문이 있어야 에이전트가 그 경로를 스키마에 만듭니다(11장).

## 3단계 — 구축

「이 브리프로 구축 시작」을 누릅니다. 에이전트가 일하는 과정이 대화창에 도구 호출로 흐릅니다. 화면에 보이는 도구 호출 하나하나가 실제로 일어난 작업입니다(실측 약 6분).

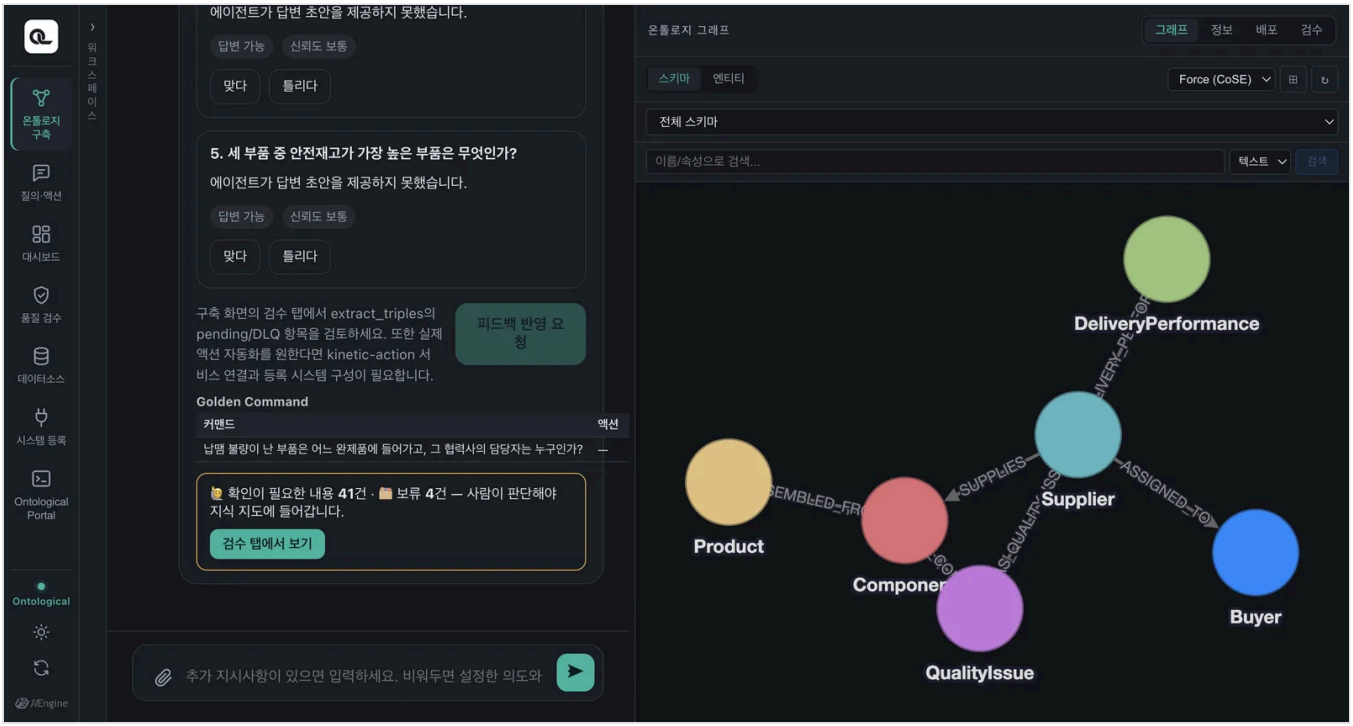
> **[참고] 에이전트가 "병합할까요, 분리할까요?"라고 물으면**
>
> 같은 그래프에 예제 「공급망 운영」의 `Supplier` 처럼 **이름이 같은 클래스**가 이미 있으면, 에이전트가 구축 중에 기존 클래스와 **병합**할지 별도 클래스로 **분리**할지 묻습니다. 이 장을 따라 할 때는 「분리」라고 답하세요. 예제 데이터를 건드리지 않고 이 장의 스키마만 따로 만듭니다(에이전트가 `QualitySupplier` 처럼 다른 이름을 쓸 수 있습니다 — 6단계의 질의는 포털의 ENTITY TYPES에서 실제 이름을 확인한 뒤 씁니다).




*구축 중 — 설계가 잡히면 오른쪽에 개념의 지도가 그려집니다 (밤 테마)*

에이전트가 하는 일을 순서대로 보면:

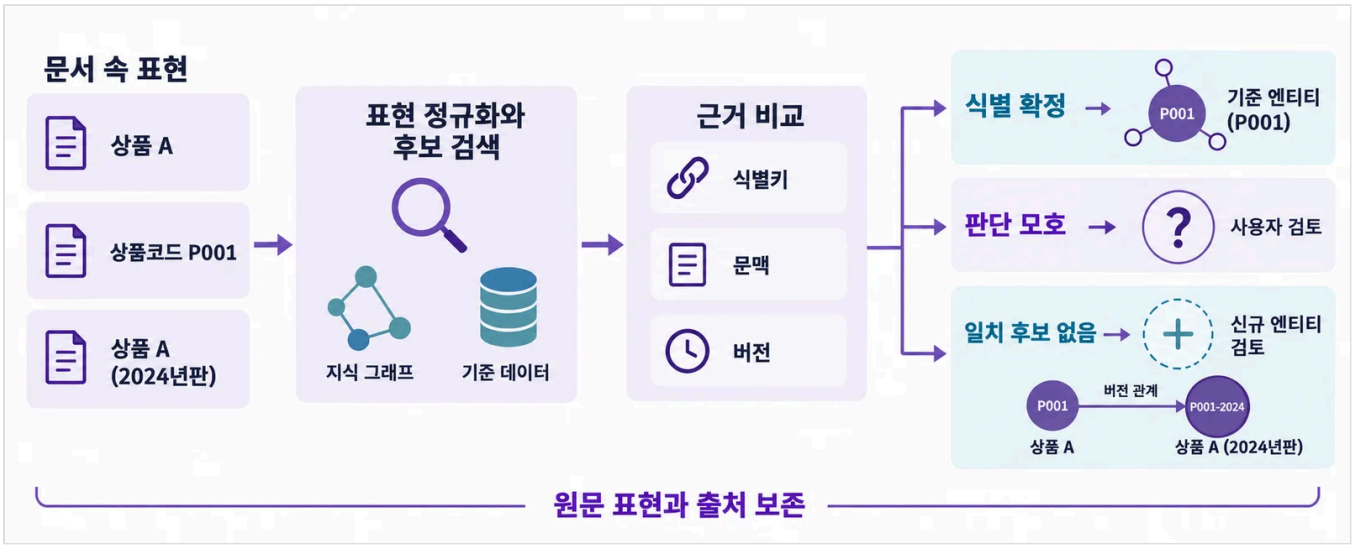
1. **문서 구조 파악** — 제목·목록·표를 읽고 무엇이 개념이고 무엇이 사실인지 나눕니다.
2. **스키마 설계** — 협력사(`Supplier`), 부품(`Component`), 완제품(`Product`), 품질 이슈(`QualityIssue`), 담당자(`Buyer`), 납기 실적(`DeliveryPerformance`) 같은 클래스와 그 사이의 관계를 정합니다.
3. **인스턴스 적재** — "골드라인 PCB가 PCB 기판을 공급한다"는 한 문장이 노드 두 개와 관계 하나가 됩니다.
4. **같은 대상 맞추기** — 문서 여러 곳의 "골드라인 PCB", "골드라인 PCB 가 납품한"이 같은 협력사인지 맞춥니다(아래 「엔티티 식별」).
5. **검증** — 골든 퀘스천을 직접 실행해 답이 나오는지 보고, 결과를 보고합니다.



*구축 보고 — 골든 케스천별 검증 결과와, 사람이 판단할 항목 (밤 테마)*

## 엔티티 식별 — 같은 것을 같다고 알아보기

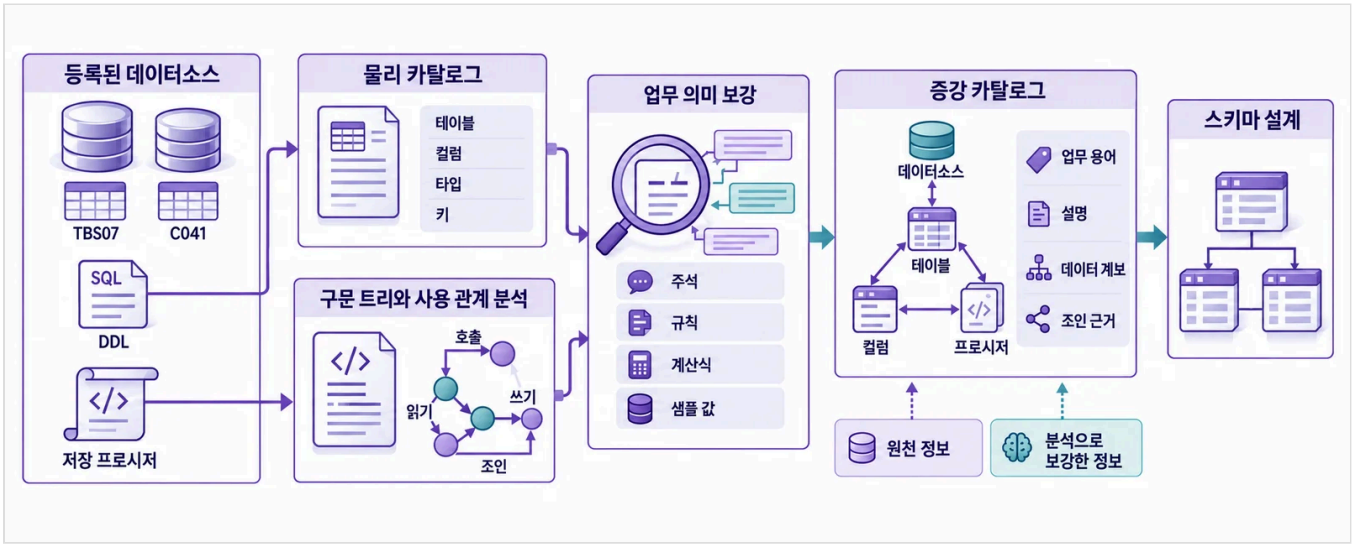
문서에는 같은 대상이 여러 이름으로 나옵니다. 에이전트는 이름·코드·문맥·버전을 대조해 같은 대상을 하나로 잇고, 애매하면 사람에게 묻습니다. 원문의 표현과 출처는 버리지 않고 남깁니다.




*엔티티 식별과 모호성 해소 — 문서 속 표현을 정규화해 후보를 찾고 식별키·문맥·버전으로 비교해 기존 엔티티에 연결하거나, 사람의 검토로 넘기거나, 새 엔티티로 판단합니다*

## 데이터베이스가 소스일 때 — 메타데이터 증강

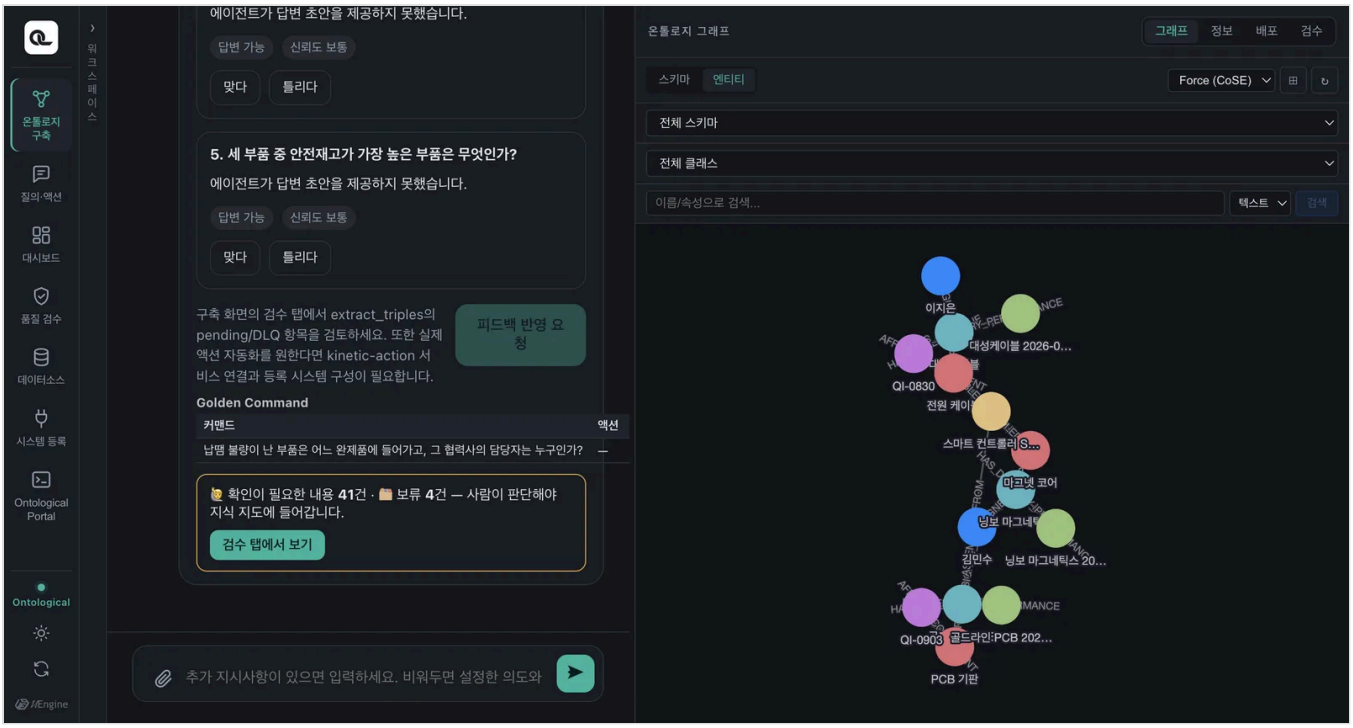
이 장에서는 문서를 썼지만, 소스로 데이터소스를 고르면 에이전트는 테이블·컬럼·키를 읽고 실제 값과 쓰임새를 살펴 **업무 의미를 보강**합니다. `TB507.C041` 같은 코드명 컬럼도 업무 용어로 찾을 수 있게 됩니다. 데이터는 복사하지 않고, 인스턴스는 가상 노드처럼 조회 시점에 원천에서 가져옵니다.




*메타데이터 증강 — 등록된 데이터소스의 물리 카탈로그와 구문·사용 관계 분석에 업무 의미를 더해 증강 카탈로그를 만들고 스키마 설계에 씁니다*

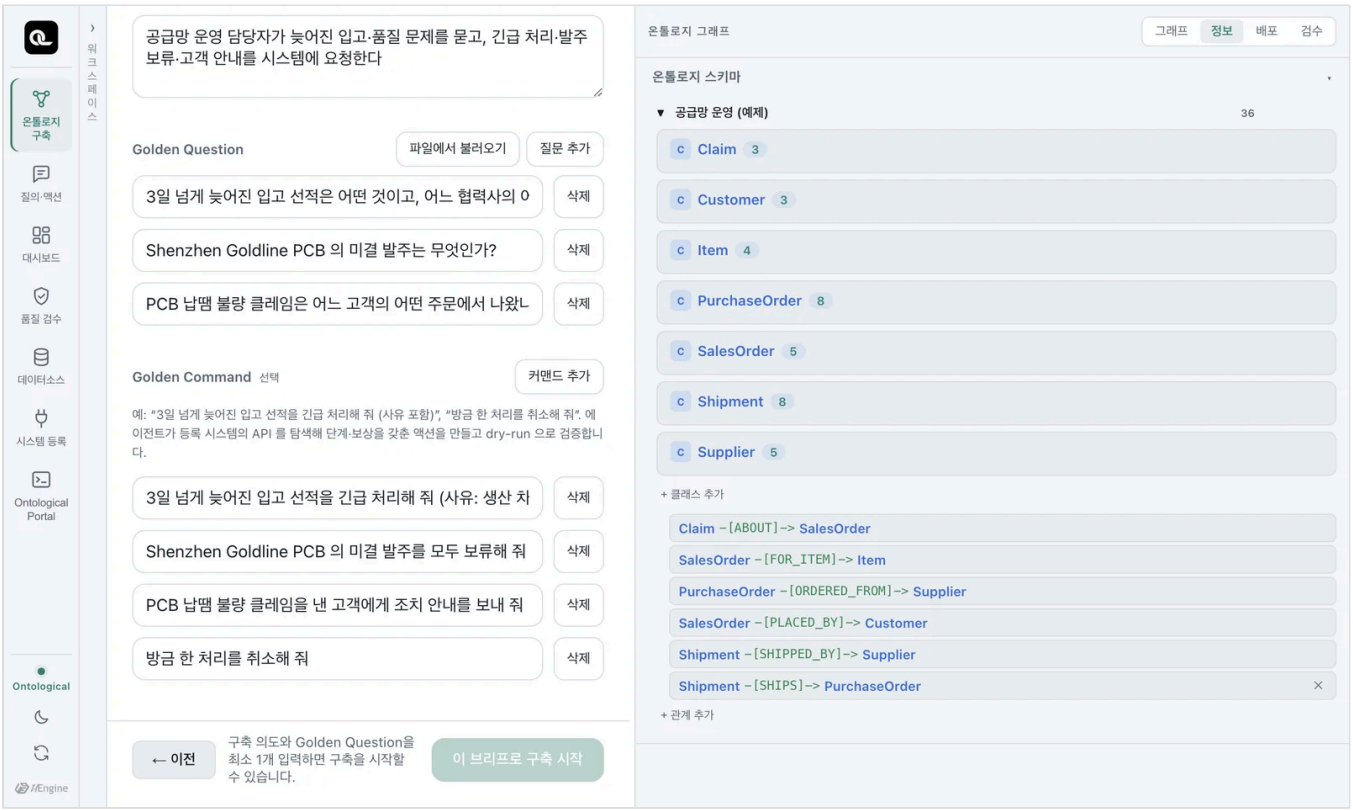
## 4단계 — 만들어진 온톨로지 보기

구축이 끝나면 오른쪽 「그래프」 탭에 지도가 남습니다. 「스키마」는 클래스와 관계, 「엔티티」는 인스턴스입니다. 노드를 끌어 옮기고, 위쪽 목록에서 배치(Force, Circle, Tree …)를 바꿀 수 있습니다.




*만들어진 온톨로지 — 협력사에서 부품으로, 부품에서 완제품으로, 이슈에서 부품으로 길이 나 있습니다 (밤 테마)*

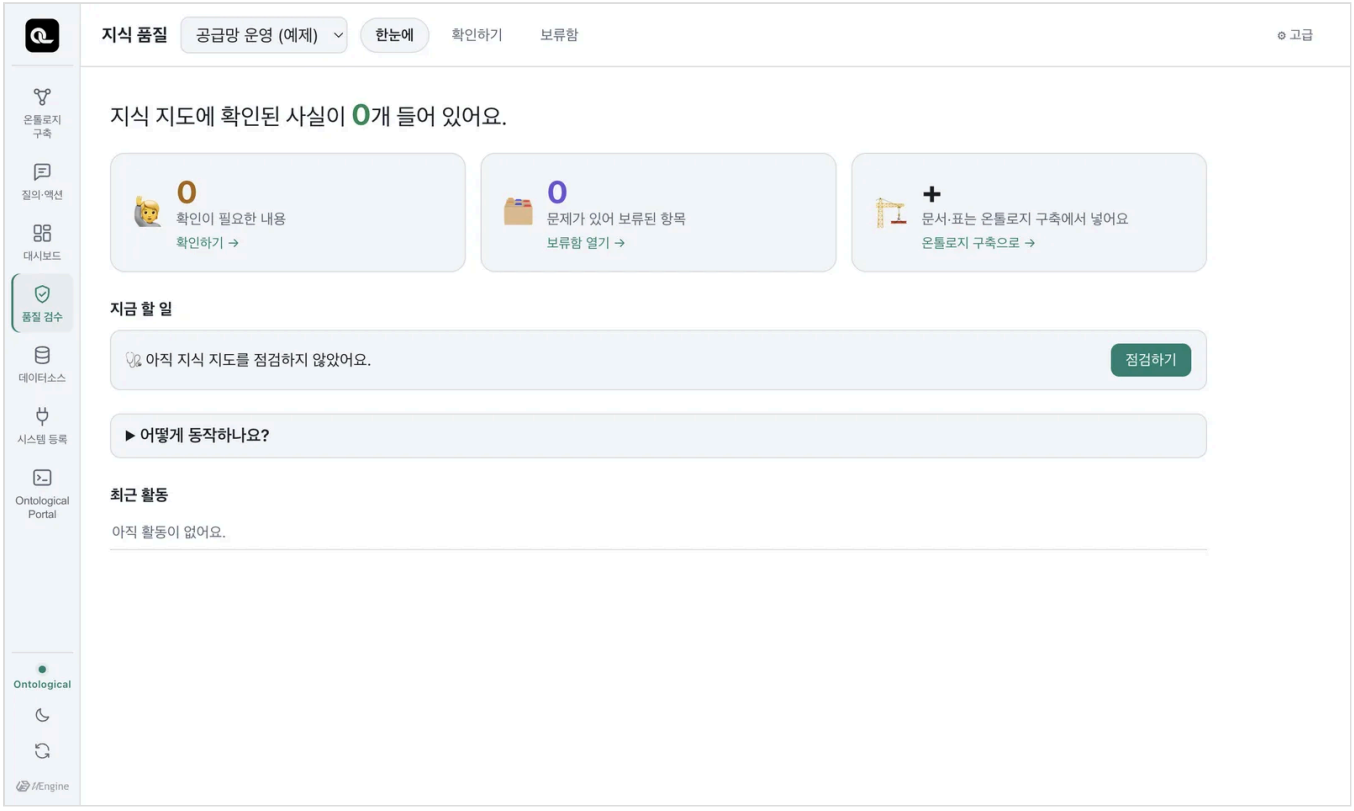
「정보」 탭에서는 스키마를 목록으로 보고 고칠 수 있습니다. 클래스마다 인스턴스 수가 보이고, ✎로 속성을 더하거나 이름을 바꾸고, 「+ 클래스 추가」, 「+ 관계 추가」로 직접 다듬을 수 있습니다.



*「정보」 탭 — 예제 온톨로지의 클래스 7개(괄호 안은 인스턴스 수)와 관계 6개*

## 5단계 — 검수와 발행

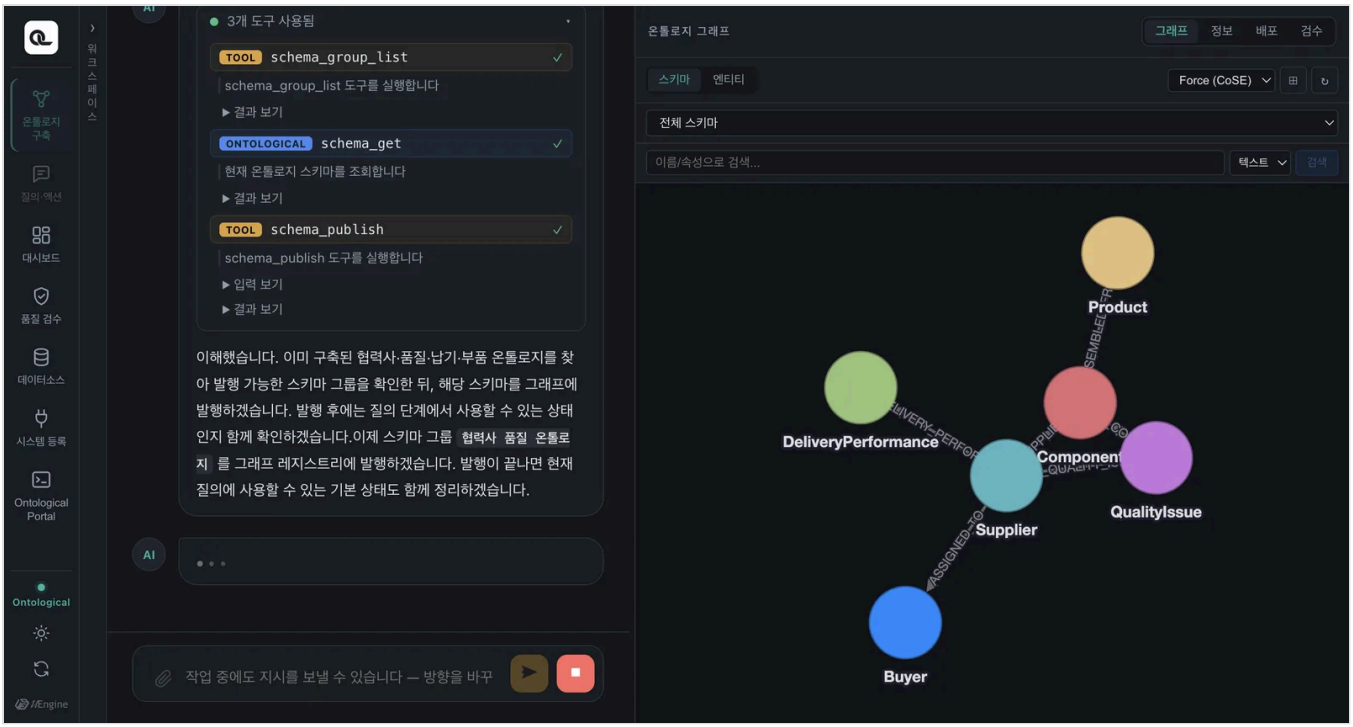
**품질 검수** 카드에는 구축이 남긴, 사람이 판단할 것들이 모입니다(원문과 다른 값, 확신이 낮은 추출, 원천 간 충돌 등). 맞으면 확정하고, 틀리면 고치거나 기각합니다. AI가 임의로 확정하지 않습니다.




*품질 검수 — 판단할 항목이 없으면 0건으로 나옵니다*

그리고 **발행**입니다. 발행해야 질의·액션·앱에서 이 온톨로지를 씁니다. 발행할지는 사람이 정합니다.

> **[주의] 현재 버전에서는 대화로 발행합니다**
>
> 이 판의 화면에는 발행 버튼이 없습니다. 골든 커맨드 없이 문서로만 구축하면 에이전트가 스스로 발행하지 않으므로, 구축 대화창에 이렇게 지시하세요.
>
> ```
> 발행해 줘
> ```
>
> 에이전트가 질의에서 쓸 수 있게 내보내고, 무엇이 반영됐는지 보고합니다(실측 약 50초).

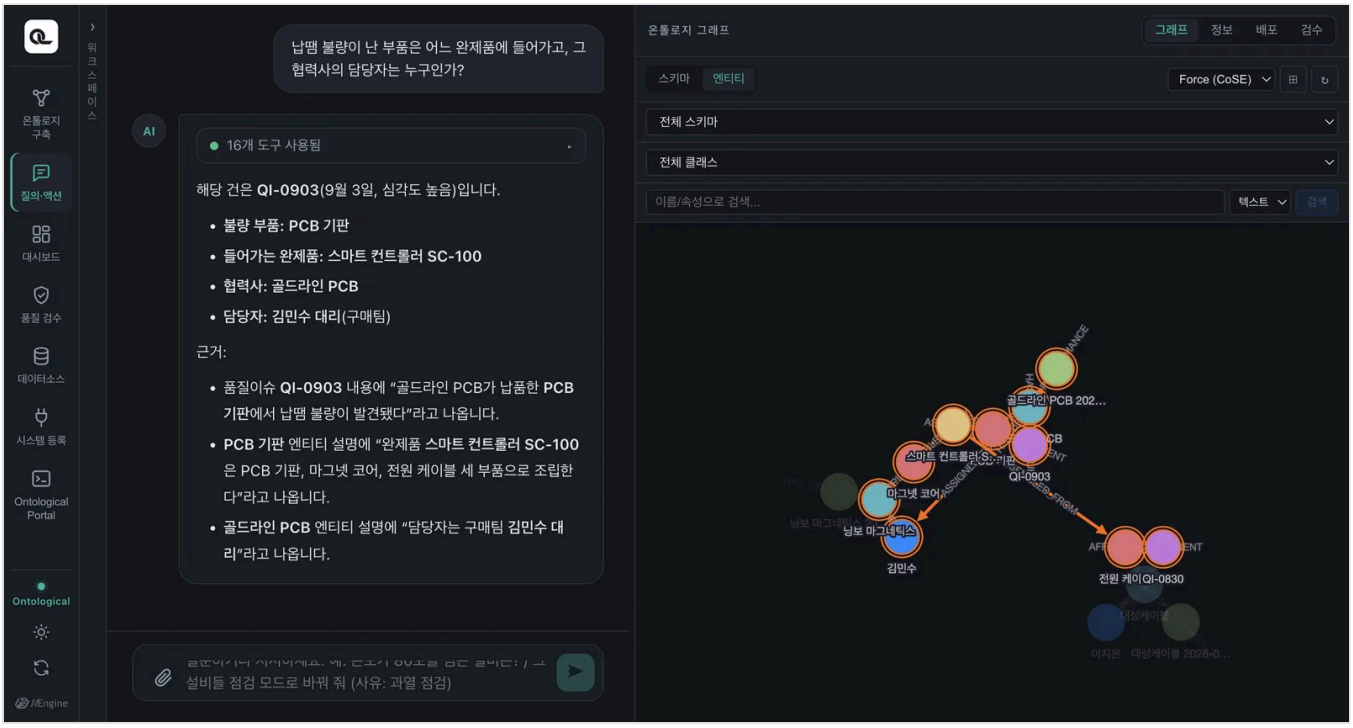



*대화로 발행을 지시하고 에이전트가 반영 결과를 보고한 모습 (밤 테마)*

## 6단계 — 물어보기

이제 「질의·액션」 탭을 새로 열고 앞에서 작성한 골든 퀘스천을 그대로 입력해 봅니다. 무엇을 어떤 순서로 찾으라고는 알려주지 않습니다.

```
납땜 불량이 난 부품은 어느 완제품에 들어가고, 그 협력사의 담당자는 누구인가?
```




*답 — 납땜 불량이 난 PCB 기판은 스마트 컨트롤러 SC-100에 들어가고, 골드라인 PCB의 담당자는 김민수 대리. 근거가 된 사실이 함께 적힙니다 (밤 테마)*

답과 함께 **어느 사실을 근거로 삼았는지**가 적힙니다. 질의 화면은 13장에서 자세히 봅니다.

> **[팁] 에이전트의 답은 매번 조금씩 다릅니다**
>
> 같은 문서로 다시 구축하면 클래스 이름이나 관계 이름이 조금 다르게 나올 수 있습니다. LLM이 설계하기 때문입니다. 그래서 골든 퀘스천이 중요합니다 — 이름이 달라도 **질문에 같은 답을 내는지**로 품질을 잽니다.
>
> **[확인 문제] 실습 과제**
>
> 1. 실습 문서로 직접 구축하고 발행한 뒤, 골든 퀘스천에 같은 답(SC-100, 김민수 대리)이 나오는지 확인하세요.
> 2. 문서에 협력사 한 곳을 더 적어(예: "**한빛정밀** — 경기도 화성의 1차 협력사. 마그넷 코어를 공급한다. 담당자는 박서준 과장.") 같은 스키마에 **추가 인제스천**하고, "마그넷 코어를 공급하는 협력사와 담당자는?"이라고 물어보세요.
> 3. 포털에서 그래프를 `ontology`로 바꾸고, 여러분이 만든 클래스를 Cypher로 조회해 보세요.
>
> **▼ 도움말**
>
> 2번: 대상 단계에서 앞서 만든 스키마를 고르면 「(추가 인제스천)」으로 표시됩니다. 같은 클래스 이름이 이미 있으면 그 클래스에 인스턴스가 더해집니다.
>
> 3번: 클래스 이름은 에이전트가 정하므로 포털 왼쪽 ENTITY TYPES에서 확인한 뒤 씁니다. 예: `MATCH (s:Supplier)-[r]->(c:Component) RETURN s.name, type(r), c.name` — 관계 이름을 모르면 이처럼 `[r]`로 받고 `type(r)`로 확인할 수 있습니다(저장된 관계에서는 이름 없는 관계 패턴도 동작합니다).

# 질의하기 — 근거와 함께 답 받기

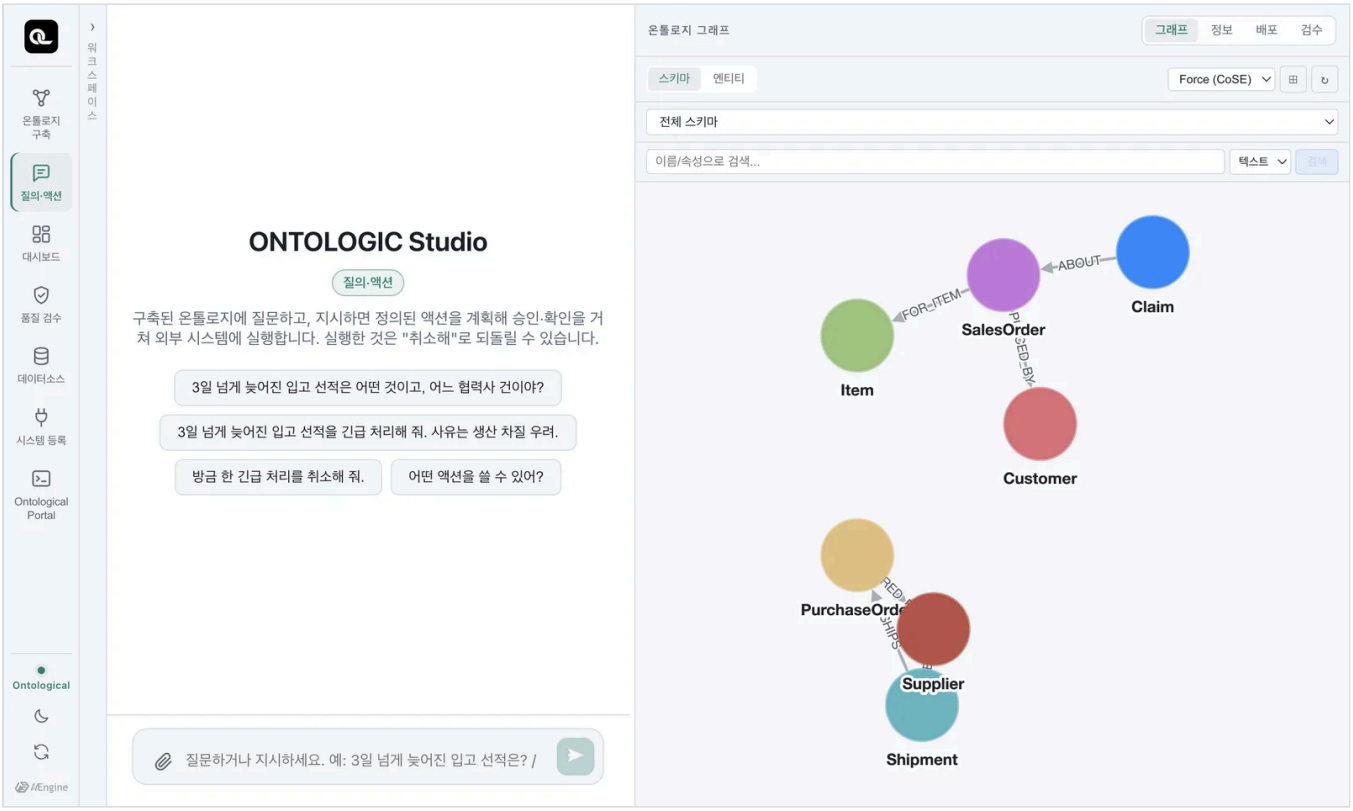
*분류: 온톨로지 구축 · 13장 · 원문: https://www.uengine.org/docs/ontological-studio/ask.html*

> 「질의·액션」 탭에서 자연어로 온톨로지에 질문하고 답변의 근거를 확인하는 방법을 익힙니다. 예제 온톨로지 「공급망 운영 (예제)」로 실습합니다.

▶ 영상: 질의하기 — 근거와 함께 답 받기 — 영상 3:01

## 질의·액션 탭

홈의 「질의·액션」 카드를 엽니다. 가운데에 예시 질문이, 오른쪽에 발행된 온톨로지의 그래프가 보입니다.




*질의·액션 첫 화면 — 예제 온톨로지의 골든 퀘스천·골든 커맨드가 예시로 나옵니다*

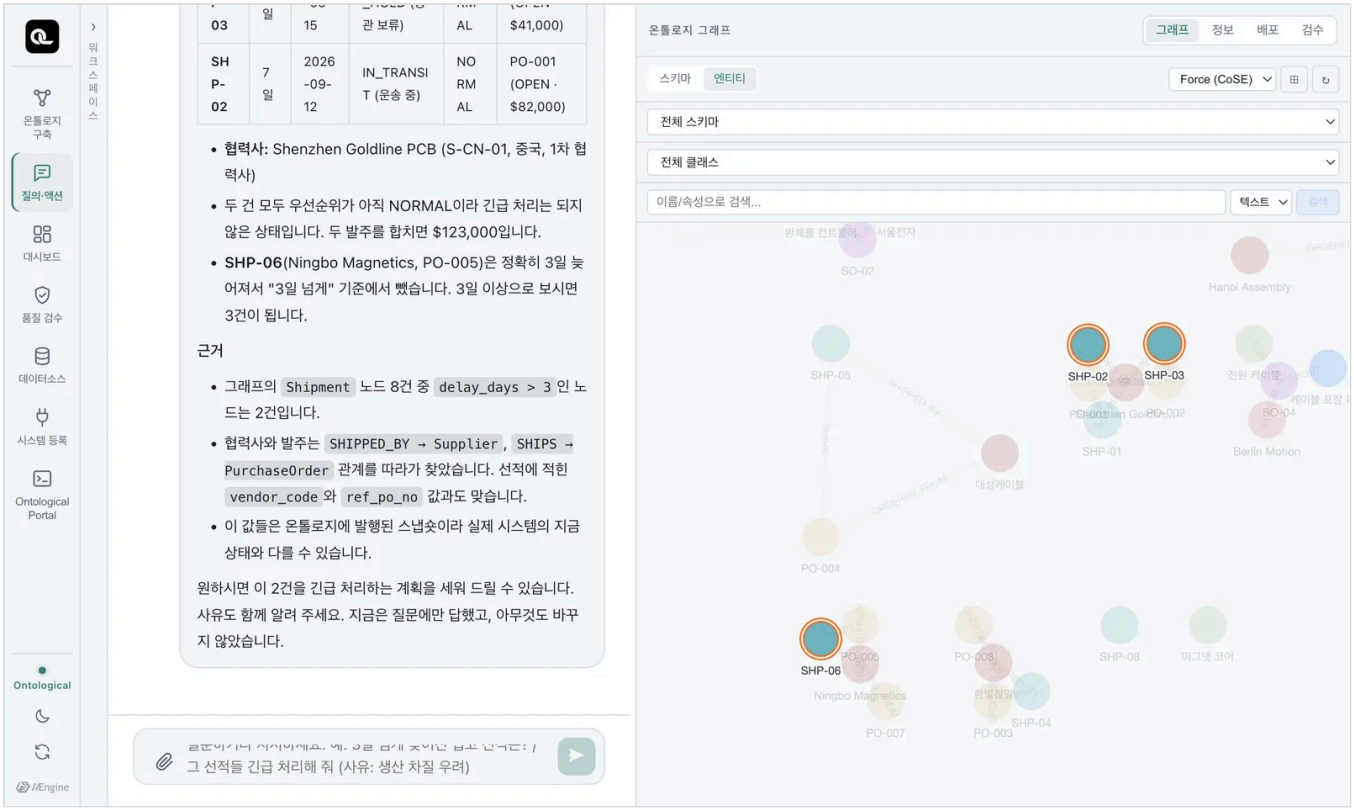
이 탭은 **질문**과 **지시**를 함께 받습니다. 질문하면 답하고, 지시하면 액션을 계획합니다(지시는 16장). 질문만 했을 때는 아무것도 바뀌지 않습니다.

## 물어보기

예시 질문을 누르거나 직접 씁니다.

```
3일 넘게 늦어진 입고 선적은 어떤 것이고, 어느 협력사 건이야?
```

에이전트는 먼저 발행된 온톨로지의 스키마를 읽고, Cypher로 그래프를 조회해 답을 만듭니다(실측 약 30초). 오른쪽 그래프에서는 답에 쓰인 노드가 강조됩니다.




*답 — SHP-03(9일)과 SHP-02(7일), 협력사 Shenzhen Goldline PCB. 아래에 근거가 적혀 있고, 오른쪽 그래프에 두 선적이 강조됩니다*

## 답을 읽는 법

좋은 답에는 세 부분이 있습니다.

1. **결론** — "3일 넘게 늦어진 선적은 SHP-03(9일)과 SHP-02(7일)로, 모두 Shenzhen Goldline PCB의 발주 건입니다."
2. **근거** — "그래프의 `Shipment` 노드 8건 중 `delay_days > 3`인 노드는 2건입니다. 협력사와 발주는 `SHIPPED_BY → Supplier`, `SHIPS → PurchaseOrder` 관계를 따라가 찾았습니다."
3. **경계** — "SHP-06은 정확히 3일이라 '3일 넘게' 기준에서 뺐습니다. 3일 이상으로 보시면 3건입니다.", "지금은 질문에만 답했고, 아무것도 바꾸지 않았습니다."

근거가 있으니 답을 **검산**할 수 있습니다. 같은 질문을 포털에서 Cypher로 실행하면 같은 결과가 나와야 합니다(8장).

```
MATCH (sh:Shipment)-[:SHIPS]->(po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier)
WHERE sh.delay_days > 3
RETURN sh.shipment_no, sh.delay_days, po.po_id, s.name
```

> **[참고] 질의 화면의 데이터는 '발행된 온톨로지'입니다**
>
> 질의·액션 탭은 `ontology` 그래프의 발행된 온톨로지를 봅니다. 예제의 인스턴스는 처음 실행할 때 원천에서 읽어 적재한 것이라, 원천이 바뀐 직후에는 값이 다를 수 있습니다. 에이전트도 답에 "이 값들은 발행된 스냅숏이라 실제 시스템의 지금 상태와 다를 수 있다"고 적습니다. 액션으로 바뀐 값(예: 선적 우선순위)은 실행 시 온톨로지에도 반영됩니다.

## 이어서 묻기

같은 대화에서 이어서 질문하면 이전 대화의 맥락을 활용합니다.

```
그 협력사의 다른 미결 발주는?
```

```
PCB 납땜 불량 클레임은 어느 고객의 어떤 주문에서 나왔어?
```

두 번째 질문은 첫 질문과 전혀 다른 관계(`Claim → SalesOrder → Customer`)를 지나지만, 새로 설계할 것이 없습니다. 이미 있는 관계를 따라갈 뿐입니다 — 3장에서 말한 "다음 질문"의 힘입니다.

## 대화 관리

- 레일의 「워크스페이스」를 펼치면 지난 대화 목록이 있고, 새 대화를 시작할 수 있습니다.
- 홈의 최근 항목에도 대화가 남습니다. 누르면 그 대화로 바로 갑니다.
- 문서를 끌어다 놓으면(📎) 대화에 첨부해 함께 물을 수 있습니다.

> **[확인 문제]**
>
> 질의·액션 탭에서 다음을 물어보고, 답의 근거를 포털의 Cypher로 검산하세요.
>
> 1. "Shenzhen Goldline PCB의 미결 발주는 무엇인가?"
> 2. "PCB 납땜 불량 클레임은 어느 고객의 어떤 주문에서 나왔나?"
> 3. "안전재고보다 재고가 적은 품목이 있어?" — 이 질문에 답이 나오나요? 안 나온다면 왜일까요?
>
> **▼ 정답 보기**
>
> 1. PO-001($82,000), PO-002($41,000). 검산: `MATCH (po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier {name:'Shenzhen Goldline PCB'}) WHERE po.status = 'OPEN' RETURN po.po_id, po.amount_usd`
> 2. Tokyo Devices의 판매 주문 SO-05 (클레임 T-3).
> 3. 예제 온톨로지에는 일별 재고(`wms.stock_daily`)가 클래스로 들어 있지 않으므로 에이전트는 "온톨로지에 재고 정보가 없다"고 답해야 맞습니다. 답에 필요한 개념이 없으면 **구축 의도와 골든 퀘스천에 이 질문을 넣고 다시 구축**하면 됩니다. 이것이 온톨로지가 자라는 방식입니다.

# 업무 시스템 등록하기

*분류: 액션 · 14장 · 원문: https://www.uengine.org/docs/ontological-studio/systems.html*

> 액션이 호출할 업무 시스템(마이크로서비스)을 등록합니다. 데이터소스에서 데이터를 읽고, 등록된 시스템의 API로 업무를 실행합니다. OpenAPI 스펙, 인증, 연결 테스트를 다룹니다.

▶ 영상: 업무 시스템 등록하기 — 영상 3:47

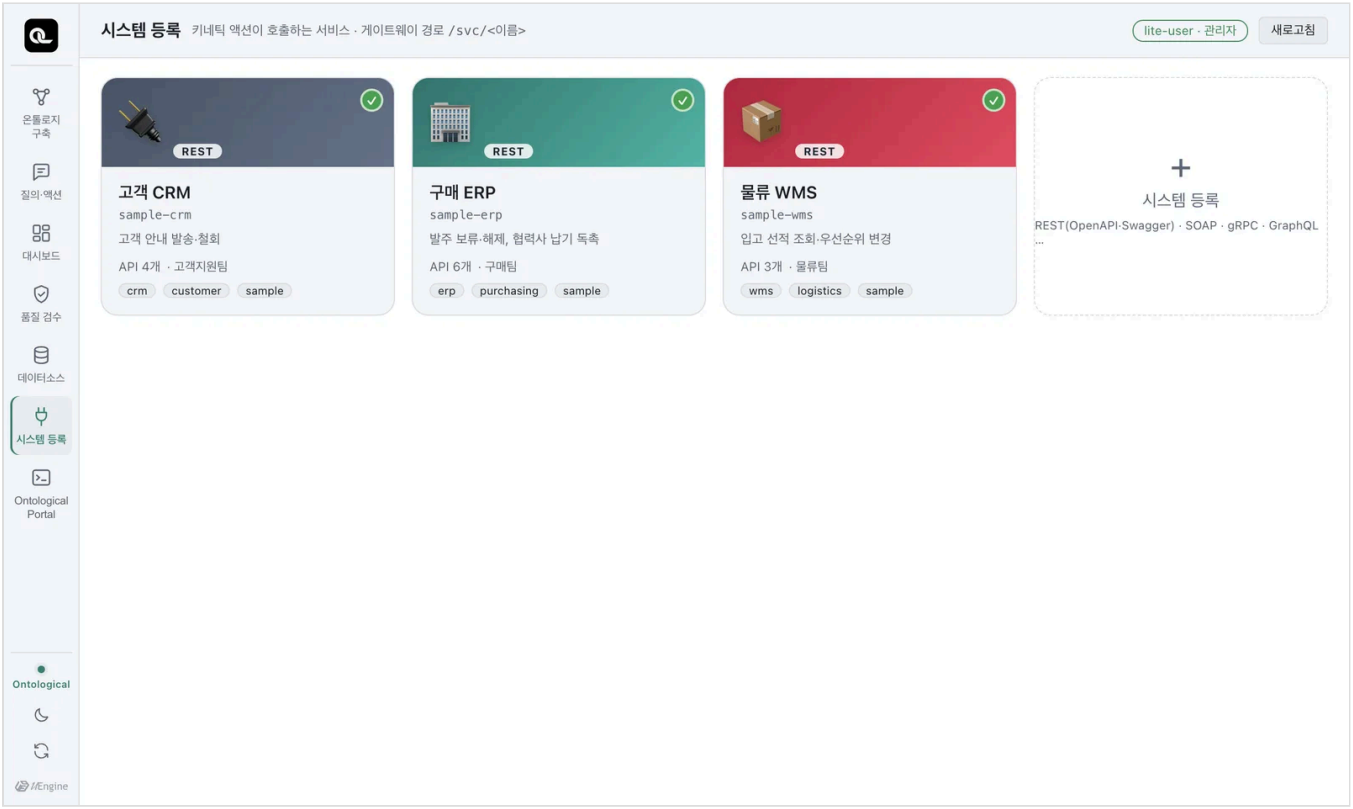
## 왜 시스템을 따로 등록하나

데이터 패브릭은 원천을 **읽기만** 합니다(6장). 발주를 보류하거나 선적 우선순위를 올리는 일은 그 데이터의 주인인 **업무 시스템**이 해야 합니다. 그래야 그 시스템의 업무 규칙(검증, 승인, 이력)이 지켜집니다.

그래서 Ontological Studio의 액션은 데이터베이스에 직접 쓰지 않고, **등록된 시스템의 API를 부릅니다.** 이 장은 그 API를 등록하는 법입니다.

## 시스템 등록 화면

홈의 「시스템 등록」 카드를 엽니다. 예제 시스템 세 개가 이미 등록돼 있습니다.



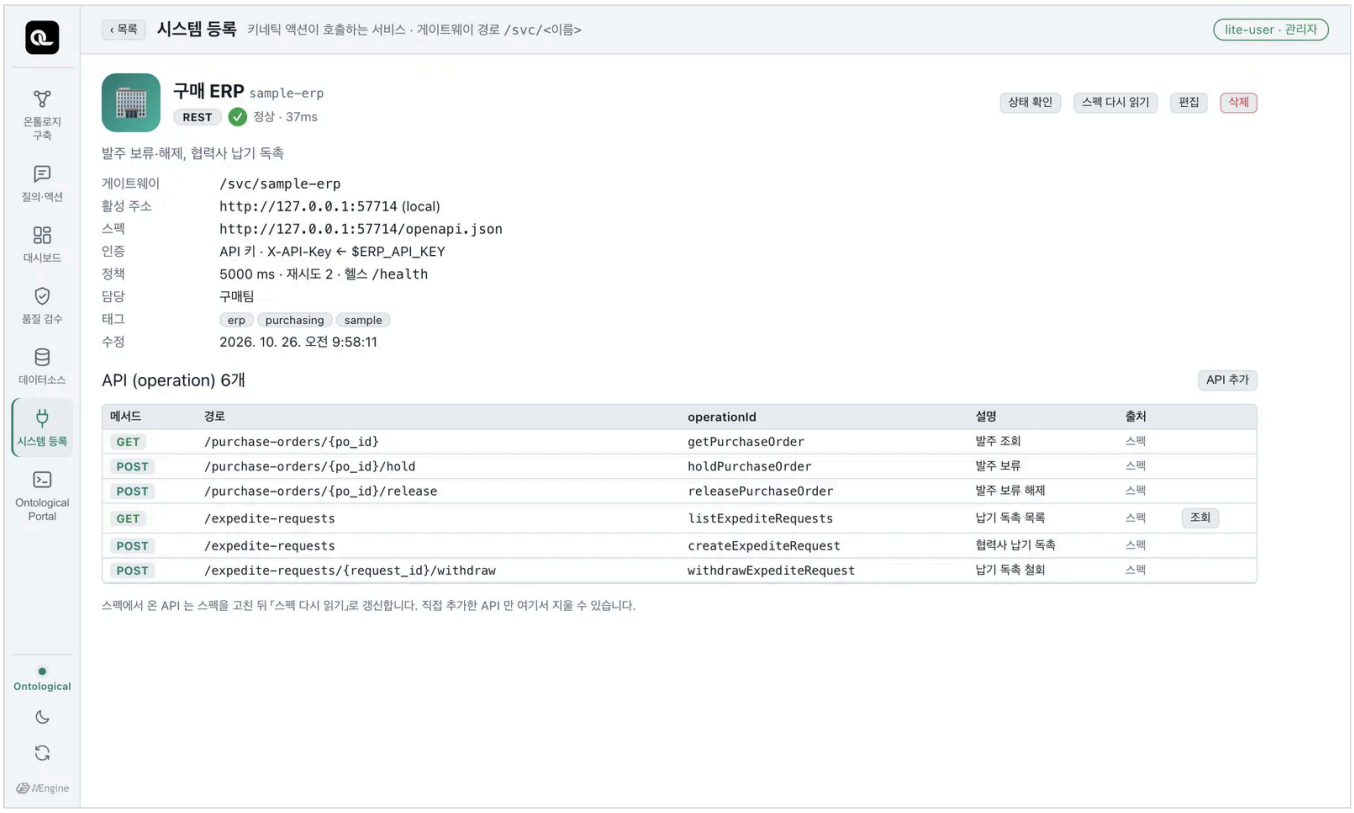

*시스템 등록 — 고객 CRM, 구매 ERP, 물류 WMS 세 시스템이 카드로 보입니다*

| 시스템 | 하는 일 (operationId) | 인증 방식 |
|---|---|---|
| `sample-erp` 구매 ERP | 발주 보류·해제( `holdPurchaseOrder` , `releasePurchaseOrder` ), 협력사 납기 독촉·철회( `createExpediteRequest` , `withdrawExpediteRequest` ) | API 키 ( `X-API-Key` 헤더) |
| `sample-wms` 물류 WMS | 선적 조회, 우선순위 변경( `setShipmentPriority` ) | Basic 인증 |
| `sample-crm` 고객 CRM | 고객 안내 발송·철회( `createNotice` , `withdrawNotice` ) | Bearer 토큰 |

실제 회사처럼 시스템마다 인증 방식이 다릅니다. 각 시스템은 예제 업무 DB의 **자기 스키마만** 고칩니다(ERP는 `erp` , WMS는 `wms` ).

## 시스템 하나 들여다보기

「구매 ERP」 카드를 누릅니다.




*구매 ERP 상세 — 상태 정상(37ms), 게이트웨이 경로, 스펙 주소, 인증, 호출 정책, 그리고 스펙에서 읽은 API 6개*

| 항목 | 뜻 |
|---|---|
| 게이트웨이 경로 | 액션 실행기가 이 시스템을 부를 때 쓰는 이름 ( `/svc/sample-erp` ) |
| 활성 주소 | 실제 서비스 주소 |
| 스펙 | OpenAPI 문서 주소. 여기서 API 목록을 읽습니다. |
| 인증 | `API 키 · X-API-Key ← $ERP_API_KEY` — **비밀 값은 등록 정보에 적지 않고** 환경 변수 이름만 남깁니다. 값은 실행기만 갖고 있다가 호출하는 순간에 붙입니다. |

| 항목 | 뜻 |
| --- | --- |
| 정책 | 타임아웃 5000ms, 재시도 2번, 상태 점검 경로 `/health` (비워 두면 활성 주소 자체를 점검) |
| API (operation) | 스펙에서 읽은 API. 메서드·경로·operationId·설명 |

「상태 확인」으로 현재 연결 상태를 확인하고, 「스펙 다시 읽기」로 바뀐 스펙을 다시 가져옵니다. 스펙에 빠진 API는 「API 추가」로 메서드와 경로를 직접 적어 넣을 수 있습니다.

## 새 시스템 등록하기

시스템 목록의 「+ 시스템 등록」을 누르면 데이터소스를 등록할 때처럼 단계가 나옵니다.

1. **기본 정보** — 이름(영문 소문자·숫자·하이픈), 표시 이름, 담당 팀, 연락처, 태그, 아이콘.
2. **스펙** — OpenAPI(Swagger) 파일을 끌어다 놓거나 스펙 주소를 적습니다. 호출할 수 있는 API 목록이 바로 읽힙니다. REST가 아닌 SOAP·gRPC·GraphQL 시스템도 등록은 되지만, 실행 어댑터가 붙기 전까지는 액션에 쓰지 못한다고 표시됩니다.
3. **인증** — API 키, Basic, Bearer, OAuth 클라이언트 자격 증명, 로그인 토큰 중에서 고르고, 비밀 값이 들어 있는 **환경 변수 이름**을 적습니다.
4. **연결 테스트** — 저장하기 전에 스펙을 읽고 상태 점검을 불러 봅니다. 통과하면 저장합니다. 상태 점검 경로가 없는 시스템이면 경로를 비워 두세요. 서버가 응답만 하면(5xx가 아니면) **연결됨**으로 보고, 적어 둔 경로가 404·405를 돌려주면 「연결됨 · (경고 내용)」과 주황색 `~` 표시로 경로를 확인하라고 알려 줍니다.

> **[팁] 공개 API로 연습하기**
>
> 회사 시스템이 없다면 공개 API로 연습할 수 있습니다. 예를 들어 Swagger Petstore( `https://petstore3.swagger.io/api/v3/openapi.json` )를 등록하면 스펙에서 API 19개를 읽고 연결 테스트가 통과합니다. 환경의 활성 주소에 `https://petstore3.swagger.io/api/v3` 을 적고, Petstore에는 `/health` 가 없으니 상태 점검 경로는 비워 두세요. 액션을 만들 대상 시스템으로 쓰기에는 데이터가 공용이라 적당하지 않으니, 등록·조회 연습까지만 하세요.
>
> **[주의] 비밀 값은 어디에?**
>
> 등록 정보에는 `$ERP_API_KEY` 같은 **이름**만 남습니다. 예제 시스템의 실제 비밀 값은 설치할 때 무작위로 만들어져 데이터 폴더의 `secrets.json` 에 있고, 실행기에 환경 변수로만 넘어갑니다. 여러분이 등록하는 시스템의 비밀 값은 `studio.env` 에 `ERP_API_KEY=...` 처럼 적어 두면 됩니다.
>
> **[확인 문제]**
>
> 1. 물류 WMS 시스템을 열어, 우선순위를 바꾸는 API의 메서드와 경로를 찾으세요.
> 2. 고객 CRM의 인증 방식은 무엇이고, 비밀 값은 등록 정보 어디에 보이나요?
> 3. 액션이 ERP 데이터베이스에 직접 `UPDATE` 하지 않고 굳이 API를 부르는 이유를 두 가지 적어 보세요.
>
> **▼ 정답 보기**
>
> 1. `POST /shipments/{shipment_no}/priority` (operationId `setShipmentPriority` ).
> 2. Bearer 토큰. 비밀 값 자체는 보이지 않고 환경 변수 이름만 보입니다.

3. ⓐ 업무 시스템의 검증·승인·이력 같은 업무 규칙을 지키기 위해. ⓑ 권한과 책임이 그 시스템에 있기 때문에(누가 언제 무엇을 바꿨는지가 그 시스템의 기록으로 남음). 그 밖에 ⓒ 시스템 내부 구조(테이블)가 바뀌어도 API는 유지되므로 액션이 덜 깨집니다.

# 골든 커맨드로 액션 만들기

*분류: 액션 · 15장 · 원문: https://www.uengine.org/docs/ontological-studio/actions-build.html*

> 골든 커맨드를 적으면 구축 에이전트가 시스템 API를 살펴 액션을 만듭니다. 예제 액션을 통해 단계, 인자 매핑, 보상, 정책이 어떻게 구성되는지 살펴봅니다.

▶ 영상: 골든 커맨드로 액션 만들기 — 영상 4:03

## 액션이란

**액션**은 온톨로지의 클래스에 정의된 업무 수행 기능입니다. 예제의 `Shipment` 클래스에는 `expediteShipment`(긴급 출하 처리)가 붙어 있습니다. 액션은 다음을 담고 있습니다.

| 구성 | 뜻 | `Shipment.expediteShipment` 에서 |
|---|---|---|
| 대상 키 | 대상 인스턴스를 가리키는 속성 | `shipment_no` |
| 입력 | 실행할 때 사람이(또는 에이전트가) 주는 값 | `reason` (긴급 처리 사유, 필수) |
| 단계 | 차례로 부를 시스템 API | ① WMS 우선순위 URGENT ② ERP 협력사 납기 독촉 |
| 보상 | 단계마다, 그것을 되돌리는 API | ① 우선순위를 실행 전 값으로 ② 납기 독촉 철회 |
| 정책 | 누가, 어떤 대상에 쓸 수 있나 | 역할 `scm_operator`, 이미 입고된 선적(`DELIVERED`)은 제외 |
| 승인 | 사람의 승인이 필요한가 | 예제는 모든 계획에 승인 필요 |
| 되쓰기 | 실행 뒤 온톨로지에 반영할 속성 | `priority` |
| 골든 커맨드 | 이 액션을 만든 지시 | “3일 넘게 늦어진 입고 선적을 긴급 처리해 줘 (사유: 생산 차질 우려)” |

## 정의를 뜯어보기

예제 액션의 정의입니다(설명을 줄였습니다).

```
class_name = "Shipment", action_name = "expediteShipment", key_property = "shipment_no"
parameters = {"reason": {"type": "string", "description": "긴급 처리 사유"}}   # 필수
steps = [
  {"name": "wms", "system": "sample-wms", "operation_id": "setShipmentPriority",
   "arg_mapping": {"path.shipment_no": "$this.shipment_no", "body.priority": "URGENT",
                   "body.reason": "$params.reason", "body.requestedBy": "$actor.sub"},
   "compensation": {"system": "sample-wms", "operation_id": "setShipmentPriority",
                    "arg_mapping": {"path.shipment_no": "$this.shipment_no",
                                    "body.priority": "$before.priority",
                                    "body.reason": "$cancel.reason"}}},
  {"name": "erp", "system": "sample-erp", "operation_id": "createExpediteRequest",
   "arg_mapping": {"body.poId": "$this.ref_po_no", "body.reason": "$params.reason"},
   "compensation": {"system": "sample-erp", "operation_id": "withdrawExpediteRequest",
                    "arg_mapping": {"path.request_id": "$steps.erp.response.id",
                                    "body.reason": "$cancel.reason"}}},
]
policy = [{"roles": ["scm_operator"], "when": "target.ship_status != 'DELIVERED'"}]
writeback_property = "priority"
```

### 인자 매핑 — 값은 어디서 오나

`arg_mapping`의 왼쪽은 API 요청의 자리(`path.…`는 URL 경로, `body.…`는 본문), 오른쪽은 값의 출처입니다.

| 출처 | 뜻 | 예 |
|---|---|---|
| `$this.<속성>` | 대상 인스턴스의 속성 | `$this.shipment_no` → `SHP-03` |
| `$params.<이름>` | 실행할 때 준 입력 | `$params.reason` → "생산 차질 우려" |
| `$actor.<클레임>` | 실행한 사람(또는 에이전트) | `$actor.sub` → `lite-agent` |
| `$before.<속성>` | **실행 직전**의 대상 속성 값 | `$before.priority` → `NORMAL` |
| `$steps.<단계>.response.<경로>` | 앞 단계 API 응답의 값 | `$steps.erp.response.id` → 납기 독촉 번호 |
| `$cancel.reason` | 취소할 때 준 사유 | "사용자 요청으로 긴급 처리 취소" |
| 그냥 값 | 고정값 | `"URGENT"` |

`$before`와 `$steps`가 **보상**을 가능하게 합니다. 우선순위를 되돌리려면 "원래 값이 무엇이었는지"를, 납기 독촉을 철회하려면 "어느 독촉을 보냈는지"를 알아야 하기 때문입니다. 지워 버리는 것이 아니라 **원래 상태로 되돌리는 것**이 보상입니다.

### 정책 — 누가, 무엇에

정책은 규칙의 목록입니다. 규칙마다 **역할**(`roles`)과 **조건**(`when`)이 있고, 하나라도 맞으면 허용, 하나도 맞지 않으면 **거부**합니다(기본 거부). 조건은 CEL이라는 작은 표현식 언어로 씁니다.

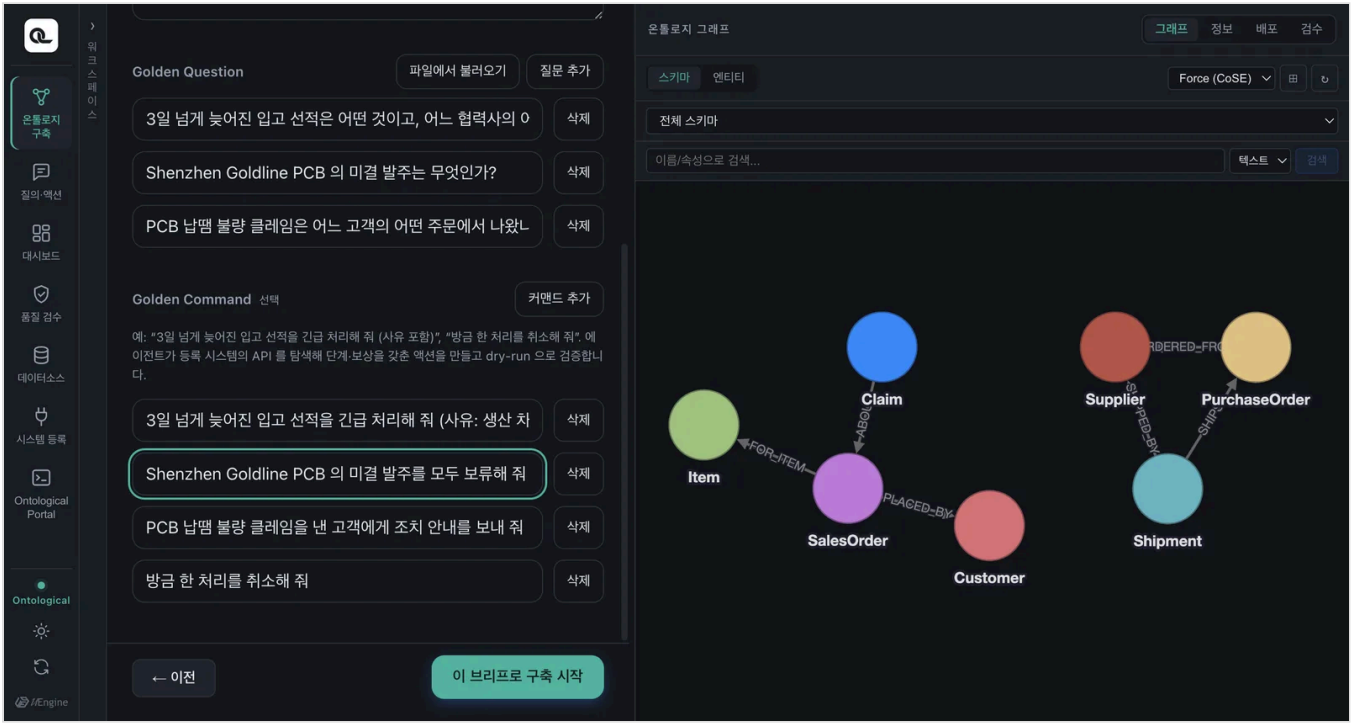
| 쓸 수 있는 것 | 예 |
|---|---|
| `target.<속성>` | `target.status == 'OPEN'` — 미결 발주만 |
| `actor.<클레임>` | `target.plant == actor.plant` — 자기 공장의 설비만 |
| `params.<이름>` | `params.supplierName == target.approvedSupplier` — 승인 공급처에만 발주 |

정책은 **대상마다** 판정됩니다. 계획을 세우면 대상 하나하나에 허용/거부와 사유가 붙습니다(16장). 그리고 실행 직전에 한 번 더 판정합니다 — 계획을 세운 뒤 데이터가 바뀌었을 수 있기 때문입니다.

## 따라 하기 — 골든 커맨드로 액션 만들기

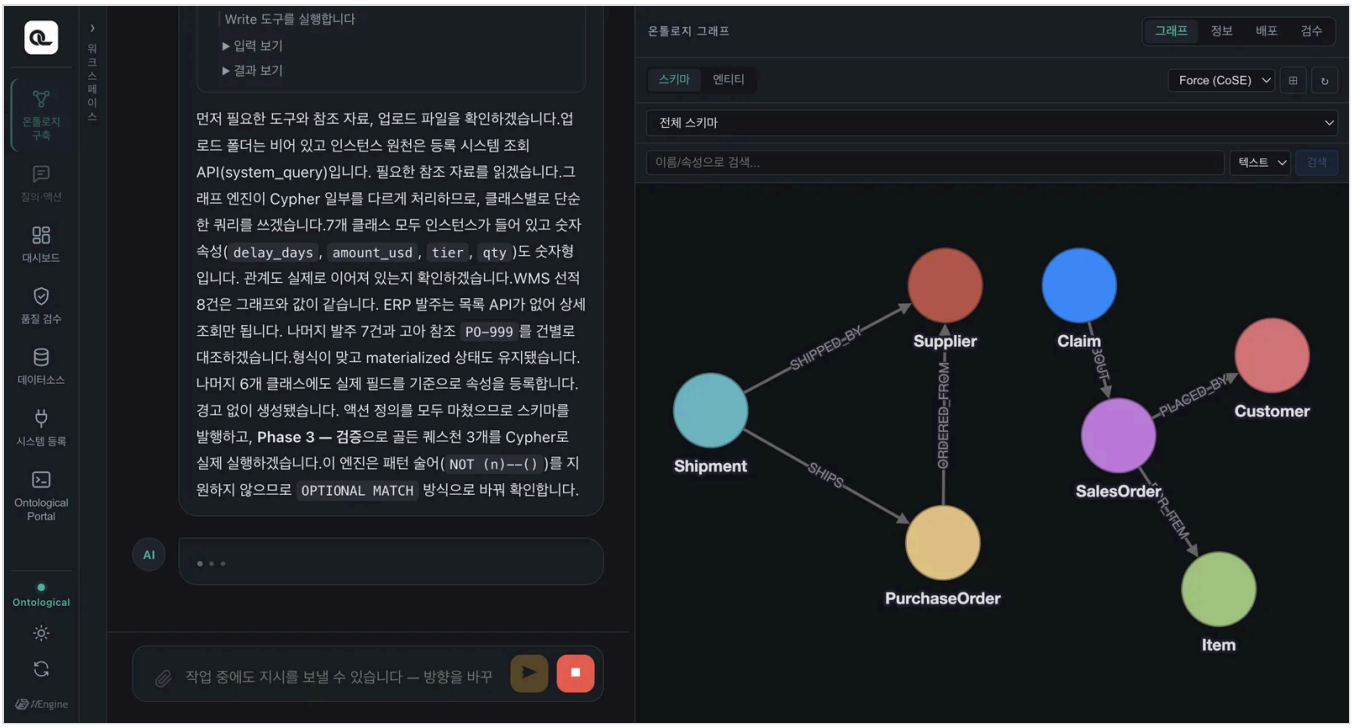
예제에는 액션이 이미 세 개 있습니다. 시연에서는 그중 「발주 보류」를 지운 뒤, 골든 커맨드로 에이전트가 다시 만들게 했습니다. 여러분은 **새 골든 커맨드**로 따라 해 봅시다.

1. 「온톨로지 구축」을 열고 **소스**에서 시스템 `sample-erp`를 고릅니다. 문서도 데이터베이스도 아닌, 시스템의 API가 이번 구축의 재료입니다.
2. **대상**에서 기존 스키마 「공급망 운영 (예제)」를 고릅니다(추가 인제스천). 개념은 이미 있으니, 할 수 있는 일만 더합니다.
3. **의도**의 Golden Command에 새 지시를 적습니다. 예: `한빛정밀의 미결 발주를 모두 보류해 줘 (사유: 단가 재협상)`
4. 「이 브리프로 구축 시작」.



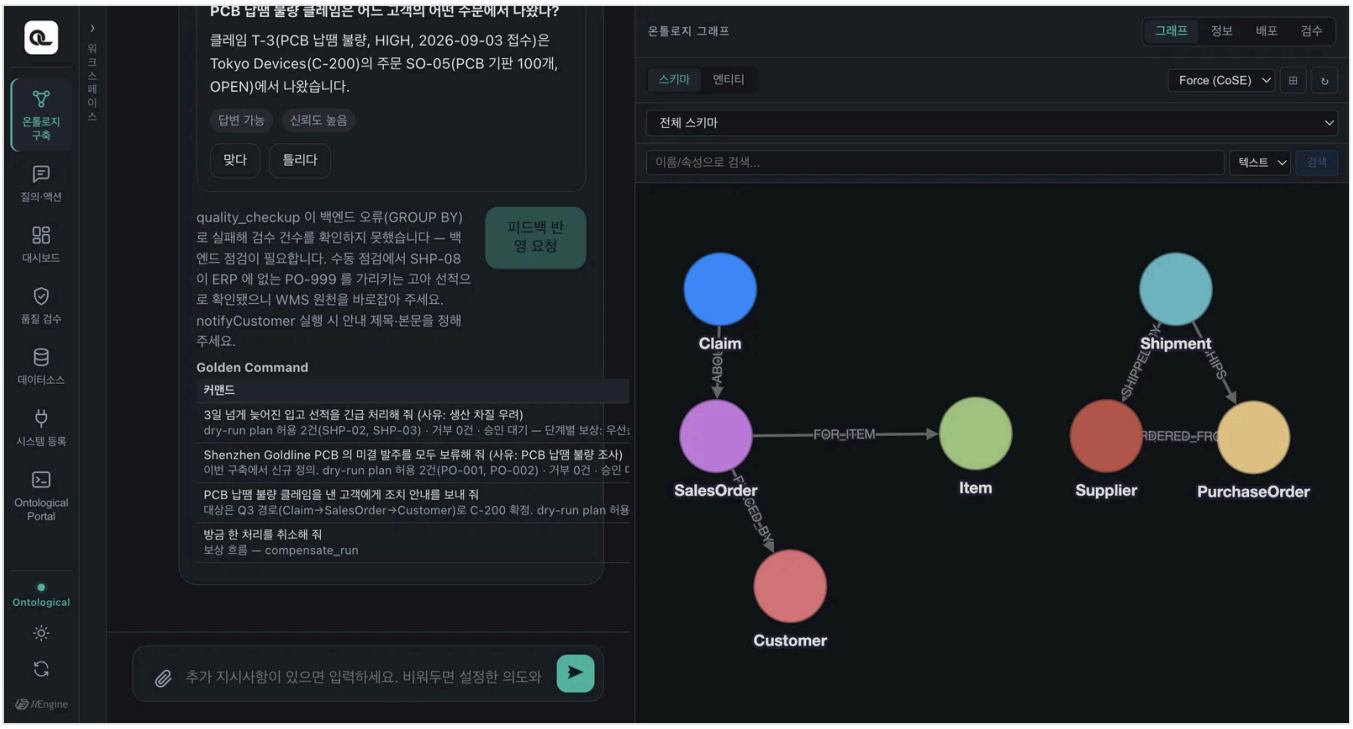

*골든 커맨드 목록 — 두 번째 커맨드에는 아직 연결된 액션이 없습니다 (시연, 밤 테마)*

에이전트는 등록된 시스템의 API를 살핍니다. 발주를 보류하는 API와 보류를 푸는 API가 있다는 것을 찾아, 보류를 단계로, 해제를 보상으로 묶고, 정책을 붙입니다(시연 실측 약 4분 반).




*구축 에이전트가 ERP의 API를 살펴 보류·해제를 단계와 보상으로 묶는 중 (시연, 밤 테마)*

끝으로 실제로 실행하지 않고 계획만 세워 보는 **드라이런**으로 골든 커맨드를 검증합니다. 대상이 몇 건이고, 정책을 통과했는지, 승인이 필요한지가 보고서에 남습니다.




*구축 보고 — 골든 커맨드마다 어떤 액션으로 충족됐고 드라이런 결과가 어땠는지 (시연, 밤 테마)*

> **[팁] 이미 같은 일을 하는 액션이 있다면**
>
> “한빛정밀의 미결 발주를 보류해 줘”는 사실 기존 액션 `holdPurchaseOrder` 로 할 수 있습니다(대상 조건만 다름). 이럴 때 에이전트는 새 액션을 만들지 않고 “기존 액션으로 충족된다”고 보고합니다. 정말 새 액션이 생기는 걸 보고 싶다

> 면, 기존 액션으로 할 수 없는 일을 시켜 보세요 — 예를 들어 CRM의 `createNotice`로 "납기가 늦어지는 발주의 품목을 산 고객에게 지연 안내를 보내 줘".

## 어떤 액션이 있는지 확인하기

질의·액션 탭에서 이렇게 물어보면 됩니다.

```
어떤 액션을 쓸 수 있어?
```

에이전트가 지금 쓸 수 있는 액션(여러분의 역할로 허용된 것만)과 각 액션의 대상·입력·단계를 알려 줍니다.

> **[확인 문제]**
>
> 1. `PurchaseOrder.holdPurchaseOrder`의 보상은 무엇을 부를까요? 그 보상의 인자에 `$before`가 필요할까요?
> 2. 고객 안내(`Customer.notifyCustomer`)의 보상은 `withdrawNotice`입니다. 철회할 안내의 번호는 어디서 알 수 있을까요?
> 3. "공장장만, 자기 공장의 설비에만, 한 번에 3대까지" 쓸 수 있는 액션의 정책 조건을 생각해 보세요.
>
> **▼ 정답 보기**
>
> 1. ERP의 `releasePurchaseOrder`(보류 해제). 예제 ERP는 보류할 때 보류 전 상태를 스스로 기록해 두고, 해제하면 그 상태로 되돌리므로 `$before`가 필요 없습니다. 시스템이 이전 상태를 기억하지 않는다면 `$before.status`를 넘겨야 합니다.
> 2. 앞 단계(`createNotice`)의 응답에 있는 안내 번호 — `$steps.crm.response.id`.
> 3. 규칙 `{"roles": ["plant_manager"], "when": "target.plant == actor.plant"}`. "한 번에 3대까지"는 정책이 아니라 액션의 최대 대상 수(max targets) 설정으로 겁니다.

# 액션 실행하기 — 계획·승인·실행·취소

*분류: 액션 · 16장 · 원문: https://www.uengine.org/docs/ontological-studio/actions-run.html*

> 질의·액션 탭에서 지시를 내려 늦은 선적 두 건을 긴급 처리하고, 원천 DB에서 결과를 확인한 뒤, 취소로 원래 상태로 되돌립니다. 실제로 예제 업무 시스템의 데이터가 바뀝니다.

▶ 영상: 액션 실행하기 — 계획·승인·실행·취소 — 영상 3:45

## 네 단계

```
지시 ──▶ ① 계획 ──▶ ② 승인 ──▶ ③ 실행 ──▶ (필요하면) ④ 취소 = 역순 보상
         대상·정책 판정    사람이        단계별 API 호출       성공한 단계를
         아직 안 바뀜      계획 카드에서  (사가)                거꾸로 되돌림
```

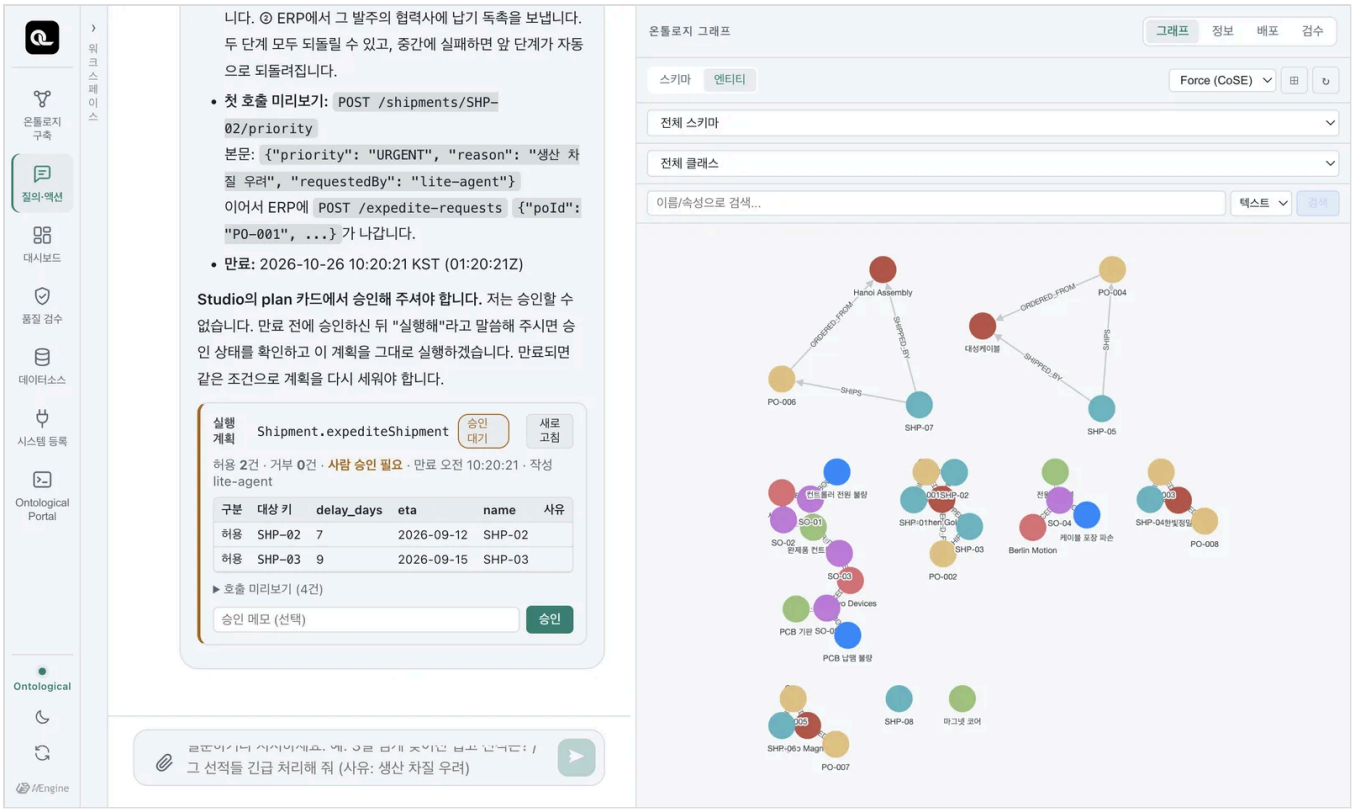
에이전트는 계획을 세우고 실행을 요청할 수 있지만, **승인은 할 수 없습니다.** 승인은 사용자가 계획 카드에서 직접 진행합니다.

## ① 계획 — 지시하기

질의·액션 탭에서 지시합니다. 13장에서 늦은 선적을 물어본 대화에 이어서 해도 됩니다.

```
3일 넘게 늦어진 입고 선적을 긴급 처리해 줘. 사유는 생산 차질 우려.
```

에이전트가 액션 `Shipment.expediteShipment`를 고르고, 실행기에 계획을 요청합니다(실측 25~50초).




*계획 카드 — 허용 2건(SHP-02, SHP-03), 거부 0건, 사람 승인 필요, 만료 시각. 위에는 첫 호출의 미리보기가 있습니다*

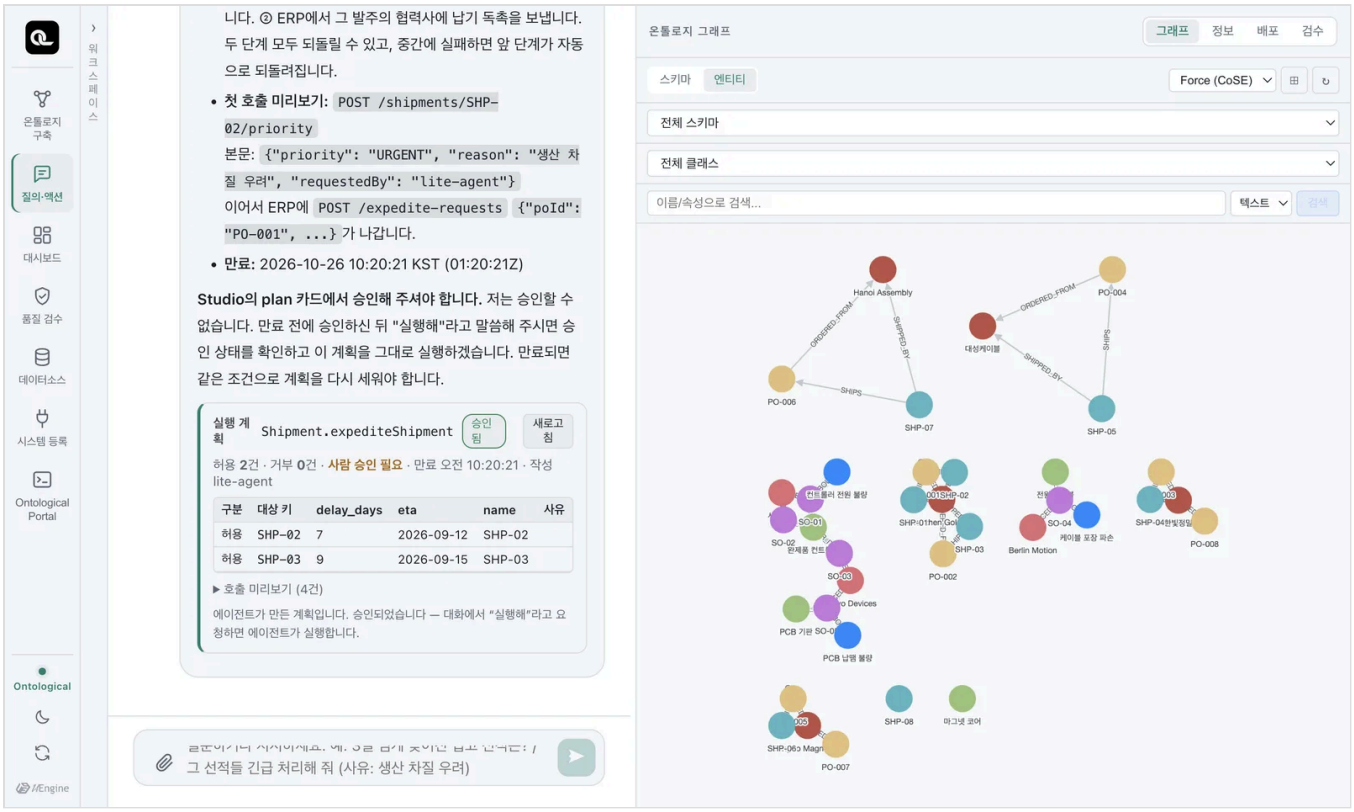
계획 카드에서 볼 것:

| 항목 | 뜻 |
|---|---|
| `Shipment.expediteShipment` · 승인 대기 | 어떤 액션의 계획이고 지금 상태가 무엇인지 |
| 허용 2건 · 거부 0건 | 대상마다 정책을 판정한 결과. 거부된 대상에는 사유가 붙습니다(예: “입고가 끝난 선적은 제외”) |
| 사람 승인 필요 | 이 계획은 승인 없이 실행되지 않습니다 |
| 만료 | 계획은 일정 시간 안에 승인·실행해야 합니다. 지나면 같은 조건으로 다시 계획합니다 |
| 호출 미리보기 (4건) | 실제로 어떤 API를 어떤 인자로 부를지(2건 × 2단계) |

**아직 아무것도 바뀌지 않았습니다.** 계획은 드라이런입니다.

## ② 승인

계획 카드의 「승인」을 누릅니다. 메모를 남길 수 있습니다. 카드에 “승인되었습니다 — 대화에서 ‘실행해’라고 요청하면 에이전트가 실행합니다”라고 나옵니다.

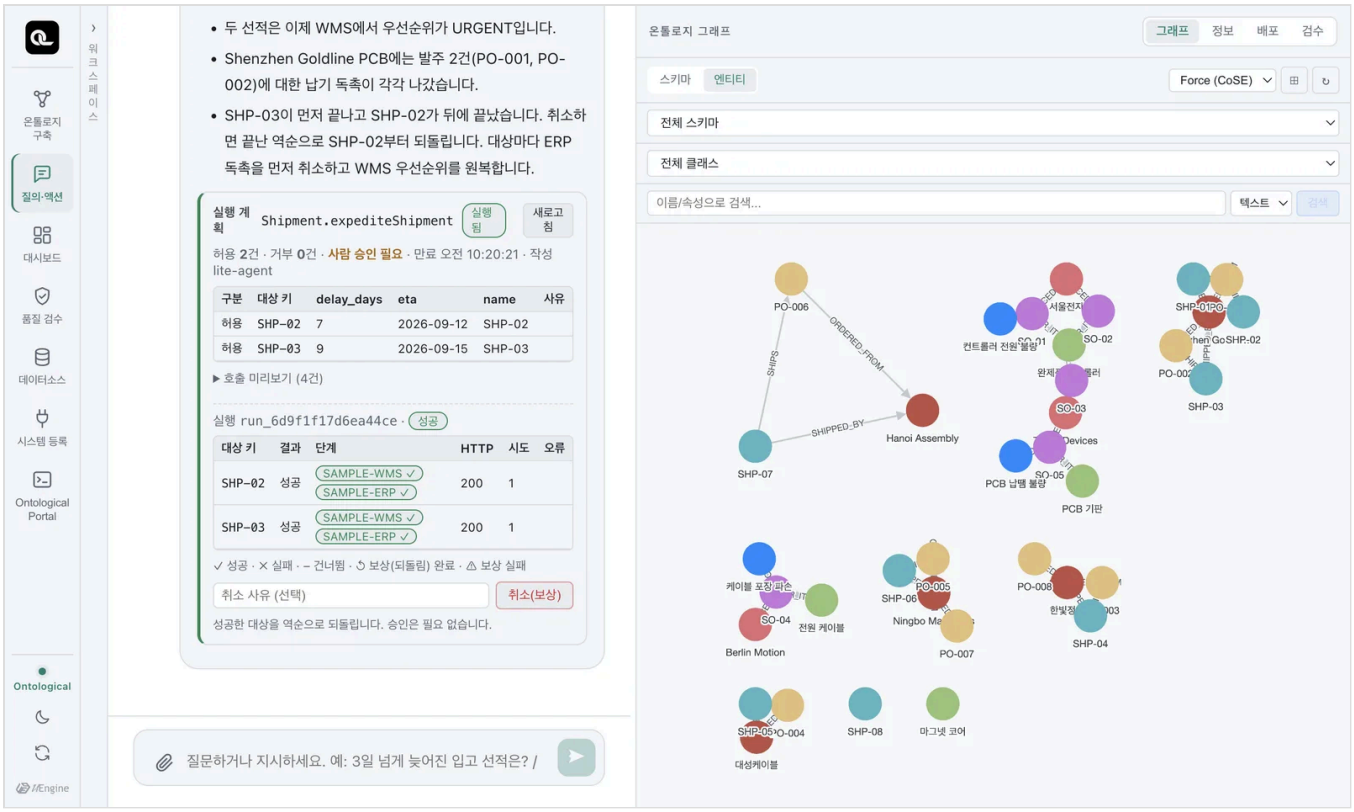



*승인한 계획 — 승인자와 시각이 기록됩니다*

## ③ 실행

```
실행해
```

에이전트는 승인된 계획을 **그대로** 실행합니다(다시 계획하지 않습니다). 실행기는 대상마다 각 단계를 차례로 실행하고, 실행 직전에 정책을 한 번 더 판정합니다.



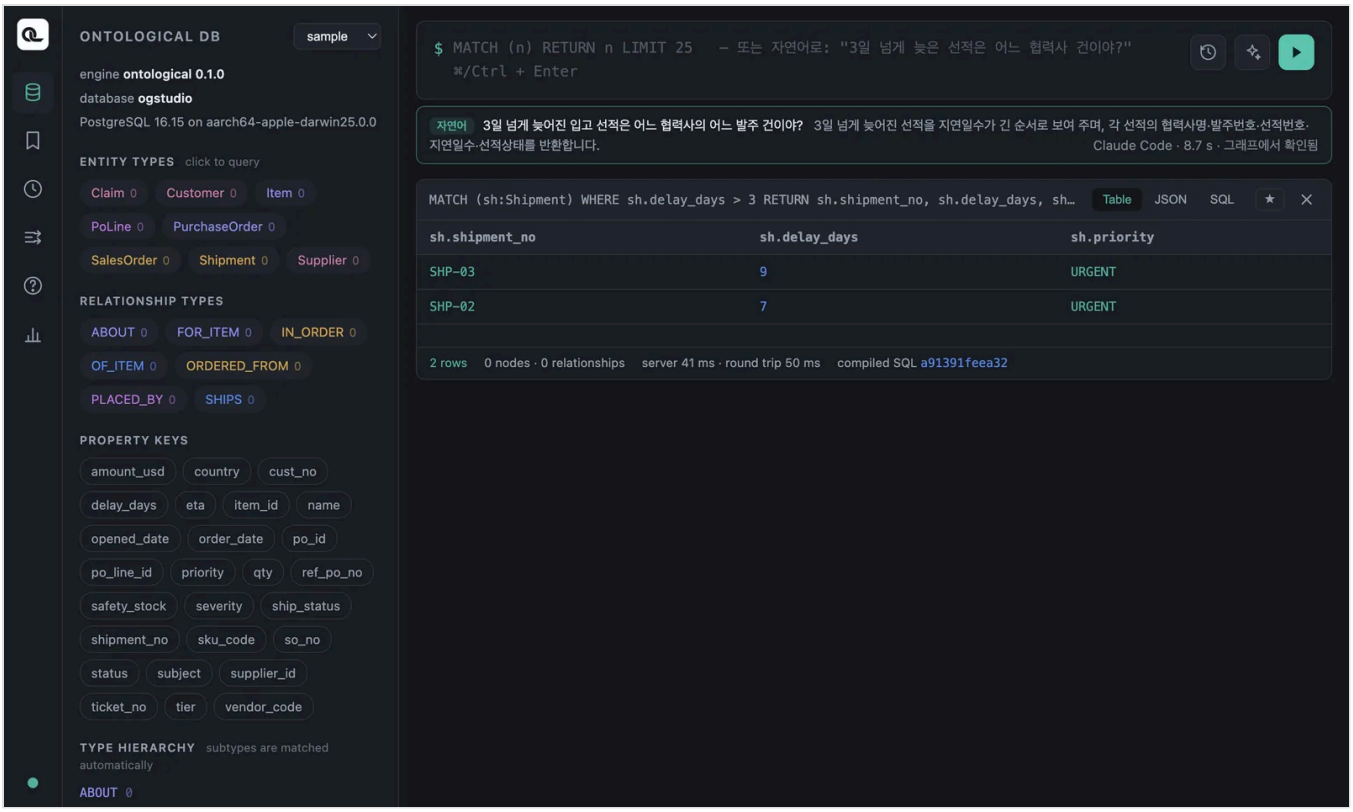

*실행 결과 — 두 선적 모두 WMS 우선순위 NORMAL → URGENT, ERP 납기 독촉 발송, 성공*

| 대상 | WMS 우선순위 | ERP 납기 독촉 | 결과 |
|---|---|---|---|
| SHP-02 | ✓ NORMAL → URGENT | ✓ PO-001 → S-CN-01, 독촉 발송 | 성공 |
| SHP-03 | ✓ NORMAL → URGENT | ✓ PO-002 → S-CN-01, 독촉 발송 | 성공 |

## 정말 바뀌었을까 — 원천에서 확인하기

포털(그래프 `sample`)에서 원천 WMS를 그대로 읽어 봅니다.

```
MATCH (sh:Shipment) WHERE sh.delay_days > 3
RETURN sh.shipment_no, sh.delay_days, sh.priority
```



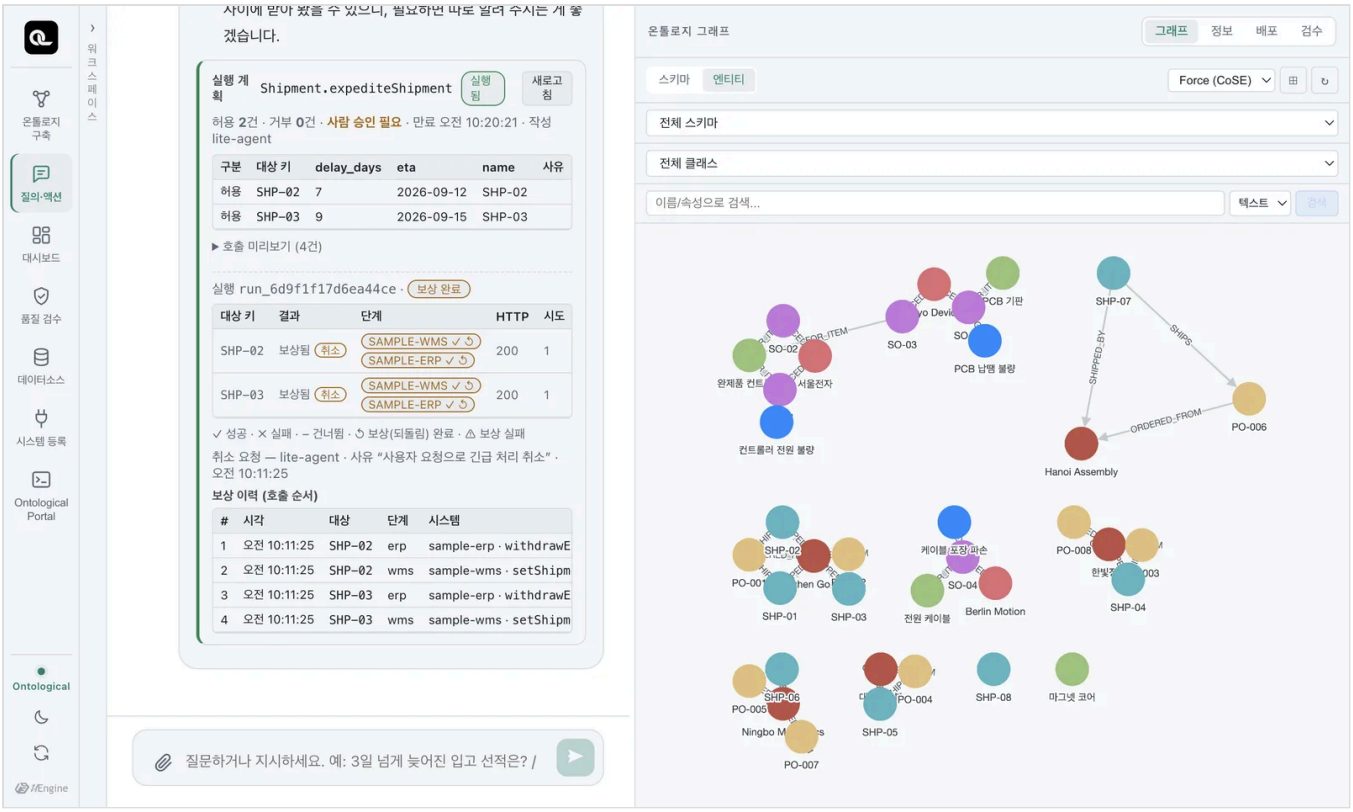

*원천 확인 — 두 선적의 priority가 URGENT로 바뀌었습니다. 복사본이 아니라 WMS 테이블을 지금 읽은 값입니다*

데이터소스 탭에서 `wms.shipment` 미리보기를 봐도 같습니다. 업무 시스템이 원천 데이터를 변경하고, 데이터 패브릭이 변경된 데이터를 읽기 때문입니다.

## ④ 취소 — 역순 보상

```
방금 한 긴급 처리를 취소해 줘.
```

에이전트가 실행 기록을 찾아 **보상**을 요청합니다. 실행기는 성공한 단계를 **역순으로** 되돌립니다 — 나중에 한 일(ERP 독촉)을 먼저 철회하고, 먼저 한 일(WMS 우선순위)을 나중에 되돌립니다.




*보상 완료 — 대상마다 ERP 독촉 철회 → WMS 우선순위 원복 순서로, 모두 보상 성공*

| # | 대상 | 단계 | 시스템 · API | 결과 |
|---|---|---|---|---|
| 1 | SHP-03 | erp | sample-erp · `withdrawExpediteRequest` | 보상 성공 (200) |
| 2 | SHP-03 | wms | sample-wms · `setShipmentPriority` | 보상 성공 (200) |
| 3 | SHP-02 | erp | sample-erp · `withdrawExpediteRequest` | 보상 성공 (200) |
| 4 | SHP-02 | wms | sample-wms · `setShipmentPriority` | 보상 성공 (200) |

대상 순서도 실행의 역순입니다 — 나중에 처리한 SHP-03을 먼저 되돌립니다.

포털에서 다시 확인하면 두 선적의 `priority`가 `NORMAL`로 돌아와 있습니다. WMS 우선순위의 보상은 `$before.priority` — **실행 직전의 값**으로 되돌리기 때문입니다. 원래 `HIGH`였다면 `HIGH`로 돌아갑니다.

## 사가 — 중간에 실패하면

여러 시스템을 차례로 부르다가 중간에 하나가 실패하면 어떻게 될까요? 예를 들어 SHP-03의 WMS 단계는 성공했는데 ERP 단계가 실패했다면, WMS만 URGENT인 어중간한 상태가 남습니다.

실행기는 이것을 **사가(saga)** 패턴으로 막습니다. 어떤 대상의 단계가 실패하면 그 대상에서 **이미 성공한 단계를 역순으로 자동 보상**합니다. 그 대상만 원래대로 돌아가고, 다른 대상은 성공한 채로 남으며, 결과는 "일부 성공"으로 보고됩니다. 시스템마다 상태가 엇갈리는 일이 생기지 않습니다.

| 상황 | 실행기의 처리 |
|---|---|
| 모든 단계 성공 | 성공 |
| 한 대상의 2단계 실패 | 그 대상의 1단계를 자동 보상 → 그 대상은 원상태, 결과는 "일부 성공" |

| 상황 | 실행기의 처리 |
|---|---|
| 같은 요청이 두 번 감 | 대상마다 멱등성 키가 붙어 있어 업무 시스템이 두 번 처리하지 않음 |
| 계획 뒤 데이터가 바뀜 | 실행 직전 정책을 다시 판정해, 이제 맞지 않는 대상은 건너뜀 |

## 지시를 잘 하는 법

- **대상 조건**을 분명히: “3일 넘게 늦은”, “Shenzhen Goldline PCB의 미결”.
- **사유**를 함께: 필수 입력이면 에이전트가 물어봅니다. 실행 기록과 업무 시스템에 남습니다.
- 계획 카드의 **거부 사유**를 읽으세요. 정책이 왜 막았는지가 보입니다. 정책을 바꾸고 싶다면 그것은 구축(액션 정의)의 일입니다.

> **[확인 문제] 실습 과제**
>
> 1. “Shenzhen Goldline PCB의 미결 발주를 모두 보류해 줘 (사유: PCB 납땜 불량 조사)”를 지시하고, 계획 카드에서 허용·거부된 발주와 그 사유를 확인하세요.
> 2. 승인·실행한 뒤 데이터소스 탭에서 `erp.purchase_order`의 상태가 `ON_HOLD`로 바뀌었는지 보고, `erp.po_hold` 테이블에 무엇이 기록됐는지 보세요.
> 3. 취소하고, 상태가 `OPEN`으로 돌아왔는지 확인하세요.
>
> **▼ 정답 보기**
>
> 1. 허용: PO-001, PO-002 (모두 OPEN). Shenzhen Goldline PCB의 발주는 이 두 건뿐이라 거부는 0건입니다. (만약 “모든 협력사의 발주를 보류해 줘”라고 하면 CLOSED인 PO-003, PO-004가 “미결 발주만” 사유로 거부됩니다. 에이전트가 대상을 고를 때 이미 `status = 'OPEN'`으로 걸러 두면, 거부 행 없이 처음부터 대상에서 빠진 채로 계획이 나올 수도 있습니다.)
> 2. `erp.purchase_order`의 PO-001, PO-002가 `ON_HOLD`. `erp.po_hold`에는 발주마다 사유·보류 전 상태(`prev_status = OPEN`)·요청자가 한 줄씩 생깁니다. `erp.po_hold`는 가상 노드가 아니므로 데이터소스 탭의 테이블 상세에서 컬럼만 보입니다 — 내용을 보려면 가상 노드로 노출하세요(6장).
> 3. 보상(`releasePurchaseOrder`)이 보류 전 상태로 되돌려 `OPEN`이 됩니다.

# 대시보드 앱 — 말로 만드는 화면

*분류: 애플리케이션 · 17장 · 원문: https://www.uengine.org/docs/ontological-studio/dashboard.html*

> 「온톨로지 앱」에서 클래스와 속성을 @멘션해 대시보드를 만듭니다. 엔진이 기본 위젯을 선택하고 에이전트가 요청에 맞는 구성안을 제안하면, 사용자가 이를 검토하고 적용합니다.

▶ 영상: 대시보드 앱 — 말로 만드는 화면 — 영상 3:51

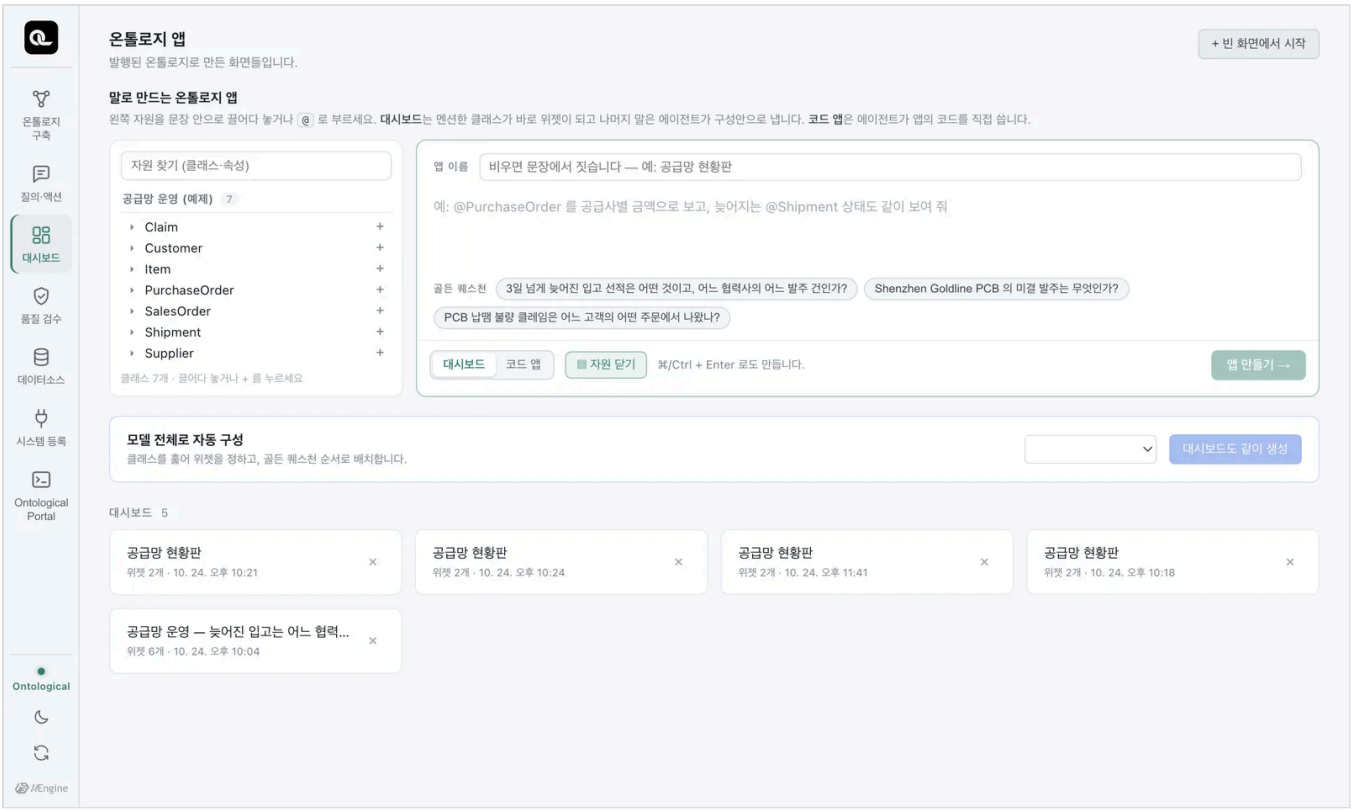
## 두 가지 앱

같은 온톨로지 위에서 앱을 두 가지 방식으로 만듭니다.

| | 대시보드 | 코드 앱 |
|---|---|---|
| 만드는 법 | 위젯(차트·테이블)을 조합 | 에이전트가 HTML·JavaScript 코드를 씀 |
| 빠르기 | 위젯은 약 1초, 구성안은 수십 초 | 1~3분 |
| 자유도 | 정해진 위젯 안에서 | 무엇이든 (화면 구성·상호작용) |
| 고치기 | 위젯 설정, 또는 말로 | 말로, 또는 코드를 직접 |
| 다루는 장 | 이 장 | 18장~19장 |

## 앱 만들기 화면

홈의 「온톨로지 앱」 카드를 엽니다. 맨 위가 「말로 만드는 온톨로지 앱」입니다.

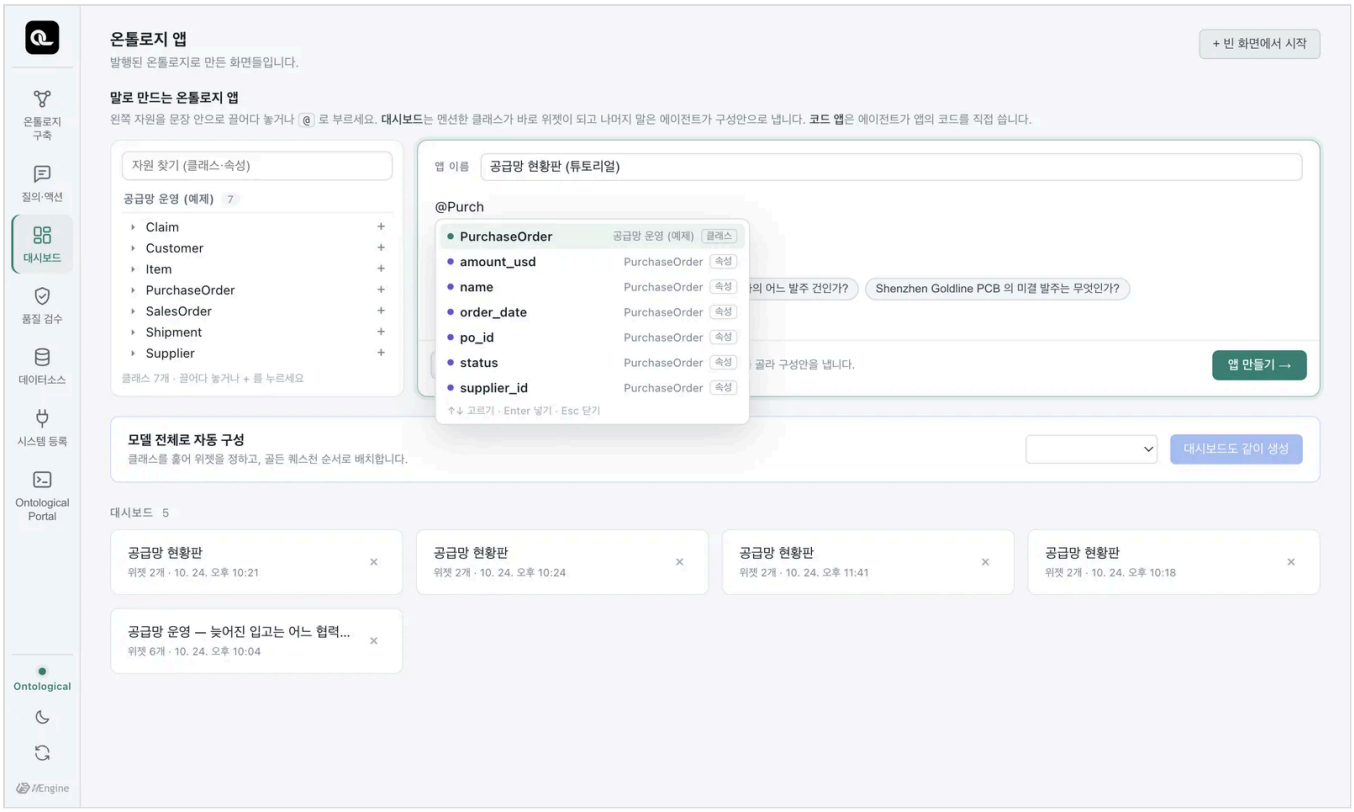



*온톨로지 앱 — 왼쪽 자원 트리(모델 › 클래스 › 속성), 오른쪽 입력창과 골든 퀘스천 예시, 아래에 만든 앱 갤러리*

| 영역 | 내용 |
|---|---|
| 왼쪽 자원 트리 | 발행된 온톨로지의 모델 › 클래스 › 속성. 문장 안으로 끌어다 놓을 수 있습니다. |
| 입력창 | 만들고 싶은 화면을 설명합니다. `@` 를 치면 클래스·속성 후보가 뜹니다. |
| 예시 칩 | 이 온톨로지의 골든 퀘스천. 누르면 입력창에 들어갑니다. |
| 대시보드 / 코드 앱 | 무엇으로 만들지 고릅니다. |
| 갤러리 | 만든 앱들 |

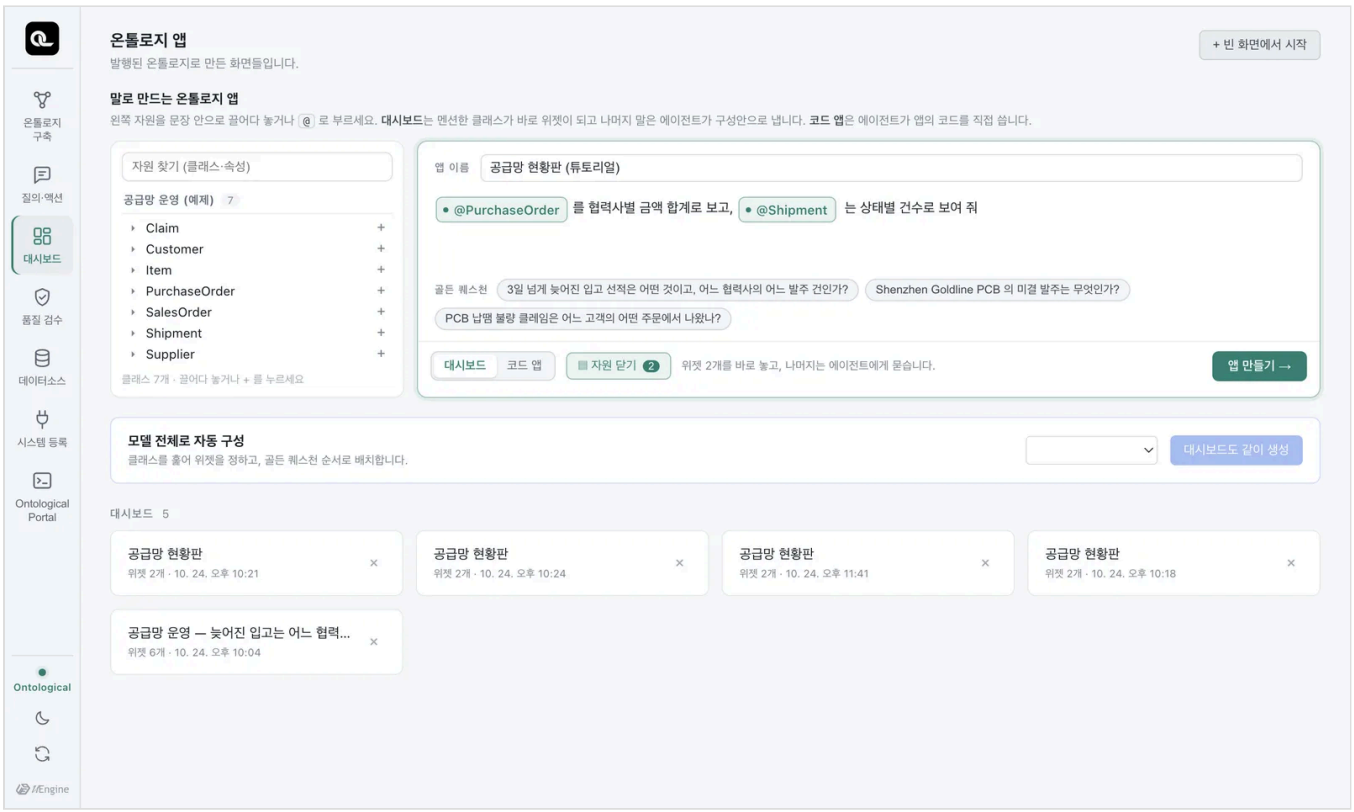
## 따라 하기 — 공급망 현황판

1. 앱 이름에 `공급망 현황판` 을 적고, 「대시보드」를 고릅니다.
2. 입력창에 `@Purch` 까지 치면 후보가 뜹니다. ↑↓로 `PurchaseOrder` 를 고르고 Enter.
3. 이어서 `를 협력사별 금액 합계로 보고,` 를 칩니다.
4. 왼쪽 트리에서 `Shipment` 를 문장 안으로 끌어다 놓고, `는 상태별 건수로 보여 줘` 를 칩니다.
5. 「앱 만들기」를 누릅니다.




*@를 치면 클래스·속성 후보가 뜹니다*

고른 자원은 **칩**이 되어 문장 안에 남습니다. 사람도 에이전트도 무엇을 가리키는지 헷갈리지 않습니다. 칩은 백스페이스 한 번에 통째로 지워집니다.

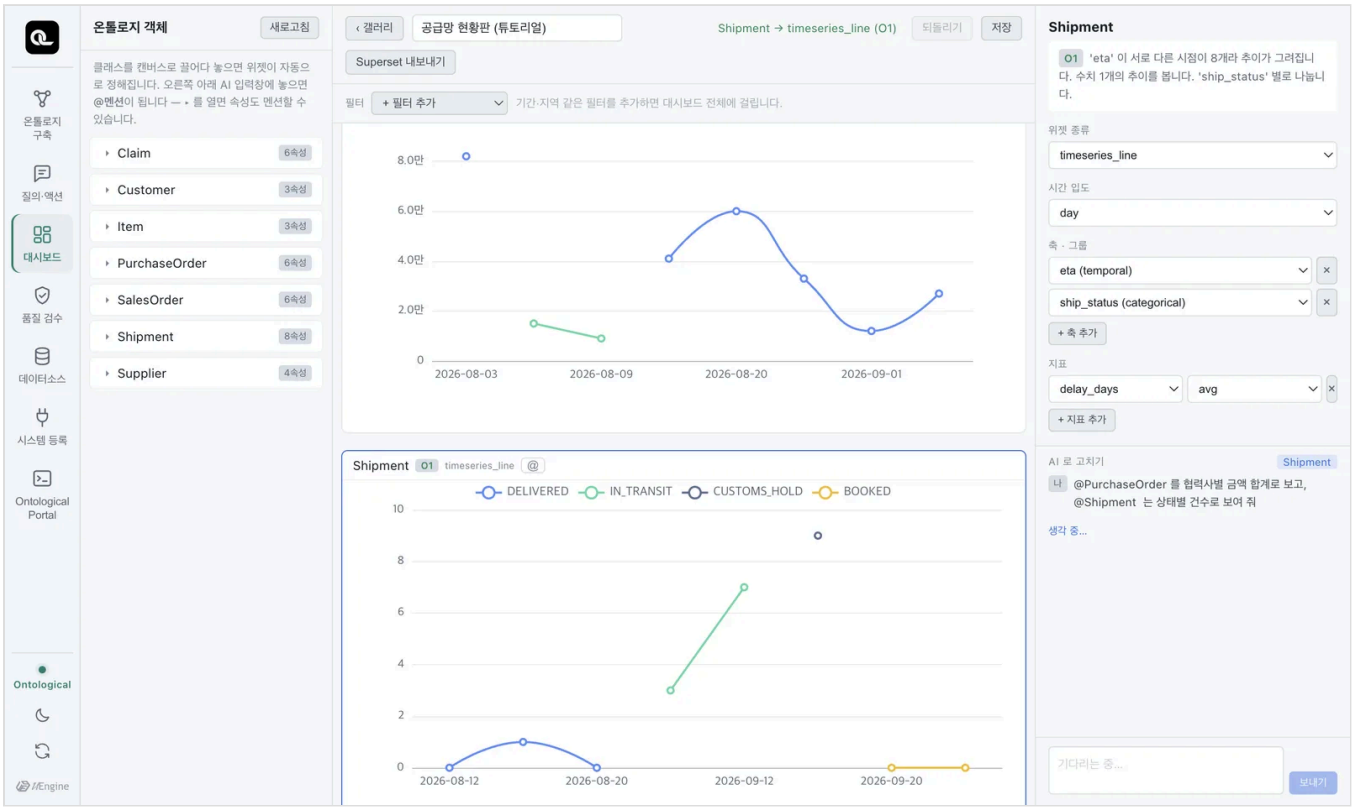



*칩 두 개(@PurchaseOrder, @Shipment)가 들어간 요청 문장*

## 엔진이 먼저, 에이전트가 다음

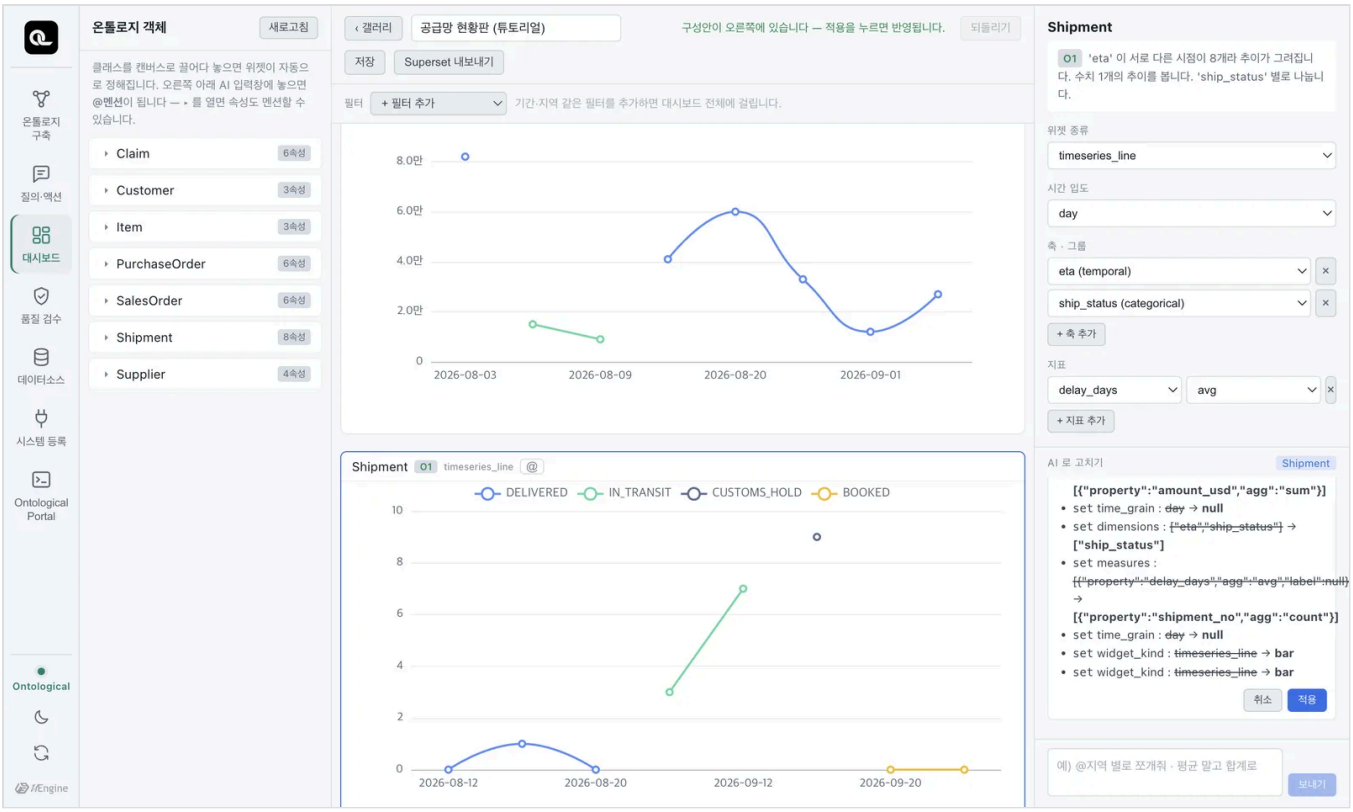
「앱 만들기」를 누르면 두 가지가 차례로 일어납니다.

**1. 엔진이 위젯을 바로 정합니다(약 1초).** 멘션한 클래스마다 데이터를 살펴(속성의 타입, 값의 종류) 적당한 위젯을 고릅니다. 이 단계는 규칙대로 움직이므로 늘 같은 결과가 나옵니다. 처음에는 날짜별 추이 그래프가 놓였습니다.



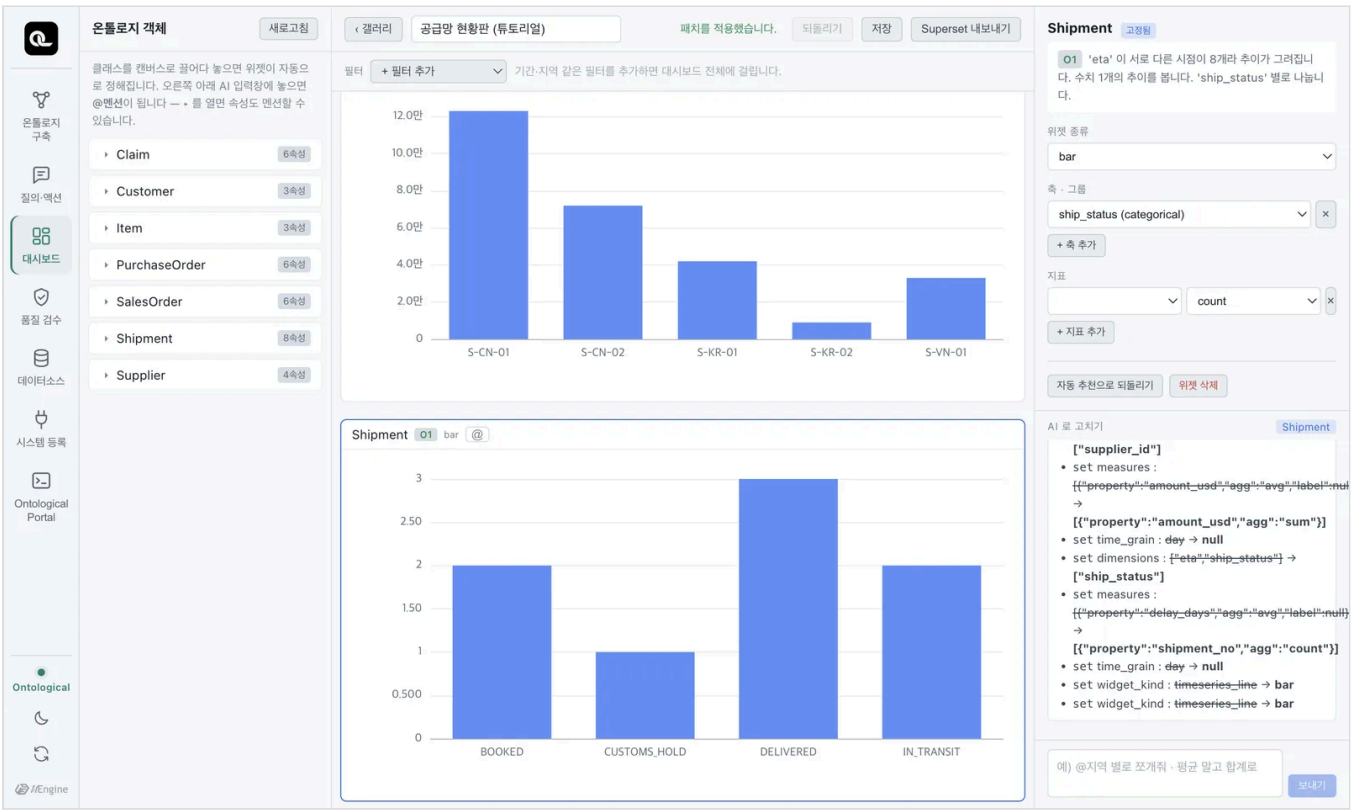

*위젯 두 개가 바로 놓였습니다 — 아직은 날짜별 추이 그래프*

**2. 에이전트가 문장의 나머지를 구성안으로 냅니다.** “협력사별 금액 합계”, “상태별 건수”는 에이전트에게 갑니다. 에이전트는 지금 구성을 읽고 무엇을 바꿀지 **구성안(diff)**을 냅니다(실측 약 35초). 오른쪽 「AI로 고치기」에 바뀔 내용이 취소선과 함께 보입니다.




*에이전트의 구성안 — 축을 supplier_id와 ship_status로, 집계를 sum과 count로, 종류를 막대로. 「적용」을 기다립니다*

**3. 사람이 적용합니다.** 구성안은 「적용」을 눌러야 반영됩니다. 마음에 안 들면 적용하지 않고 말로 다시 고쳐 달라고 하면 됩니다.




*적용 후 — 협력사별 발주 금액 합계 막대와 선적 상태별 건수 막대*

## 이어서 고치기

- **위젯을 멘션**: 위젯 상단의 `@`를 누르면 그 위젯이 입력창에 칩으로 들어갑니다. “@Shipment 위젯을 원형 그래프로” 처럼 이어 고칩니다.
- **직접 설정**: 위젯을 고르면 오른쪽에 종류·축·지표 설정이 나옵니다. 「자동 추천으로 되돌리기」도 있습니다.
- **필터**: 위쪽 「+ 필터 추가」로 대시보드 전체에 걸리는 조건을 겁니다.
- **선택 연동**: 막대 하나를 누르면 그 값이 대시보드의 선택이 되어, 관련 위젯에 필터로 걸립니다.
- **저장**: 오른쪽 위 「저장」. 「되돌리기」로 방금 적용한 구성안을 되돌립니다.

낮/밤 테마를 바꾸면 차트의 축과 글자 색도 함께 바뀝니다.

> **[확인 문제] 실습 과제**
>
> 1. `@Claim`을 심각도별 건수로, `@SalesOrder`를 고객별 수량 합계로 보여 주는 「고객 품질 현황」 대시보드를 만드세요.
> 2. 위젯 하나를 멘션해 테이블로 바꿔 달라고 요청해 보세요.
> 3. 「대시보드」 대신 「코드 앱」을 고르면 무엇이 달라질지 예상해 보고, 다음 장으로 가세요.
>
> **▼ 도움말**
>
> 1. 예: “@Claim 은 severity 별 건수로, @SalesOrder 는 고객별 qty 합계로 보여 줘”. 속성도 `@severity`처럼 멘션할 수 있습니다(트리에서 클래스를 펼치면 속성이 보입니다).
> 2. 위젯 상단의 `@`를 누르고 “테이블로 바꿔 줘”라고 입력합니다. 구성안이 나오면 「적용」을 누릅니다.

# 코드 앱 — 에이전트가 코드를 쓰는 앱

*분류: 애플리케이션 · 18장 · 원문: https://www.uengine.org/docs/ontological-studio/code-app.html*

> 같은 입력창에서 「코드 앱」을 고르면, 코딩 에이전트가 온톨로지 질의를 시험하면서 업무 앱 코드를 작성합니다. 생성된 코드를 확인하고 자연어로 수정한 뒤 앱 탭으로 열어 봅니다.

▶ 영상: 코드 앱 — 에이전트가 코드를 쓰는 앱 — 영상 3:44

## 무엇이 만들어지나

코드 앱은 빌드가 필요 없는 작은 웹 앱입니다.

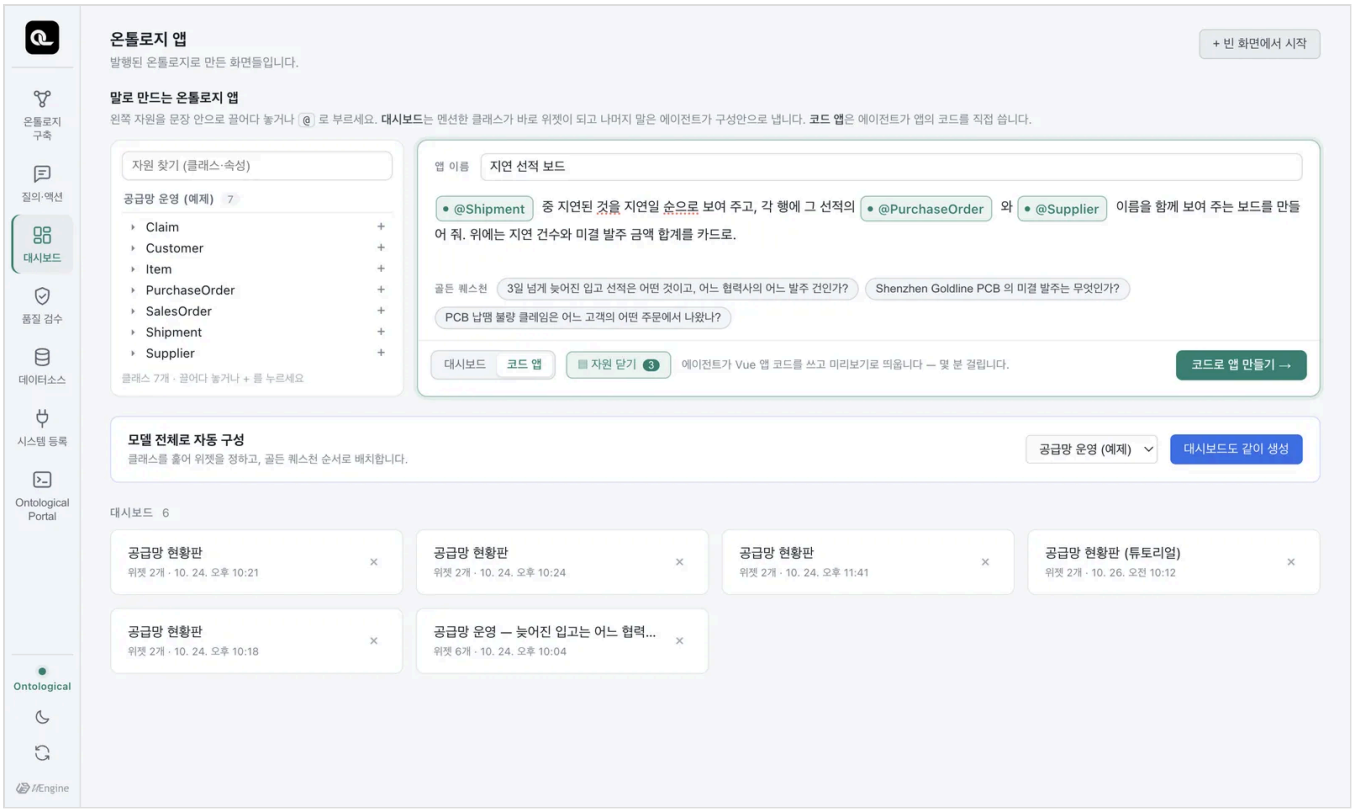
| 파일 | 내용 |
|---|---|
| `index.html` | 화면의 뼈대. 런타임( `/apps/_runtime/…` )을 불러옵니다. |
| `app.js` | Vue 3 앱 코드. 온톨로지 질의와 화면 로직 |
| `style.css` | 화면 꾸밈. 색은 테마 변수( `--og-*` )로만 칠합니다. |

앱이 사용하는 라이브러리(Vue 3, ECharts)와 SDK( `og.js` )는 Ontological Studio에 포함되어 있어 인터넷 없이도 실행됩니다. 데이터는 **읽기 전용 Cypher**로만 가져옵니다.

## 따라 하기 — 지연 선적 보드

1. 「온톨로지 앱」 화면에서 「코드 앱」을 고르고, 앱 이름을 `지연 선적 보드` 로 적습니다.
2. 입력창에 이렇게 적습니다(@는 멘션으로):
   `@Shipment 중 지연된 것을 지연일 순으로 보여 주고, 각 행에 그 선적의 @PurchaseOrder 와 @Supplier 이름을 함께 보여 주는 보드를 만들어 줘. 위에는 지연 건수와 미결 발주 금액 합계를 카드로.`
3. 「코드로 만들기」를 누릅니다.

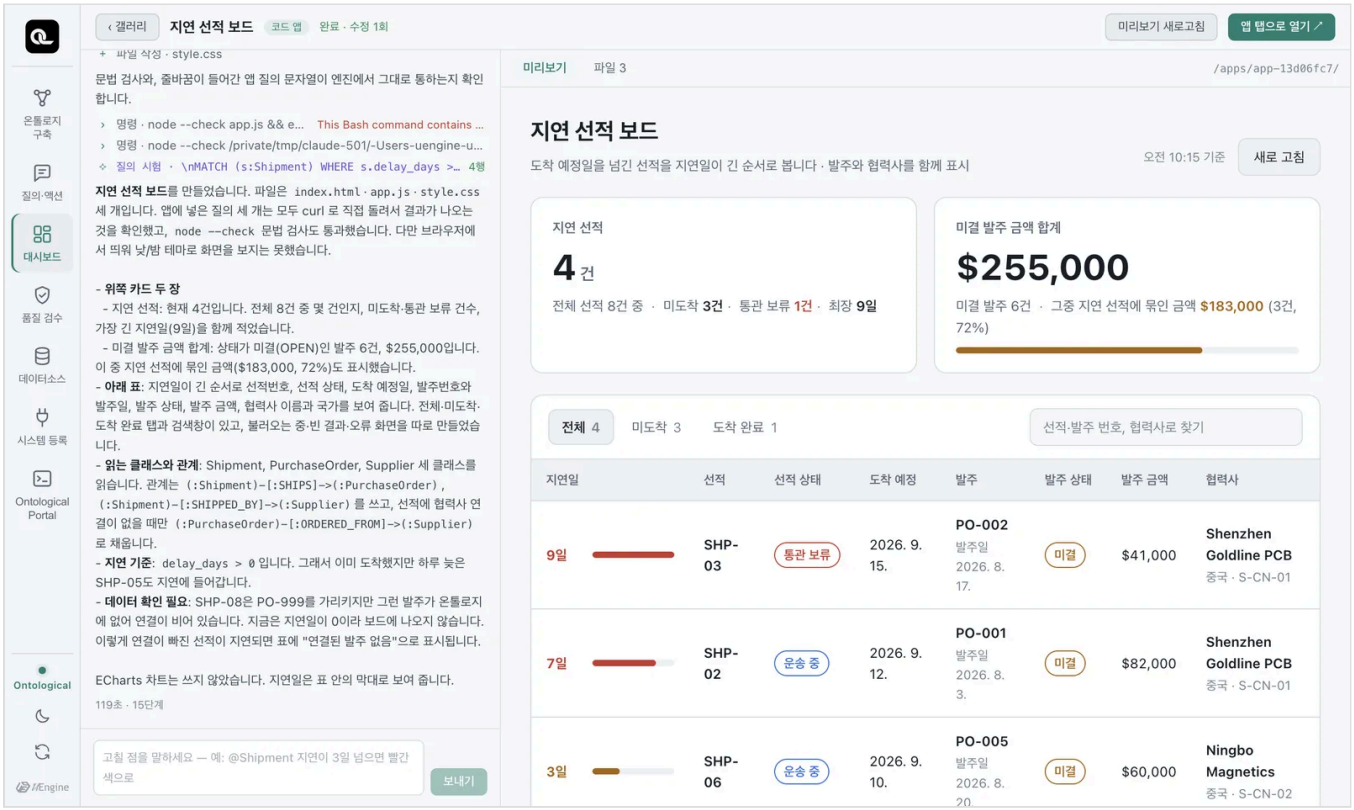
> **[팁] 멘션 후보 고르기**
>
> `@Supplier` 를 적으면 이름이 정확히 같은 클래스가 맨 위에 옵니다. 12장 연습으로 만든 스키마에도 비슷한 클래스( `QualitySupplier` 등)가 있으면 후보에 함께 뜨니, 오른쪽에 **「공급망 운영 (예제)」**라고 적힌 것을 고르세요.




*코드 앱 요청 — 같은 멘션 입력창에서 「코드 앱」을 고른 모습*

코딩 에이전트(여기서는 Claude Code)가 앱 폴더에서 일을 시작합니다. 왼쪽에 무엇을 하는지가 그대로 흐릅니다.

1. **질의를 먼저 시험합니다** — 온톨로지에서 Cypher를 직접 실행해 데이터 형식을 확인합니다(「질의 시험 · … 4행」).
2. **파일을 씁니다** — `index.html`, `app.js`, `style.css`.
3. **문법을 검사합니다** — `node --check app.js`.
4. **보고합니다** — 무엇을 만들었고, 어떤 클래스와 관계를 읽는지, 못 한 것은 무엇인지.



*2분 뒤 — 왼쪽에 에이전트의 보고, 오른쪽 미리보기에 실제 온톨로지 데이터로 채워진 보드*

실측으로 121초가 걸렸습니다. 보고서를 읽어 보면 에이전트가 데이터의 이상까지 짚었습니다 — "SHP-08은 PO-999를 가리키지만 그런 발주가 온톨로지에 없어 연결이 비어 있습니다." 8장에서 OPTIONAL MATCH로 찾았던 그 선적입니다.

## 코드 읽기

「파일」 탭에서 코드를 봅니다. `app.js`의 핵심은 Cypher 문자열과 `query()` 호출입니다(아래는 실제로 만들어진 코드에서 몇 줄을 줄여 옮긴 것입니다).

*「파일」 탭 — index.html·app.js·style.css*

```
import { createApp, ref, computed, onMounted } from '/apps/_runtime/vue.js'
import { query } from '/apps/_runtime/og.js'

// 지연된 선적 — 싣고 오는 발주(SHIPS)와 보낸 협력사(SHIPPED_BY)를 함께 읽는다.
const Q_DELAYED = `
MATCH (s:Shipment) WHERE s.delay_days > 0
OPTIONAL MATCH (s:Shipment)-[:SHIPS]->(p:PurchaseOrder)
OPTIONAL MATCH (s:Shipment)-[:SHIPPED_BY]->(v:Supplier)
OPTIONAL MATCH (p:PurchaseOrder)-[:ORDERED_FROM]->(pv:Supplier)
RETURN s.shipment_no AS shipment_no, s.delay_days AS delay_days, s.eta AS eta,
       s.ship_status AS ship_status, p.po_id AS po_id, p.amount_usd AS po_amount,
       v.name AS supplier, pv.name AS po_supplier
ORDER BY delay_days DESC`

const Q_OPEN_PO = `
MATCH (p:PurchaseOrder) WHERE p.status = $status
RETURN count(p) AS n, sum(p.amount_usd) AS total`

// …
const [delayed, open] = await Promise.all([
  query(Q_DELAYED),
  query(Q_OPEN_PO, { status: 'OPEN' }),
])
```

4부에서 배운 Cypher가 그대로 들어 있습니다. `OPTIONAL MATCH`로 짝이 없는 선적도 남기고, `$status` 같은 **매개변수**로 값을 따로 넘깁니다. 코드 앱은 `ontology` 그래프(발행된 온톨로지)를 읽으므로, `sample`에는 없는 `SHIPPED_BY` 관계도 씁니다.

## 말로 고치기

왼쪽 아래 입력창에 고칠 점을 말하면, **같은 에이전트 세션**이 이어서 고치고 미리보기가 다시 뜹니다. 멘션도 됩니다.

```
지연이 3일 넘는 @Shipment 은 빨간 배지로 강조하고, 맨 위에 전체 미결 금액 합계를 크게 보여 줘
```

시연에서는 이런 수정이 약 70초 걸렸습니다. 상단의 「완료 · 수정 2회」처럼 몇 번 고쳤는지가 보입니다.

## 앱 탭으로 열기

오른쪽 위 「앱 탭으로 열기」를 누르면 만든 앱이 Studio 옆에 독립된 탭으로 뜹니다. 업무 화면처럼 쓰면 됩니다. 홈의 최근 항목에도 남습니다.

## 에이전트가 지키는 규칙

코딩 에이전트는 앱 폴더의 `AGENTS.md`(Claude Code는 `CLAUDE.md`) 규칙을 따릅니다. 주요 내용:

- 이 폴더 하나가 앱 하나. 폴더 밖을 읽거나 고치지 않는다.
- 빌드 도구(npm, Vite)와 외부 CDN을 쓰지 않는다 — 오프라인에서도 떠야 한다.

- 데이터는 온톨로지에서만. **목 데이터·하드코딩 금지**, 없는 것은 없다고 화면에 쓴다.
- 질의는 쓰기 전에 반드시 직접 시험한다.
- 값을 바꾸는 일은 **액션 버튼**으로만(20장).
- 낮/밤 두 테마에서 모두 읽혀야 한다. 불러오는 중·빈 결과·오류를 각각 보여 준다.
- 끝나면 한국어로 무엇을 만들었는지 보고하고, 못 한 것은 못 했다고 쓴다.

> **[확인 문제] 실습 과제**
>
> 1. "협력사별 미결 발주 금액과 늦어지는 선적을 한눈에 보는 협력사 리스크 보드. 협력사 카드를 누르면 그 협력사의 발주와 선적 목록이 열리게"를 코드 앱으로 만들어 보세요.
> 2. 만들어진 `app.js`에서 Cypher를 하나 골라 포털(그래프 `ontology`)에 붙여 넣고 실행해, 앱의 숫자와 같은지 확인하세요.
> 3. 말로 한 가지 고쳐 보고, 「파일」 탭에서 무엇이 바뀌었는지 찾아보세요.

# 코드 앱 SDK — 직접 읽고 고치기

*분류: 애플리케이션 · 19장 · 원문: https://www.uengine.org/docs/ontological-studio/sdk.html*

> 코드 앱에서 사용하는 SDK( `og.js` )의 함수를 살펴보고, 에이전트 없이 앱 코드를 직접 수정해 봅니다. 프로그래밍 수업에서 JavaScript를 조금 써 봤다면 충분합니다.

▶ 영상: 코드 앱 SDK — 직접 읽고 고치기 — 영상 3:49

## 앱 하나의 구조

```
<데이터 폴더>/codeapps/<앱 id>/
  ├─ index.html     ← /apps/_runtime/og.css · echarts.min.js 를 싣고 app.js 를 모듈로 부른다
  ├─ app.js         ← Vue 3 앱 (ES 모듈)
  ├─ style.css
  ├─ AGENTS.md, CLAUDE.md   ← 에이전트 규칙 (앱이 아니라 에이전트가 읽는 파일)
```

macOS의 데이터 폴더는 `~/Library/Application Support/Ontological Studio/` 입니다. 미리보기 주소는 `/apps/<앱 id>/` 이고, 화면 오른쪽 위에 보입니다.

`index.html` 은 이렇게 생겼습니다. `/apps/_runtime/…` 줄은 지우지 않습니다.

```
<!doctype html>
<html lang="ko">
<head>
  <meta charset="utf-8" />
  <title>지연 선적 보드</title>
  <link rel="stylesheet" href="/apps/_runtime/og.css" />
  <link rel="stylesheet" href="./style.css" />
  <script src="/apps/_runtime/echarts.min.js"></script>
</head>
<body>
  <div id="app"></div>
  <script type="module" src="./app.js"></script>
</body>
</html>
```

## SDK 함수

```
import { query, schema, theme, onTheme, color, actions, act } from '/apps/_runtime/og.js'
```

| 함수 | 하는 일 |
|---|---|
| `await query(cypher, params?)` | **읽기 전용** Cypher를 실행하고 행 배열을 반환합니다. 쓰기 절(`CREATE`·`MERGE`·`SET`·`DELETE`)은 서버가 거절합니다. 실패하면 엔진의 메시지로 `Error`를 던집니다. |
| `await schema()` | 발행된 온톨로지의 클래스·속성·관계. 앱이 스스로 둘러볼 때 |
| `theme()` | `'light'` 또는 `'dark'` — Studio의 테마 설정을 따릅니다 |
| `onTheme(fn)` | 테마가 바뀌면 `fn`을 부릅니다. 차트를 다시 칠할 때. 해제 함수를 반환합니다 |
| `color(name)` | 테마 색 토큰의 값. `color('accent')`, `color('fg')` — 캔버스 차트에 넘길 때 |
| `await actions(className?)` | 발행된 액션 목록(20장) — 0.1.1부터 |
| `await act(opts)` | 계획 → 승인 → 실행 → 취소 창을 띄웁니다(20장) — 0.1.1부터 |

### query()가 돌려주는 모양

```
const rows = await query('MATCH (s:Supplier) RETURN s.name AS name, s.tier AS tier ORDER BY name')
// → [{ name: 'Hanoi Assembly', tier: 2 }, { name: 'Ningbo Magnetics', tier: 2 }, …]
```

(12장 연습 2로 「한빛정밀」을 추가 인제스천했다면 `{ name: '한빛정밀', tier: 1 }`도 함께 나옵니다.)

- `RETURN`의 이름(`AS`)이 객체의 키가 됩니다. 숫자는 숫자로 옵니다.
- 노드 전체를 반환하면(`RETURN s`) 속성 객체에 `_labels`·`_id`가 붙어 옵니다.
- 값은 문자열로 이어 붙이지 말고 **매개변수**로 넘깁니다. 따옴표 문제와 주입 공격을 막습니다.

```
const pos = await query(
  'MATCH (p:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier) WHERE s.name = $name RETURN p.po_id AS po, p.
status AS status',
  { name: 'Shenzhen Goldline PCB' })
```

### 테마 색

`style.css`에서는 색을 이 변수들로만 칠합니다. 그러면 낮/밤이 저절로 따라갑니다.

| 변수 | 쓰임 |
|---|---|
| `--og-bg` `--og-panel` `--og-raised` | 바탕, 패널, 떠 있는 카드 |
| `--og-line` | 선, 테두리 |
| `--og-fg` `--og-dim` `--og-mute` | 글자(진함 → 흐림) |
| `--og-accent` `--og-on-accent` | 강조색과 그 위의 글자 |
| `--og-blue` `--og-warn` `--og-danger` `--og-ok` `--og-purple` `--og-soft` | 상태 색 |

## 따라 하기 — 손으로 고치기

에이전트에게 맡기지 않고 직접 한 줄을 고쳐 봅시다. 18장에서 만든 「지연 선적 보드」의 지연 기준을 "1일 넘게"에서 "3일 넘게"로 바꿉니다.

1. 「파일」 탭에서 위치를 확인하고, 데이터 폴더의 `codeapps/<앱 id>/app.js`를 편집기(VS Code 등)로 엽니다.
2. `WHERE s.delay_days > 0`을 `WHERE s.delay_days > 3`으로 바꾸고 저장합니다.
3. Studio의 「미리보기 새로 고침」을 누릅니다. 4건이던 표가 2건(SHP-03, SHP-02)이 됩니다.

> **[팁] 에이전트와 번갈아 고쳐도 됩니다**
>
> 손으로 고친 뒤 말로 이어서 고치라고 해도, 에이전트는 파일을 다시 읽고 이어서 작업합니다. 반대로 에이전트가 고친 코드를 손으로 다듬어도 됩니다. 코드가 곧 앱입니다.

## 차트 하나 더하기 (선택)

ECharts는 전역 `echarts`로 있습니다. 협력사별 미결 금액을 막대로 그려 봅시다. 테마가 바뀔 때 차트를 다시 그리는 부분도 함께 작성합니다.

```
import { query, color, onTheme } from '/apps/_runtime/og.js'

const rows = await query(`
  MATCH (p:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier) WHERE p.status = 'OPEN'
  RETURN s.name AS supplier, sum(p.amount_usd) AS amount ORDER BY amount DESC`)

const chart = echarts.init(document.getElementById('chart'))
function draw() {
  chart.setOption({
    textStyle: { color: color('dim') },
    xAxis: { type: 'category', data: rows.map(r => r.supplier), axisLine: { lineStyle: { color: color
('line') } } },
    yAxis: { type: 'value', splitLine: { lineStyle: { color: color('line') } } },
    series: [{ type: 'bar', data: rows.map(r => r.amount), itemStyle: { color: color('accent') } }],
  })
}
draw()
onTheme(draw)   // 낮/밤이 바뀌면 다시 칠한다
```

`index.html`에 `<div id="chart" style="height:320px"></div>`를 하나 더해야 합니다.

> **[확인 문제]**
>
> 1. `query('MATCH (s:Supplier) SET s.tier = 1')`을 부르면 어떻게 될까요?
> 2. 사용자가 입력한 협력사 이름으로 발주를 찾을 때, Cypher 문자열에 이름을 이어 붙이면 왜 안 될까요?
> 3. 색을 `#14b8a6` 처럼 직접 적으면 무엇이 문제일까요?
>
> **▼ 정답 보기**
>
> 1. 쓰기 절이 있어 서버가 거절하고, `query()`가 `Error`를 던집니다. 값을 바꾸는 일은 액션으로만 합니다.
> 2. 이름에 따옴표(예: `O'Neil`)가 들어가면 문장이 깨지고, 악의적인 입력이 질의를 바꿀 수 있습니다(주입). `$name` 매개변수로 넘기면 값으로만 다뤄집니다.

> 3. 테마가 바뀌어도 그 색은 그대로라 밤 테마에서 안 보이거나 어색해집니다. `--og-*` 변수나 `color()`를 쓰세요.

# 앱에 액션 버튼 달기

*분류: 애플리케이션 · 20장 · 원문: https://www.uengine.org/docs/ontological-studio/app-actions.html*

> 온톨로지에 행위(액션)가 있으면, 대시보드 위젯과 코드 앱에 그 행위를 실행하는 버튼이 생깁니다. 버튼으로 실행할 때도 채팅과 마찬가지로 계획 → 승인 → 실행 → 취소 절차를 거칩니다. 골든 퀘스천이 앱이 **무엇을 보여 줄지**를, 골든 커맨드가 앱이 **무엇을 할 수 있을지**를 정합니다.

▶ 영상: 앱에 액션 버튼 달기 — 영상 3:52

> **[참고] 0.1.1의 새 기능**
>
> 이 장의 기능(위젯의 액션 버튼, SDK의 `actions()` · `act()`)은 Ontological Studio 0.1.1부터 들어 있습니다. 0.1.0을 쓰고 있다면 릴리스 페이지에서 새로 받으세요. 화면은 시연 영상에서 가져왔습니다(밤 테마).

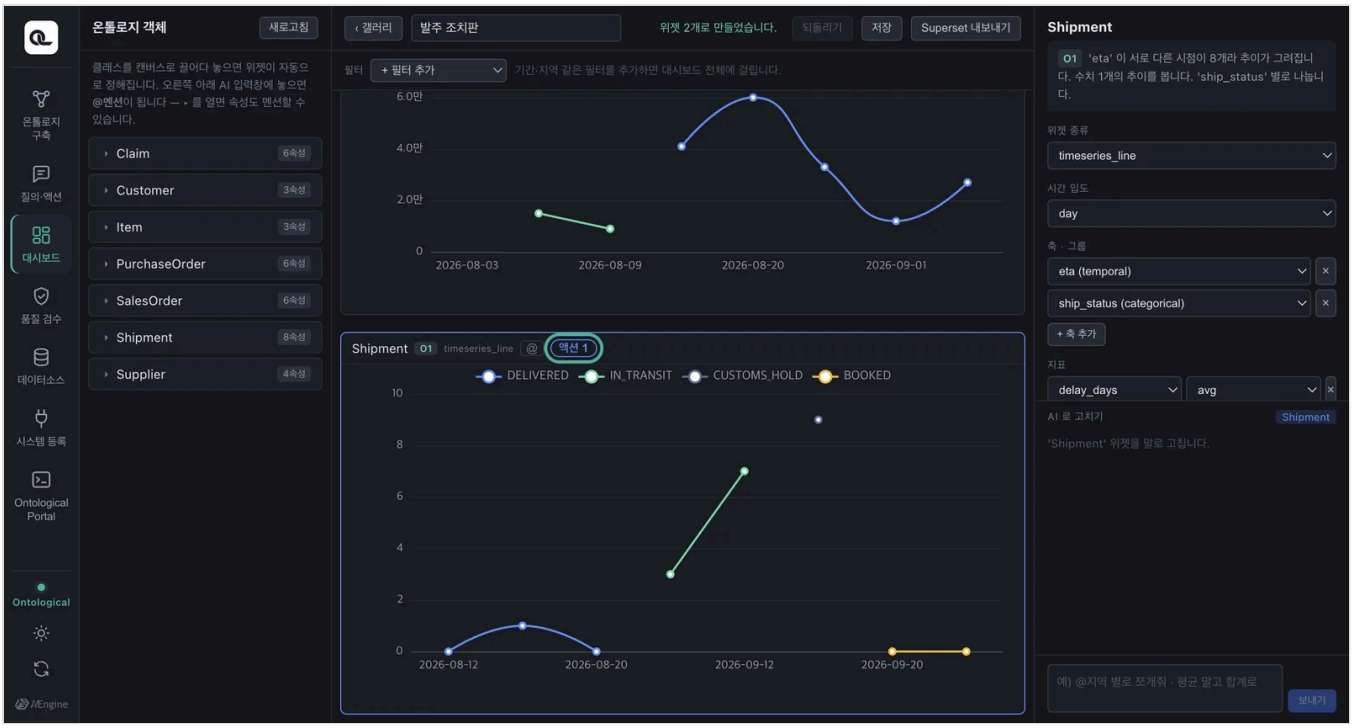
## 왜 앱에 버튼이 필요한가

대시보드에서 늦은 선적을 발견했다면, 다음 할 일은 "그 선적을 긴급 처리"하는 것입니다. 질의·액션 탭으로 옮겨 가서 다시 말로 지시하는 대신, **보고 있는 화면에서 바로** 할 수 있어야 합니다. 그렇다고 앱이 업무 시스템을 마음대로 부르면 안 됩니다. 그래서 버튼은 채팅의 계획 카드와 **같은 길**(액션 실행기)을 씁니다.

| 지켜지는 것 | 어떻게 |
|---|---|
| 앱은 업무 시스템을 직접 부르지 않는다 | 버튼은 액션 실행기의 계획·승인·실행만 부릅니다 |
| 정책 | 대상마다 판정, 거부 사유 표시 |
| 사람의 승인 | 승인·실행·취소는 사람이 창에서 누를 때만 일어납니다 |
| 되돌리기 | 같은 창에서 취소(역순 보상) |

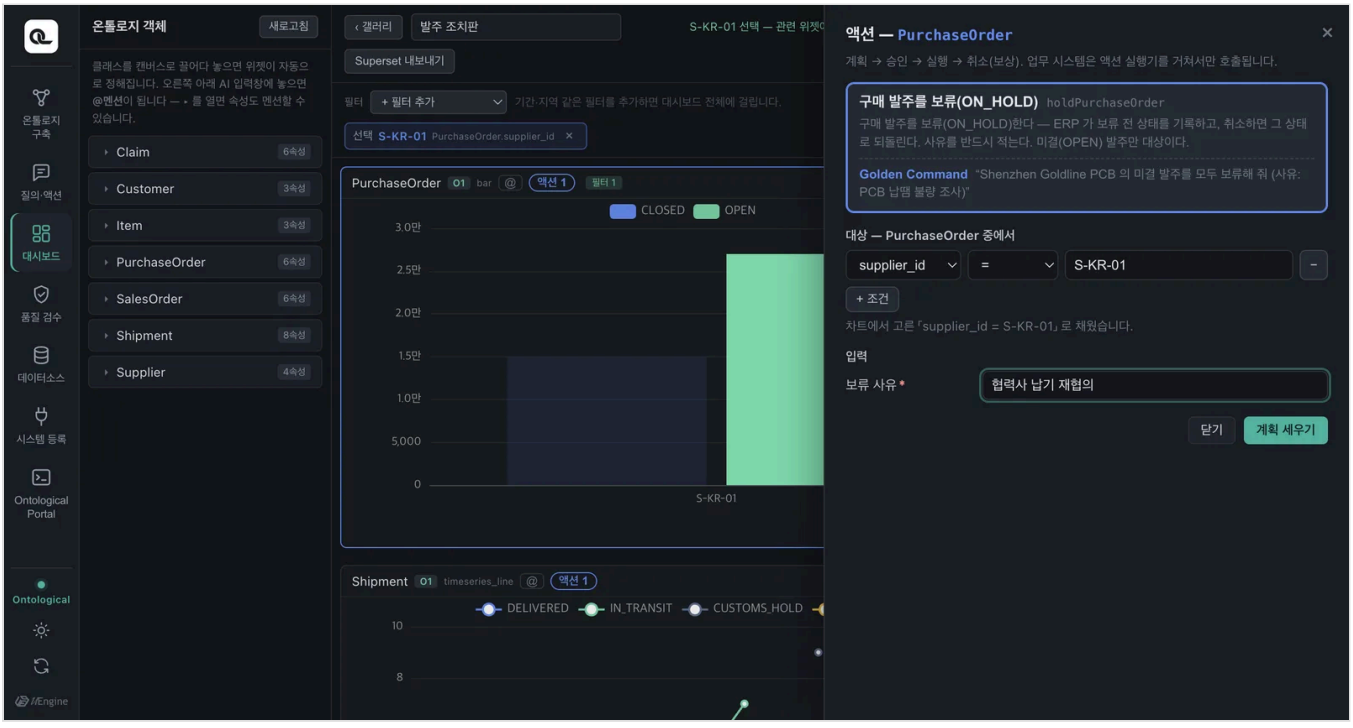
## 대시보드의 액션 버튼

위젯이 보여 주는 클래스에 발행된 액션이 있으면, 위젯 상단에 **「액션 N」** 버튼이 붙습니다. 이 화면을 쓰는 사람의 권한으로 쓸 수 있는 액션만 보입니다.




*위젯 상단에 액션 버튼이 붙은 대시보드*

1. 발주 위젯을 협력사별 금액 막대로 두되, **상태(status)를 계열로 나눠** 둡니다(AI에게 "상태별로 색을 나눠 줘"라고 하면 됩니다. 17장의 합계 막대 그대로라면 OPEN → ON_HOLD 변화가 막대에 보이지 않습니다). 협력사 `S-KR-01`의 막대를 누릅니다. 이 값이 대시보드의 **선택**이 됩니다.
2. 「액션」 버튼을 누르면 오른쪽에 서랍이 열립니다. **발주 보류** 액션과 그것을 만든 **골든 커맨드**가 함께 보이고, 대상 조건은 방금 고른 협력사로 채워져 있습니다. 보류 사유만 적습니다.
3. 「계획」 — 대상마다 정책이 판정합니다. 미결인 PO-008은 허용, 이미 닫힌 PO-003은 "미결 발주만" 정책으로 거부됩니다.
4. 「승인」→「실행」 — ERP가 발주를 보류하고, 서랍 옆의 막대가 곧바로 다시 그려집니다(OPEN → ON_HOLD).
5. 「취소」 — 보상(ERP 보류 해제)이 실행되고 막대가 다시 OPEN으로 돌아옵니다.



*액션 서랍 — 발주 보류 액션, 이 액션을 만든 골든 커맨드, 대상 조건(고른 막대), 입력*

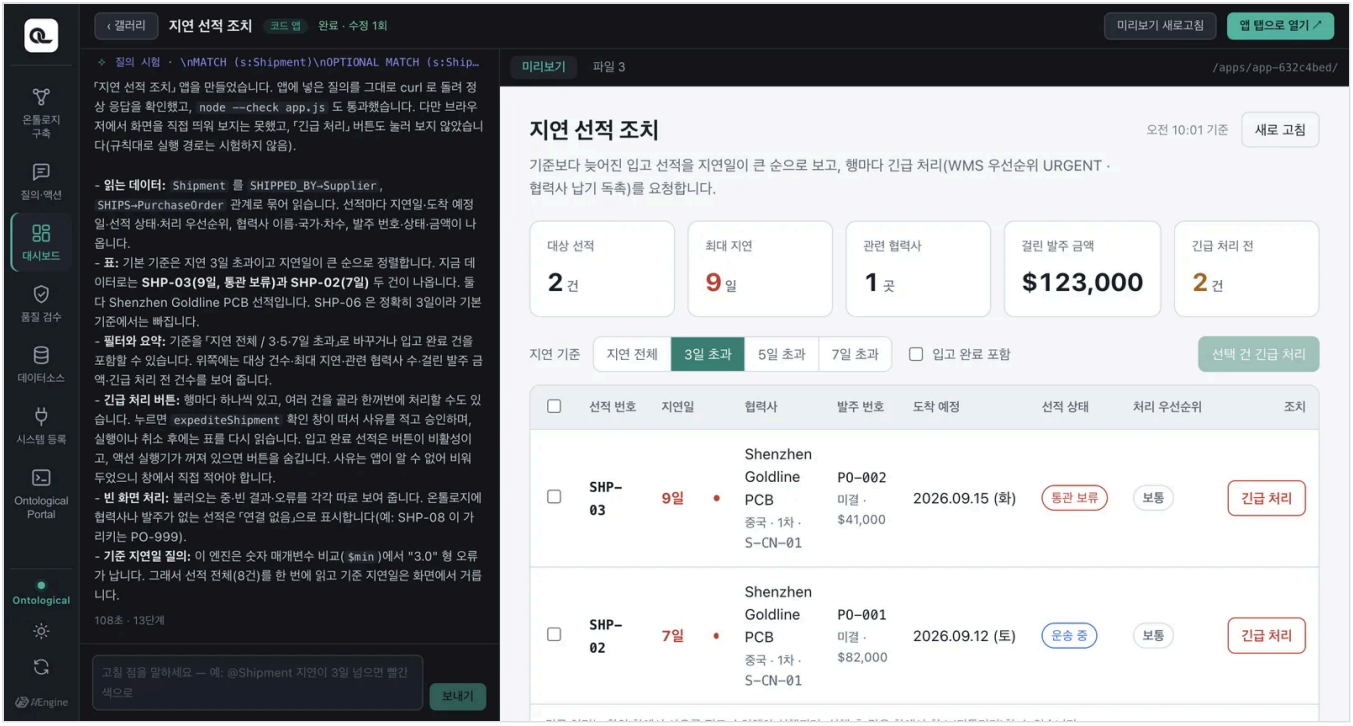
*계획 — PO-008 허용, PO-003 거부(미결 발주만). 사람의 승인을 기다립니다*

*승인·실행 후 — 막대가 ON_HOLD로 다시 그려졌습니다*

## 코드 앱의 액션 버튼

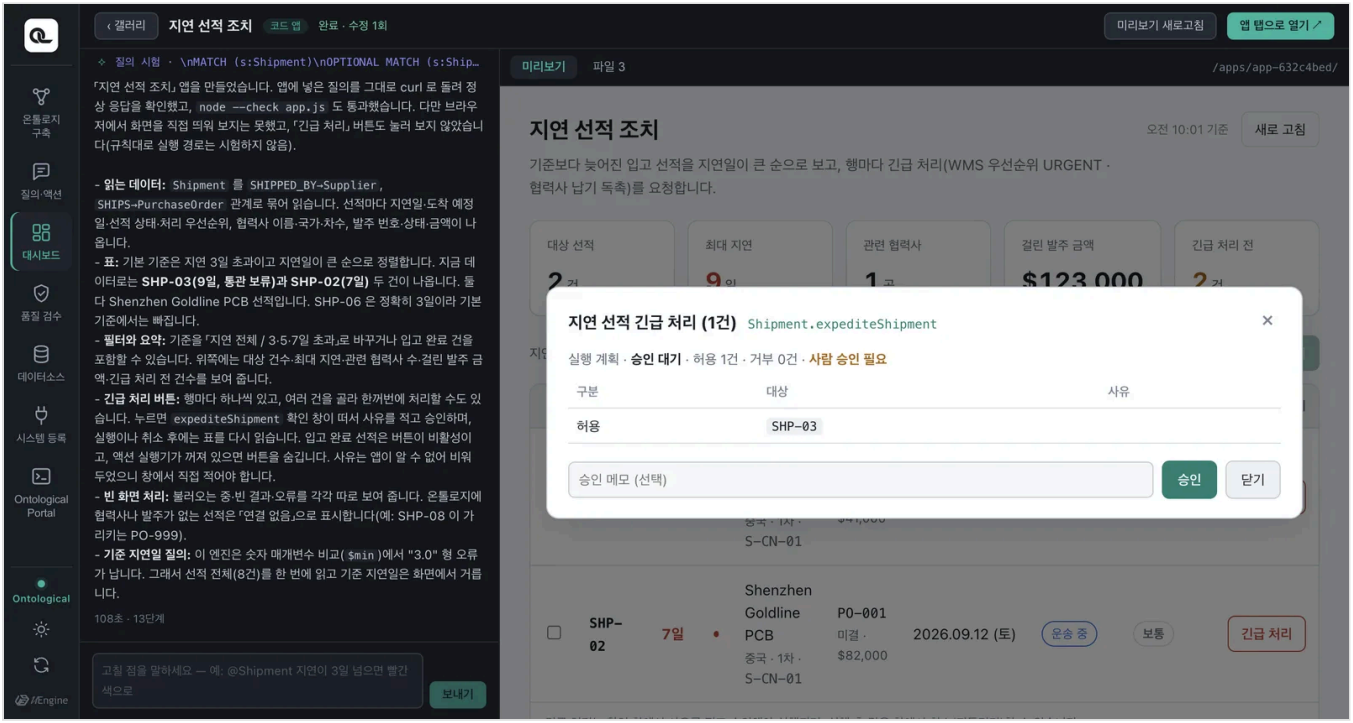
코드 앱을 만들 때, 에이전트의 프롬프트에는 그 온톨로지의 **액션과 골든 커맨드 목록**이 함께 들어갑니다. 그래서 에이전트는 어느 행에 어떤 버튼을 달지 압니다.

```
3일 넘게 늦어진 @Shipment 을 지연일이 큰 순으로 보여 주고, 행마다 긴급 처리 버튼을 달아 줘
```

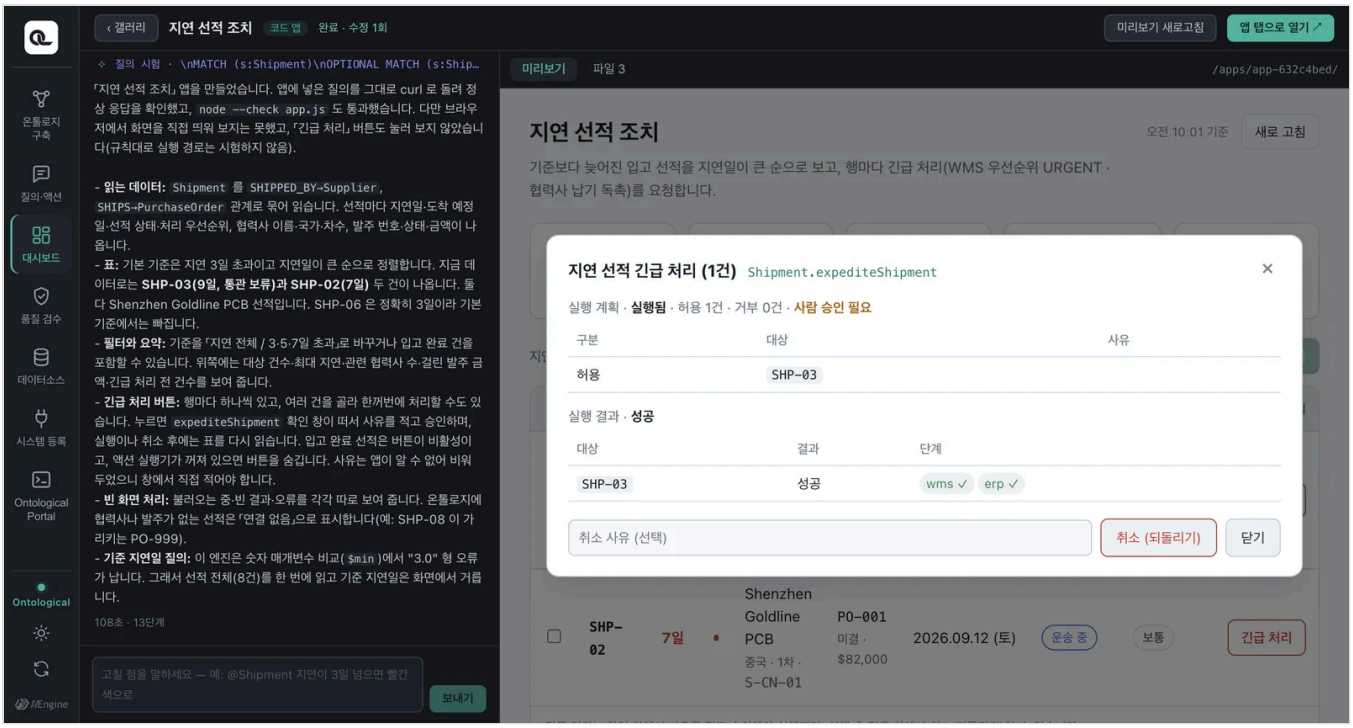



*행마다 「긴급 처리」 버튼이 달린 코드 앱*

버튼을 누르면 **SDK가 띄우는 확인 창**이 뜹니다. 앱 코드가 그린 창이 아니라 SDK의 창이라 어떤 앱에서도 같은 모습이고, 앱이 흉내 낼 수 없습니다. 골든 커맨드, 대상, 사유를 확인하고 계획 → 승인 → 실행합니다.




*SDK 확인 창 — 액션, 대상, 정책 판정, 승인 대기*




*실행 결과 — WMS 우선순위 변경과 ERP 납기 독촉 두 단계가 차례로 성공. 같은 창에서 취소할 수 있습니다*

창을 닫으면 앱이 데이터를 다시 읽어, 그 선적의 우선순위가 “긴급”으로 바뀐 것이 보입니다.

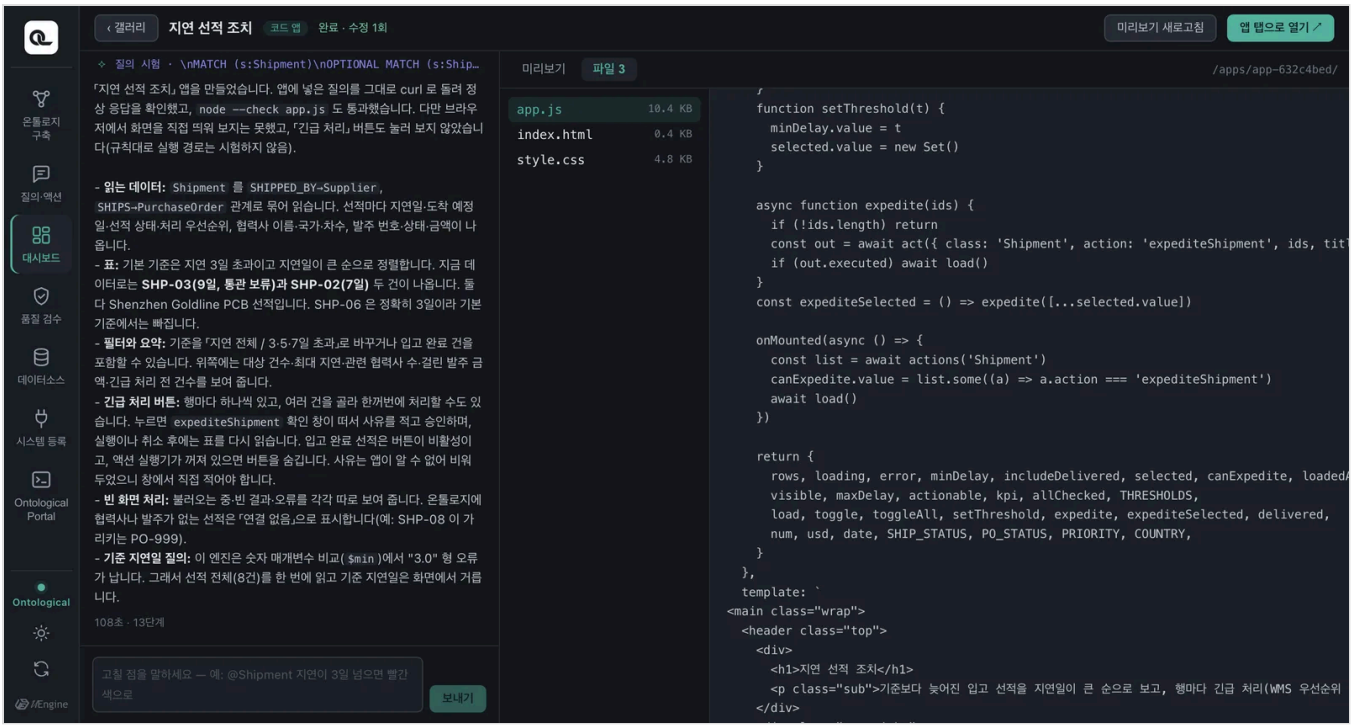
## 코드로 보면

버튼에서 SDK의 `act()` 함수를 호출합니다.

```
import { query, actions, act } from '/apps/_runtime/og.js'

// 이 클래스에 쓸 수 있는 액션이 있을 때만 버튼을 그린다 (실행기가 꺼져 있으면 빈 배열)
const canExpedite = (await actions('Shipment')).some(a => a.action === 'expediteShipment')

async function expedite(row) {
  const out = await act({
    class: 'Shipment',
    action: 'expediteShipment',
    ids: [row.shipment_no],                  // 액션의 대상 키 값 (여러 개를 한 번에 넘겨도 됨)
    params: { reason: '생산 차질 우려' },     // 모르는 필수 입력은 빼 두면 창이 입력란을 보여 준다
  })
  if (out.executed) await load()             // 실행·취소했으면 데이터를 다시 읽는다
}
```




*에이전트가 쓴 코드 — 버튼이 act() 한 줄로 이어져 있습니다*

| act() 인자 | 뜻 |
|---|---|
| `class`, `action` | 실행할 액션 |
| `ids` | 대상 키 값의 배열 (예: 선적 번호) |
| `where` | `ids` 대신 조건으로 대상 고르기: `[{prop: 'delay_days', op: '>', value: 3}]` |
| `params` | 입력값. 필수 입력을 빼면 창이 입력란을 보여 줍니다 |
| 반환값 | 창이 닫히면 `{ executed, cancelled, plan, run }` |

> **[주의] 에이전트는 실행 경로를 시험하지 않습니다**
>
> 코딩 에이전트는 질의는 직접 시험하지만, 계획·승인·실행은 **시험하지 않습니다** — 실제 업무 시스템이 바뀌기 때문입니다. 액션 목록만 확인하고 버튼을 답니다. 승인 요청은 브라우저 화면에서 온 것만 받고, 에이전트의 터미널 호출은 거절됩니다.

## 정리 — 질문과 지시가 앱이 되기까지

| 구축할 때 적은 것 | 만들어진 것 | 앱에서 |
|---|---|---|
| 골든 퀘스천 | 클래스·관계·속성 | 대시보드 위젯, 코드 앱의 질의 |
| 골든 커맨드 | 액션(단계·보상·정책) | 위젯의 액션 버튼, 코드 앱의 `act()` 버튼 |

> **[확인 문제] 실습 과제**
>
> 1. 「공급망 현황판」 대시보드의 발주 위젯에서 액션 버튼으로 한 협력사의 미결 발주를 보류하고, 취소해 보세요.

2. 코드 앱으로 “PCB 납땜 불량 클레임을 낸 고객 목록과, 행마다 조치 안내 보내기 버튼”을 만들어 보세요. 어떤 액션이 쓰이나요?

**▼ 정답 보기**

1. `Customer.notifyCustomer` (골든 커맨드 “PCB 납땜 불량 클레임을 낸 고객에게 조치 안내를 보내 줘”). 필수 입력이 제목과 본문 두 개라, 앱이 모르면 확인 창이 입력란을 보여 줍니다. 보상은 CRM의 안내 철회입니다.

# 종합 과제 — 품질 사고 대응 앱 만들기

*분류: 마무리 · 21장 · 원문: https://www.uengine.org/docs/ontological-studio/project.html*

> 지금까지 배운 내용을 종합합니다. 품질 사고를 주제로 원천 데이터를 연결하고, 온톨로지를 구축해 질의와 조치를 수행한 뒤 그 과정을 하나의 앱으로 만듭니다. 팀 프로젝트(2~3명)로 2~3시간 분량입니다.

▶ 영상: 종합 과제 — 품질 사고 대응 앱 만들기 — 영상 3:45

## 상황

월요일 아침, 도쿄의 고객 Tokyo Devices가 "PCB 납땜 불량" 클레임(T-3)을 냈습니다. 품질팀장이 묻습니다.

1. 이 불량 PCB는 어느 협력사에서 왔고, 그 협력사의 **아직 들어오지 않은 물량**은 얼마나 되나?
2. 같은 PCB를 쓰는 **다른 고객의 미결 주문**은?
3. 조사가 끝날 때까지 그 협력사의 미결 발주를 **보류**하고, 영향받는 고객에게 **안내**를 보내라.
4. 이 모든 것을 매일 보는 **한 화면**으로 만들어라.

## 과제

### 1단계 — 데이터 잇기 (5~6장)

- `erp.po_line`을 `PoLine` 가상 노드로 노출하고 `IN_ORDER`, `OF_ITEM` 관계를 만듭니다(6장에서 했다면 그대로 씁니다).
- **산출물**: `SHOW VIRTUAL RELATIONSHIPS` 결과 캡처

### 2단계 — Cypher로 답하기 (7~9장)

포털(그래프 `sample`)에서 질문 1과 2에 답할 수 있는 Cypher를 작성합니다.

- 질문 1: 클레임 → 판매 주문 → 품목 ← 발주 품목 → 발주 → 협력사, 그리고 그 발주를 싣고 오는 선적 중 `DELIVERED`가 아닌 것.
- 질문 2: 같은 품목의 판매 주문 중 `status = 'OPEN'`인 것과 그 고객.
- `EXPLAIN FABRIC`으로 원천 몇 곳을 건드리는지 확인합니다.
- **산출물**: Cypher 두 개, 결과 테이블, EXPLAIN FABRIC 결과

### 3단계 — 온톨로지 넓히기 (11~13장)

예제 온톨로지에는 발주 품목이 없습니다. 「공급망 운영 (예제)」에 **추가 인제스천**을 수행해 발주 품목을 추가합니다.

- 소스: 데이터소스 `erp`
- 골든 퀘스천 추가: "PCB 납땜 불량 클레임의 품목을 공급한 협력사와, 그 협력사의 미입고 물량은?"
- 구축 후 질의·액션 탭에서 이 질문을 던져 2단계의 Cypher와 같은 답이 나오는지 검산합니다.

- **산출물**: 구축 보고서 캡처, 질의 답과 2단계 결과의 비교

### 4단계 — 조치하기 (14~16장)

- 질의·액션 탭에서 “그 협력사의 미결 발주를 모두 보류해 줘 (사유: PCB 납땜 불량 조사)”와 “영향받는 고객에게 조치 안내를 보내 줘”를 지시합니다.
- 계획 카드의 허용·거부와 사유를 기록하고, 승인·실행 후 데이터소스 탭에서 `erp.purchase_order`, `crm.notice`가 바뀐 것을 확인합니다.
- 마지막에 **모두 취소**해 원상태로 돌리고, 보상 이력의 순서를 설명합니다.
- **산출물**: 계획 카드, 실행 결과, 원천 확인, 보상 이력 캡처와 설명

### 5단계 — 앱 만들기 (17~20장)

- 코드 앱 「품질 사고 대응」: 위에 클레임 요약, 가운데 영향 협력사와 미입고 물량, 아래에 영향 고객의 미결 주문.
- 말로 두 번 이상 고치고, `app.js`의 질의 하나를 2단계 Cypher와 비교합니다.
- (0.1.1부터) 협력사 행에 「발주 보류」, 고객 행에 「안내 보내기」 버튼.
- **산출물**: 앱 화면, `app.js`에서 질의 부분 발췌

## 평가 기준 (예시)

| 항목 | 기준 | 배점 |
|---|---|---|
| 데이터 연결 | 가상 노드·관계가 맞는 키로 이어졌다 | 15 |
| Cypher | 질문 1·2에 맞는 답, 라벨·방향이 정확, EXPLAIN 해석 | 25 |
| 구축 | 골든 퀘스천이 구체적이고, 질의 답이 Cypher 결과와 일치 | 20 |
| 액션 | 정책 거부 사유를 설명, 실행·원천 확인·보상 순서 설명 | 20 |
| 앱 | 실제 데이터, 빈 결과·오류 처리, 고친 과정 | 20 |

## 생각해 볼 거리

- 2단계에서 직접 쓴 Cypher와, 3단계에서 에이전트가 쓴 답은 무엇이 같고 무엇이 달랐나요?
- 4단계의 “안내 보내기”를 정말 운영에 쓴다면, 정책에 어떤 조건을 더해야 할까요? (예: 같은 고객에게 하루 한 번만)
- 원천 DB를 복사하지 않는 방식(데이터 패브릭)의 장단점을, 2단계에서 본 EXPLAIN FABRIC 결과를 근거로 설명해 보세요.

# Cypher·패브릭 치트시트

*분류: 부록 · 22장 · 원문: https://www.uengine.org/docs/ontological-studio/cheatsheet.html*

> 이 튜토리얼에서 사용한 문장을 한곳에 모았습니다. 포털에서 실습할 때 참고하세요. 예제는 `sample` 그래프 기준입니다.

## 읽기

| 하고 싶은 일 | Cypher |
|---|---|
| 노드 찾기 | `MATCH (s:Supplier) RETURN s` |
| 속성 고르기 | `MATCH (s:Supplier) RETURN s.name, s.country` |
| 키로 노드 찾기 | `MATCH (s:Supplier {supplier_id: 'S-CN-01'}) RETURN s` |
| 거르기 | `WHERE s.country = 'China' AND s.tier = 1` |
| 목록 포함 | `WHERE sh.ship_status IN ['IN_TRANSIT', 'CUSTOMS_HOLD']` |
| 문자열 | `WHERE s.name CONTAINS 'Gold'` · `STARTS WITH` · `ENDS WITH` |
| 날짜 | `WHERE c.opened_date >= '2026-08-29'` |
| 정렬·자르기 | `ORDER BY po.amount_usd DESC SKIP 2 LIMIT 3` |
| 이름 바꾸기 | `RETURN po.po_id AS 발주` |
| 중복 제거 | `RETURN DISTINCT s.country` |
| 조건 값 | `CASE WHEN sh.delay_days > 3 THEN '지연' ELSE '정상' END` |
| 목록 펼치기 | `UNWIND ['SHP-02','SHP-03'] AS no MATCH (sh:Shipment {shipment_no: no}) RETURN sh` |

## 관계

| 하고 싶은 일 | Cypher |
|---|---|
| 관계 한 단계 조회 | `MATCH (po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier)` |
| 반대 방향으로 조회 | `MATCH (s:Supplier)<-[:ORDERED_FROM]-(po:PurchaseOrder)` |
| 여러 관계 조회 | `(sh:Shipment)-[:SHIPS]->(po:PurchaseOrder)-[:ORDERED_FROM]->(s:Supplier)` |
| 관계도 돌려받기 | `MATCH (a:Shipment)-[r:SHIPS]->(b:PurchaseOrder) RETURN a, r, b` |
| 갈래 둘 | `MATCH (i:Item)<-[:FOR_ITEM]-(so:SalesOrder), (i:Item)<-[:OF_ITEM]-(l:PoLine)` |
| 연결된 데이터가 없어도 포함 | `OPTIONAL MATCH (sh:Shipment)-[:SHIPS]->(po:PurchaseOrder)` |

| 하고 싶은 일 | Cypher |
| --- | --- |
| 연결된 데이터가 없는 항목만 | `… OPTIONAL MATCH … WITH sh, po WHERE po IS NULL RETURN sh` |

## 집계

| 하고 싶은 일 | Cypher |
| --- | --- |
| 세기 | `RETURN count(s)` · `count(*)` |
| 묶어 세기 | `RETURN po.status, count(*) AS n, sum(po.amount_usd) AS total` |
| 목록으로 모으기 | `RETURN s.name, collect(po.po_id) AS orders` |
| 집계로 다시 거르기 | `WITH s, count(po) AS n WHERE n >= 2 RETURN s.name, n` |
| 소수 합계로 거르기 | `WITH s, sum(toFloat(po.amount_usd)) AS total WHERE total > 50000` |
| 목록 길이 | `size(collect(po.po_id))` |

## 쓰기 (저장 그래프에서만)

| 하고 싶은 일 | Cypher |
| --- | --- |
| 그래프 만들기 | `CREATE GRAPH tut` |
| 노드 | `CREATE (:Employee {emp_no: 'E1', name: '김하늘'})` |
| 관계 | `MATCH (a:Employee {emp_no:'E1'}), (b:Employee {emp_no:'E2'}) CREATE (a)-[:MENTORS]->(b)` |
| 값 바꾸기 | `MATCH (e:Employee {emp_no:'E1'}) SET e.salary = 4500` |
| 없으면 만들기 | `MERGE (d:Dept {name:'구매'})` |
| 지우기 | `MATCH (e:Employee {emp_no:'E2'}) DETACH DELETE e` |

## 데이터 패브릭

| 하고 싶은 일 | 문장 |
| --- | --- |
| 데이터소스 목록 | `SHOW DATASOURCES` |
| 연결 | `CREATE DATASOURCE mydb WITH ENGINE = 'postgres', PARAMETERS = {host: '…', port: 5432, database: '…', schema: 'public', user: '…', password: '…'}` |
| 테이블 | `SHOW TABLES FROM erp` |
| 컬럼 | `SHOW COLUMNS FROM erp.po_line` |

| 하고 싶은 일 | 문장 |
|---|---|
| 다시 읽기 · 해제 | `REFRESH DATASOURCE erp` · `DROP DATASOURCE erp` |
| 가상 노드 | `CREATE VIRTUAL NODE PoLine IN sample FROM erp.po_line KEY po_line_id` |
| 속성 이름 바꾸기 | `… KEY po_line_id PROPERTIES (po_line_id, po_id, item_id, qty AS quantity)` |
| 가상 관계 | `CREATE VIRTUAL RELATIONSHIP IN_ORDER IN sample FROM PoLine(po_id) TO PurchaseOrder(po_id)` |
| 목록 | `SHOW VIRTUAL NODES` · `SHOW VIRTUAL RELATIONSHIPS` |
| 지우기 | `DROP VIRTUAL RELATIONSHIP IN_ORDER IN sample` · `DROP VIRTUAL NODE PoLine IN sample` |
| 실행 계획 | `EXPLAIN FABRIC MATCH …` |

## 시간 여행

| 하고 싶은 일 | 방법 |
|---|---|
| 그때의 사실 | 포털 ⏱ 막대의 valid at |
| 그때 알던 것 | 포털 ⏱ 막대의 as known at |
| 시간 버전 켜기 (SQL) | `SELECT og_enable_temporal('hr', 'Employee');` |
| 변경 기록 (SQL) | `BEGIN; SET LOCAL og.valid_from = '2024-01-01'; SELECT og_cypher('hr', $$…$$); COMMIT;` |
| 정정 기록 (SQL) | `SET LOCAL og.valid_from = '-infinity'` |

## 함수

`count` `sum` `avg` `min` `max` `collect` · `toUpper` `toLower` `trim` `substring` `replace` `split` `toString` · `abs` `ceil` `floor` `round` `sqrt` `rand` `toInteger` `toFloat` · `id` `elementId` `labels` `type` `length` `size` `keys` `exists` · `coalesce` `timestamp` `datetime` · `vector.similarity` `vector.distance` `genai.vector.encode`

## 코드 앱 SDK

```
import { query, schema, theme, onTheme, color, actions, act } from '/apps/_runtime/og.js'
const rows = await query('MATCH (s:Supplier) WHERE s.country = $c RETURN s.name AS name', { c: 'China' })
const out = await act({ class: 'Shipment', action: 'expediteShipment', ids: ['SHP-03'], params: { reason: '…' } })  // 0.1.1부터
```

# 자주 묻는 질문과 문제 해결

*분류: 부록 · 23장 · 원문: https://www.uengine.org/docs/ontological-studio/faq.html*

> 실습 중 자주 발생하는 문제의 해결 방법과 주요 용어를 정리했습니다.

## 설치와 실행

**macOS에서 "악성 코드가 없음을 확인할 수 없습니다" / "확인되지 않은 개발자"라고 나옵니다.** : 공증되지 않은 배포판이라 그렇습니다. 창을 닫고 시스템 설정 › 개인정보 보호 및 보안 › 「그래도 열기」를 누르세요. macOS 14 이하라면 앱을 오른쪽 클릭 › 열기도 됩니다(1장).

**macOS에서 "손상되었기 때문에 열 수 없습니다. 해당 항목을 휴지통으로 이동해야 합니다"라고 나옵니다.** : 앱 서명이 덜 된 채 올라갔던 0.1.1 Apple Silicon용 초기 배포본입니다. 휴지통으로 옮기지 말고, 터미널에서 `xattr -dr com.apple.quarantine "/Applications/Ontological Studio.app"` 을 실행한 뒤 다시 여세요(1장).

**Intel Mac에서 앱이 실행되지 않습니다.** : Intel용 설치 파일( `Ontological-Studio-mac-x64.dmg` )을 받았는지 확인하세요. Apple Silicon용( `…-mac-arm64.dmg` )은 Intel Mac에서 열리지 않습니다. Intel용은 0.1.1부터 있으니, 0.1.0을 받았다면 릴리스 페이지에서 새로 받으세요(1장).

**Windows에서 SmartScreen이 막습니다.** : 「추가 정보 › 실행」을 누릅니다.

**홈에 「에이전트 설정 필요」가 보입니다.** : Claude Code 또는 Codex CLI를 설치하고 터미널에서 한 번 로그인하세요. 그다음 메뉴 **런타임 › 에이전트**에서 고르거나 앱을 다시 켭니다. CLI 없이 쓰려면 「설정」에서 `studio.env` 에 `OPENAI_API_KEY` 를 넣습니다.

**처음 상태로 되돌리고 싶습니다.** : 앱을 끄고 데이터 폴더(macOS `~/Library/Application Support/Ontological Studio/` )를 지운 뒤 다시 켭니다. 예제 DB와 예제 온톨로지가 새로 만들어집니다. 내가 만든 온톨로지·앱도 함께 지워지니 주의하세요.

**앱을 끄는 데 몇 초 걸립니다.** : 정상입니다. 데이터베이스를 안전하게 닫고 서비스들을 차례로 내립니다.

## 데이터소스와 Cypher

**포털에서 예제 노드가 안 보입니다.** : 왼쪽 위 그래프 선택이 `sample` 인지 확인하세요. 예제의 가상 노드는 `sample` 그래프에 있습니다. 구축한 온톨로지는 `ontology` 그래프에 있습니다(4장).

**`unknown function 'date'` 오류가 납니다.** : `date()` 함수는 아직 없습니다. 날짜 컬럼은 `'2026-08-29'` 같은 문자열과 바로 비교하세요.

**관계 패턴을 썼는데 결과가 비어 있습니다.** : 관계 이름과 양쪽 노드의 라벨을 모두 적었는지 확인하세요. 가상 관계는 `-[r]->` 처럼 이름 없이 찾으면 결과가 나오지 않습니다. 방향( `->` , `<-` )도 확인하세요.

**`invalid input syntax for type bigint` 오류가 납니다.** : 소수 타입 컬럼의 합계를 `WITH … WHERE`로 다시 거를 때 나는 알려진 문제입니다. `sum(toFloat(x))`로 바꿔 쓰세요(8장).

**`EXISTS { … }`가 안 됩니다.** : 하위 질의는 아직 없습니다. `OPTIONAL MATCH … WITH … WHERE x IS NULL`로 씁니다.

**가상 노드에 값을 넣으려고 하니 `read-only`라고 합니다.** : 가상 노드는 원천 테이블을 비춘 것이라 읽기 전용입니다. 값은 원천 시스템에서, Ontological Studio에서는 액션으로 바꿉니다(6장).

**가상 노드를 지울 수 없습니다.** : 그 노드를 잇는 가상 관계부터 지우세요. 오류 메시지에 남은 관계 이름이 나옵니다.

**원천 DB 연결이 실패합니다.** : 호스트·포트에 이 PC에서 접속할 수 있는지, 계정에 그 스키마를 읽을 권한이 있는지 확인하세요. 데이터소스 이름은 영문 소문자·숫자·밑줄만 됩니다. Windows판은 지금 PostgreSQL만 됩니다.

## 구축과 질의

**구축이 끝났는데 질의에서 "발행된 온톨로지가 없다"고 합니다.** : 발행하지 않았기 때문입니다. 구축 대화창에 "발행해 줘"라고 지시하세요(12장).

**같은 문서로 다시 구축했더니 클래스 이름이 다릅니다.** : 에이전트(LLM)가 설계하므로 이름은 조금씩 다를 수 있습니다. 골든 퀘스천에 같은 답을 내는지로 판단하세요. 원하는 이름이 있으면 구축 의도에 적어 두거나, 「정보」 탭에서 직접 고치세요.

**에이전트의 답이 틀린 것 같습니다.** : 답에 적힌 근거를 보고, 같은 질문을 포털에서 Cypher로 검산하세요. 온톨로지에 필요한 개념이나 관계가 없다면 골든 퀘스천에 그 질문을 넣고 다시 구축합니다.

**예제 Supplier와 내가 만든 Supplier가 섞입니다.** : 클래스 이름은 온톨로지(스키마)를 넘어 전역입니다. 같은 이름의 클래스를 다른 온톨로지에서 만들면 인스턴스가 한 라벨에 섞입니다. 실습용 온톨로지에서는 `QualitySupplier`처럼 다른 이름을 쓰거나 구축 의도에 이름을 지정하세요.

### 액션

**계획을 세웠는데 대상이 0건입니다.** : 조건에 맞는 대상이 없거나, 모두 정책으로 거부된 것입니다. 계획 카드의 거부 사유를 보세요. 대상 0건인 계획은 실행되지 않습니다.

**"실행해"라고 했는데 실행되지 않습니다.** : 계획 카드에서 먼저 「승인」을 눌렀는지 확인하세요. 에이전트는 승인할 수 없습니다. 계획이 만료됐다면 다시 지시해 계획을 새로 세웁니다.

**취소했는데 일부만 되돌아갔습니다.** : 보상 이력에서 실패한 단계를 보세요(⚠ 보상 실패). 업무 시스템이 그 사이 다른 이유로 상태를 바꿨다면 보상이 거절될 수 있습니다. 그런 건은 담당자가 업무 시스템에서 직접 확인해야 합니다.

## 용어

| 용어 | 뜻 |
|---|---|
| 온톨로지 | 업무의 개념(클래스)과 관계에 이름을 붙인 지도. 그 위에 인스턴스가 올라갑니다 |
| 클래스 · 라벨 | 노드의 종류 ( `Supplier` ) |
| 인스턴스 · 노드 | 클래스에 속한 개별 데이터 ( `S-CN-01` ) |

| 용어 | 뜻 |
|---|---|
| 관계 · 엣지 | 두 노드의 연결, 이름과 방향이 있음 ( `ORDERED_FROM` ) |
| 그래프 | Ontological DB 안의 독립된 데이터 공간 ( `sample` , `ontology` ) |
| 데이터 패브릭 | 원천 데이터를 복사하지 않고 조회 시 읽어 연결하는 방식 |
| 가상 노드 · 가상 관계 | 원천 테이블과 조인 규칙을 그래프의 노드·관계로 정의한 것 (읽기 전용) |
| Cypher | 그래프 질의 언어. 패턴을 그림처럼 씁니다 |
| 골든 퀘스천 | 온톨로지가 반드시 답해야 할 질문 — 설계의 출발점, 완성의 기준 |
| 골든 커맨드 | 온톨로지로 반드시 할 수 있어야 할 지시 — 액션 설계의 출발점 |
| 발행 | 구축한 온톨로지를 질의·액션·앱에서 쓰도록 내보내는 일 (사람이 결정) |
| 액션 | 클래스에 붙은 “할 수 있는 일” — 단계·보상·정책 |
| 보상 | 실행한 단계를 원래 상태로 되돌리는 호출 |
| 사가 | 여러 시스템에 걸친 실행에서, 실패하면 성공한 단계를 역순으로 보상하는 방식 |
| 정책 | 누가 어떤 대상에 액션을 쓸 수 있는지 정하는 규칙 (기본 거부) |
| 드라이런 | 실제로 실행하지 않고 계획만 세워 보는 것 |
| 바이템포럴 | 유효 시간(언제의 사실)과 시스템 시간(언제 알게 됐나)을 함께 기록하는 방식 |
| MCP | 에이전트가 도구를 부르는 표준 규약. Studio의 도구는 MCP로 에이전트에게 제공됩니다 |